

中图分类号：TP311.5

单位代码：11414

学 号：2019211267



中国石油大学
CHINA UNIVERSITY OF PETROLEUM

硕士学位论文

题 目 基于深度学习的代码漏洞检测方
法研究

学科专业 计算机科学与技术

研究方向 程序理解

硕 士 生 吕宁宁

指导教师 朱雪峰

二〇二三年五月

摘 要

随着软件越来越庞大，程序复杂度越来越高，程序漏洞也是越来越多。如何在众多复杂程序中准确的识别漏洞，保障软件开发的安全，是信息安全领域面临的重要难题。先前的研究表明，现有的基于深度学习的方法可以显著提高模型漏洞检测的准确性，并实现了自动学习代码特征，弥补了机器学习中人工提取特征信息的不足。然而，代码表示形式和深度学习模型的差异使得各种方法仍然存在一定的局限性。在实践中，它们的假阳性率（FPR）和假阴性率（FNR）仍然很高。

代码漏洞检测首先需要进行特征提取，传统的特征提取方法存在着提取片面特征以及特征不准确的问题，提取的特征可能包含过多的冗余信息，进而影响网络训练，降低漏洞检测准确率。其次，代码之间包含语义信息，在特征提取时没有考虑到这方面的因素也会对漏洞检测结果造成影响。最后，深度学习模型在代码漏洞方面的网络结构较单一，缺乏创新性。为更准确的检测代码漏洞，提升系统性能，优化网络结构，本文提出了一种基于注意力机制的代码漏洞检测方法。

为解决现有基于深度学习的漏洞检测方法的局限性，本文主要研究内容如下。

（1）针对代码漏洞检测准确率低、网络结构单一问题，对深度神经网络模型进行优化，提出基于注意力机制的代码漏洞检测模型。该模型结合 Transformer 模型和 BiLSTM 网络的优势，使用 Transformer 编码器进行全局代码语义特征提取，得到准确、完善的漏洞特征；使用 BiLSTM 网络进行代码特征学习与分类检测，可以学习漏洞特征上下文相关信息，通过两个模型结合，加深网络结构，提高漏洞特征提取能力，进而提高模型检测代码漏洞的准确率。

（2）针对代码语义信息影响漏洞检测模型准确率和时间开销问题，对 Transformer 编码器中位置编码模块采用基于相对距离的位置编码方式，提出基于相对距离位置编码方式的注意力模型。该编码方式将词位置的相对距离加入到注意力机制模型中，相对位置距离越小，代码关联度越大，此处的编码越密集，注意力权重越大。该编码方式模糊远距离代码带来的影响，加重对敏感函数附近的代码信息关注度，同时将程序间语义关系表示出来，增强了模型对漏洞特征提取的能力，进而提高模型检测漏洞的准确率并降低模型训练时间。

（3）设计并实现基于注意力机制的深度学习漏洞检测模型。在 NVD 和 SARD 收集的 C/C++ 数据上进行评估该模型。首先验证了基于注意机制的代码漏洞检测模型的有效性，实验结果表明该模型与 VulDeePecker 模型相比，在两个数据集上

准确率分别提高 0.8%和 1.3%，AUC 值分别提高 3.2%和 2.7%，时间开销分别减少 1.69 倍和 1.54 倍；其次，与现有的一些漏洞检测方法进行了对比实验，该模型在评价指标 F1-score 比其它模型要高；最后，验证基于相对距离的位置编码方式对模型性能影响，该编码方式与其他编码方式相比，在准确率、F-score 和 AUC 值上都提高了，并且降低了模型训练时间开销，说明基于相对距离的位置编码方式更适合代码漏洞检测领域。

关键词：漏洞检测；深度学习；特征提取；位置编码

Research on code vulnerability detection method based on deep learning

ABSTRACT

As software becomes more and more massive and programs become more and more complex, program vulnerabilities are also increasing. How to accurately identify vulnerabilities in many complex programs and secure software development is an important challenge in the field of information security. Previous research has shown that existing deep learning-based approaches can significantly improve the accuracy of vulnerability detection and enable automatic learning of code features, compensating for the lack of manual extraction of feature information in machine learning. However, the differences in code representation and deep learning models make the various methods still have some limitations. In practice, their false positive rate (FPR) and false negative rate (FNR) remain high.

Code vulnerability detection first requires feature extraction. The traditional feature extraction method has the problem of extracting one-sided features and inaccurate features, and the extracted features may contain too much redundant information, which affects network training and reduces the accuracy of vulnerability detection. Second, the code contains semantic information between them, and failure to take this aspect into account in feature extraction can also have an impact on the vulnerability detection results. Finally, the network structure of deep learning models in terms of code vulnerabilities is more homogeneous and lacks innovation. In order to better detect code vulnerabilities accurately, improve system performance and optimize network structure, this paper constructs a code vulnerability detection model based on attention mechanism.

To address the limitations of existing deep learning-based vulnerability detection methods, this paper makes the following main contributions.

(1) To address the problems of low accuracy of code vulnerability detection and single network structure, we optimize the deep neural network model and propose an attention mechanism-based code vulnerability detection model. The model combines

the advantages of Transformer model and BiLSTM network, using Transformer encoder for global code semantic feature extraction to get accurate and perfect vulnerability features; using BiLSTM network for code feature learning and classification detection, which can learn vulnerability feature context-related information. By combining the two models, the network structure is deepened, and the vulnerability feature extraction capability is improved, which in turn improves the accuracy of the model in detecting code vulnerabilities.

(2) To address the problem of code semantic information affecting the accuracy and time overhead of the vulnerability detection model, a relative distance-based position encoding approach is used for the position encoding module in the Transformer encoder, and an attention model based on the relative distance position encoding approach is proposed. This encoding method adds the relative distance-based of word positions into the attention mechanism model, the smaller the relative position distance, the greater the code association, the more intensive the encoding here, and the greater the attention weight. The encoding approach blurs the influence brought by distant codes, increases the attention force on the code information near sensitive functions, and at the same time represents the semantic relationship between programs, which enhances the model's ability to extract vulnerability features and thus improves the model's accuracy in detecting vulnerabilities and reduces model training time.

(3) Design and implement a deep learning vulnerability detection model based on the attention mechanism. The model is evaluated on C/C++ data collected by NVD and SARD. Firstly, the effectiveness of the attention mechanism-based code vulnerability detection model is verified, and the experimental results show that the model improves the accuracy by 0.8% and 1.3%, the AUC values by 3.2% and 2.7%, and the time overhead by 1.69 times and 1.54 times, respectively, compared with the VulDeePecker model on the two datasets. Secondly, comparison experiments are conducted with some existing vulnerability detection methods, and the model has higher F1-score in evaluation metrics than other models. Finally, the impact of the relative distance-based location coding approach on the model performance is verified, and the coding approach improves in accuracy, F-score and AUC values and reduces the model training time overhead compared with other coding approaches, indicating that the relative

ABSTRACT

distance-based location coding approach is more suitable for the field of code vulnerability detection.

Key Words: Vulnerability detection; Deep learning; Feature extraction; Position embedding

目 录

第 1 章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	3
1.2.1 基于代码相似度的代码漏洞检测.....	3
1.2.2 基于规则的代码漏洞检测.....	7
1.2.3 基于人工智能的代码漏洞检测.....	8
1.3 本文的研究工作与内容安排	16
第 2 章 相关研究工作	19
2.1 软件漏洞概述.....	19
2.1.1 软件漏洞定义.....	19
2.1.2 典型的软件漏洞.....	19
2.2 典型的深度神经网络.....	20
2.2.1 卷积神经网络.....	20
2.2.2 循环神经网络.....	21
2.2.3 长短期记忆网络	22
2.3 Transformer 结构.....	23
2.4 本章小结.....	27
第 3 章 基于注意力机制的代码漏洞检测算法	28
3.1 系统架构.....	28
3.2 模型构建.....	31
3.2.1 数据处理	31
3.2.2 向量转换.....	35
3.2.3 Transformer 编码器	38
3.2.4 BiLSTM 网络结构	40
3.2.5 基于注意力机制的 BiLSTM 网络	41
3.3 实验结果与分析.....	42
3.3.1 实验环境与数据集.....	42
3.3.2 实验评价指标.....	44
3.3.3 实验结果分析.....	45
3.4 本章小结.....	52
第 4 章 基于位置编码优化的代码漏洞检测算法	54
4.1 模型构建.....	54
4.1.1 基于多注意力机制的 BiLSTM 网络	55
4.1.2 位置编码模型.....	56

4.2 位置编码算法.....	57
4.2.1 One-hot 位置信息编码.....	57
4.2.2 绝对位置信息编码.....	58
4.2.3 相对位置信息编码.....	58
4.2.4 基于相对距离的位置信息编码.....	60
4.3 实验结果及分析.....	62
4.3.1 实验设置.....	62
4.3.2 实验评价指标.....	63
4.3.3 实验结果分析.....	63
4.4 本章小结.....	72
第 5 章 总结和展望	74
5.1 总结.....	74
5.2 展望.....	75
参考文献.....	77

第 1 章 绪 论

1.1 研究背景及意义

随着信息技术的飞速发展，软件在世界各地的经济、军事、社会等各个方面都发挥着重要作用。与此同时，软件中潜在的安全问题正在成为一个新兴的全球性挑战，软件漏洞是安全问题的根源之一。技术高超的黑客可以根据自己的意愿利用软件漏洞做很多有害的事情，例如窃取用户的私人信息、停止关键设备。在当今多用户持续交互的环境中，如果黑客利用某个时间节点发起攻击，其损失和成本都难以预测。根据 Common Vulnerabilities and Exposures (CVE) 组织发布的统计数据^[1]，在 2000 年，软件漏洞被发现的数量不到四千多，然而目前覆盖的漏洞数量几乎接近 20000 个。漏洞数量在过去几年达到顶峰，这隐形地增加了网络用户的危险系数。虽然信息技术的广泛应用，极大地促进了经济社会的发展。但是，一旦信息系统受到攻击，可能会造成巨大的损失，软件漏洞是网络攻击的根本原因。近年来，发布的软件漏洞数量迅速增长。在 US-CERT 的 2022 年度报告中，就这一年的时间，漏洞数据库收集的漏洞达到 27447 个，连续四年创纪录^[2]。因此，尽早发现和修补漏洞至关重要。并且，随着网络信息技术的发展，开源软件因其开放性和灵活性被广泛应用于各行各业，但也带来了潜在的软件安全问题。加上软件数量的大规模增加和复杂性的增加，传统的人工处理这些安全问题的方法效率较低，无法满足当前的网络空间安全需求。因此，开发更多智能技术应用于软件中潜在的安全问题是软件安全领域研究人员的重要研究课题^[3]。

针对软件安全问题，出现一系列的软件安全方案，在软件漏洞检测方面进行多方尝试，并取得一定成效。传统的软件漏洞检测有静态检测方法^[4]和动态检测方法^[5]。静态检测方法^[4]不需要执行程序，可以全面检测代码漏洞，覆盖率较高，漏洞检测准确率较高。但是，没有考虑程序执行环境带来的影响，忽略由硬件设施等带来的漏洞风险。动态检测方法^[5]是在程序执行环境中进行漏洞检测，可以通过分析不同的执行路径和函数功能，检测各种漏洞和风险，但随着软件规模扩大和复杂度提升，动态检测的时间和空间开销不断提高，进而影响检测效率，并且不能覆盖全部路径，造成的漏报率较高。对现有文献整理分析，传统的静态漏洞检测方法主要有基于代码相似度的代码漏洞检测方法和基于规则的代码漏洞检测方法。基于代码相似度的代码漏洞检测方法主要应用于代码克隆引起的软件缺陷，

这一方法有很大的局限性，经常出现很高的假阳性问题。基于规则的代码漏洞检测，需要依赖专家定义特征规则，只能检测特定类型的漏洞，存在一定的局限性。

随着软件规模扩大，传统静态检测方法在检测过程中时间消耗较大，不适用于检测复杂的大规模程序。并且伴随人工智能算法的发展，机器学习在处理复杂代码漏洞检测方面展现优势，避免了依赖专家手工定义检测规则，但需要依赖于专家经验对漏洞特征进行定义，也存在一定的局限性^[6]。在标记特征时，一般需要确定代码的维度和属性进行漏洞挖掘，一般有程序包、程序、组件、文件、函数等级别进行漏洞特征标签^[7-8]，这个过程过分依赖专家经验，耗时耗力，并且特征标记的准确性直接影响漏洞检测的结果。深度学习作为机器学习的延申，在图像处理、语义分割、文本识别等方面表现出良好的优势^[9]，同时深度学习可以自主学习漏洞特征，极大解决的过度依赖专家这个问题^[10]。深度学习神经网络自主学习代码特征，可以清晰准确的描述代码漏洞特征，降低人工经验不足造成的错误。随着大量开源程序的出现，为深度学习方法提供了丰富的数据集，这使得大量研究者对基于深度学习的代码漏洞检测方法进行研究。

深度学习通过模拟大脑处理问题方式，建立良好的神经网络模型，主要的神经网络包括递归神经网络（Recurrent Neural Network, RNN）^[11]、深度信念网络（Deep Belief Network, DBN）^[12]和卷积神经网络（Convolutional Neural Networks, CNN）^[13]。近几年，不断有人尝试将深度学习应用到代码漏洞处理上，并取得很好的效果，主要应用递归神经网络和循环神经网络结构。之后，根据代码漏洞特征检测的特点，将长短期记忆网络（Long Short Term Memory Network, LSTM）应用在代码漏洞检测上^[14]。

使用深度学习算法进行自动化检测软件漏洞，主要流程如图 1.1 所示。首先需要对源代码进行处理，生成中间表示形式，之后将源代码转换成含有特征信息的向量形式，再进行漏洞检测和分类判断。在对源码进行处理时，可以通过词法分析生成 Token 序列、语法分析生成抽象语法树和控制流图等方式进行代码表征^[15]。但过多与漏洞无关的信息影响特征表示，因此本文利用程序切片思想，对源代码进行预处理，再生成 Token 序列，提高代码表征的准确性。在网络结构上，应用于代码漏洞检测的神经网络结构单一，搭建的模型存在过度拟合问题以及代码检测准确性不高的问题。本文根据代码特点，合理搭建深度学习模型，实现基于深度学习的代码漏洞检测模型。

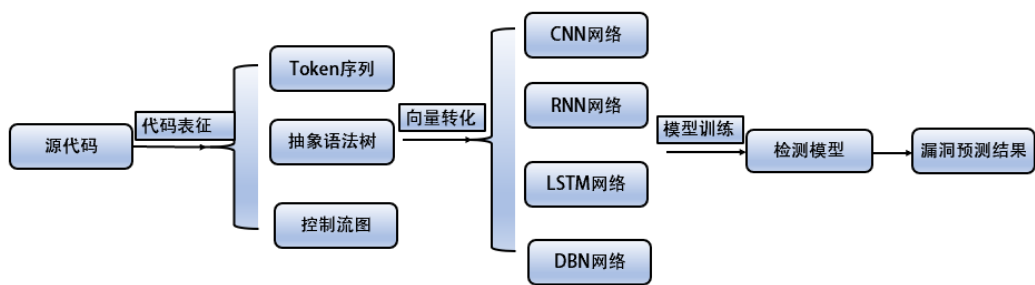


图 1.1 自动化检测技术流程
Fig. 1.1 Automatic detection technology process

1.2 国内外研究现状

目前，根据不同的检测原理，将本部分分为三个方面，分别是代码相似度方面、规则方面、人工智能方面的代码漏洞检测。基于代码相似度的代码漏洞检测主要应用在由代码克隆引起的相关漏洞检测中，因此该部分主要介绍代码克隆漏洞检测的研究现状。根据模型中的机器学习算法不同，基于人工智能的代码漏洞检测从传统机器学习方法和深度学习方法两方面介绍。接下来，如图 1.2 所示，从这几个方面对现有漏洞检测方法进行总结。

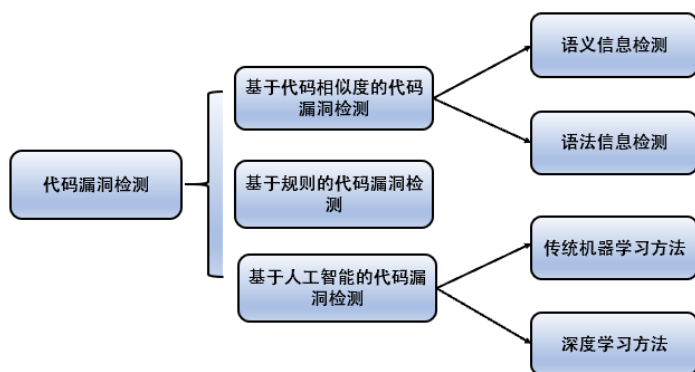


图 1.2 代码漏洞检测研究方法
Fig. 1.2 Research methods of code vulnerability detection

1.2.1 基于代码相似度的代码漏洞检测

代码相似性漏洞检测也称为克隆检测，其核心思想是类似的代码程序具有相似的软件漏洞。在代码重用方面，代码克隆产生的错误经常存在，并对软件开发

埋下隐患。根据克隆技术在不断探索和发展过程中发现的漏洞，研究人员定义了四种常见的克隆类型：Type-1 型（T1）、Type-2 型（T2）、Type-3 型（T3）和 Type-4 型（T4）^[16]。Type-1（T1）是指存在疑似相同的代码段，可能包含空格、布局或注释的变化。Type-2（T2）是允许标识符、文字、类型、空白、布局和注释变化的代码片段。与 Type-2 型克隆相比，Type-3 型（T3）克隆允许进行额外的修改，如更改、添加或删除语句。类型 Type-4（T4）克隆进一步包括语义上等价但语法上不同的代码片段。

在克隆检测过程中，首先要把源代码转化成一种中间表示形式，进而进行相似度检测。源代码的表示决定了信息提取的上限，这进一步限制了模型设计和相似度度量算法的选择，最终影响了检测效果。传统方法是基于文本的表示方法，处理数据只涉及删除程序代码中的注释和空格。随着代码解释器的发展，可以将源码转换成树结构，提出基于树的表示方法。基于树的表示方法可以考虑更多的程序结构信息，从而实现在语法级别的检测。目前，在代码表示、模型设计、相似度度量算法、性能指标等维度，代码克隆检测技术有很大的发展。随着深度学习技术发展和应用，在基于代码相似度漏洞检测技术上也有一定的应用。下面从语法和语义方面介绍深度学习技术在代码克隆检测技术方面的研究进展。

（1）基于语法的克隆检测方法

在语法层上的代码表示主要考虑程序源代码的语法规则。通常，构造抽象语法树（AST）分层显示程序的结构信息，然后再进一步处理并检测。目前，基于深度学习的方法通过构建抽象语法树截取关键节点信息，将其转换成特征向量或哈希值，然后选择合适的神经网络模型进行训练，最后使用该模型实现克隆比较。当前最主要用 AST 树表示代码特征，用 DNN 和 LSTM 网络进行处理。在表 1.1 中，介绍这几种方法的代码表示形式、涉及到的神经网络和检测的克隆代码类型。

表 1.1 基于语法的克隆检测技术特点

Table 1.1 Technical characteristics of grammar-based clone detection

System/writer	Code representation	Network	Clone types
CCLearner ^[17]	Token AST	DNN	T1-T3
CDLH ^[18]	AST	LSTM	T1-T4
White ^[19]	AST FBT	RtvNN	T1-T4
Marastoni ^[20]	Binary	CNN	T4

CCLearner 模型^[17]是在句子粒度进行检测的深度学习检测代码克隆的方法，由于这些克隆几乎总是具有相同的程序语法结构，但可能包含略有不同的算术或逻辑操作，因此，CCLearner 将令牌分成八个类别来创建八个不同的特性，定义了八种标记类型 C1-C8。处理流程使用解释器 ANTLR 解析每个方法并提取特征类型 C1-C3，然后使用 Eclipse 中子类 ASTParser 为每个方法创建抽象语法树，以进一步提取 C4-C8 标记。之后，通过计算每个方法中的标记和方法对之间的相似度，并将该相似度输入到分类器用于训练，进而达到检测给定代码库中的克隆漏洞。为了评估 CCLearner 对 DNN 参数的设置和所选特征的敏感性程度，对不同配置的 DNN 进行了实验，并研究了不同的特征集。观察实验数据可知，CCLearner 在 DNN 中使用 2 个隐藏层和 300 次训练迭代时效果最佳。由于方法标记仅揭示语法层的代码关系，因此该方法测量代码功能之间相似性的能力是有限的。

CDLH 是一种端到端深度学习框架^[18]。该框架以有监督的方式捕获关于代码片段功能行为的词汇和语法信息，并进行散列表示，以进一步将实值表示转换为二进制散列码。与 CCLearner 相比，学习的特征更加紧凑，显著提高了检测效率并节省了存储空间。同时，研究不同长度的散列码对 CDLH 克隆检测性能的影响，实验发现 CDLH 对 8 到 48 的不同代码长度不敏感，这也是 CDLH 的缺陷。

White 等^[19]人同时考虑代码结构和标识符来建模，避免过度依赖通用的手工特征表示代码片段，并使用贪婪的方式在多个树结构之间进行转换以表征代码，并使用循环神经网络（Recurrent Neural Network, RNN）模型将代码片段中的相似项映射到相似的连续向量。此外，该模型基于递归神经网络（Recursive Neural Network, RvNN）模型来学习不同粒度级别的代码片段。这个模型极大解决过度依赖专家经验问题。

Marastoni^[20]提出了一种新的程序相似性检测方法，该方法将二进制代码转换为图像数据，输入 CNN 网络训练模型。同时，使用 Tigress C 模糊器扩展原始数据集，以完全训练模型。由于 CNN 网络需要查看从二进制文件提取的整个图像来完成每个文件的分类。但二进制文件大小限制了系统的性能。

基于语法的代码表示方法考虑了程序代码的结构信息，对于程序语句的序列转换和较小的修改更具容错性。然而，基于树的方法仍然面临一些缺点。随着程序逻辑结构的增强，构造程序抽象语法树的复杂性增加，这进一步导致提取关键代码和向量变换操作的计算开销增加。

（2）基于语义的克隆检测方法

基于语义层的代码表示不仅考虑了源代码的语法，还使用了程序代码的控制流和数据流信息来解决代码功能层的代码克隆检测问题。由程序生成的数据流图 (Data Flow Graph, DFG)、控制流图 (Control Flow Graph, CFG) 和程序依赖图 (Program Dependency Graph, PDG) 是当前主流的表达方法。DeepSim 使用 CFG 和 DFG 节点信息；ZEEK 使用代码符号化和哈希技术；CCDLC 和 Sheneamer 等人使用 PDG 在更深层次上考虑程序语义信息，如表 1.2 所示。

表 1.2 基于语义的克隆检测技术特点

Table 1.2 Technical characteristics of semantic-based clone detection

System/writer	Code representation	Network	Clone types
CCDLC ^[21]	BDG PDG AST	CNN	T3-T4
Sheneamer ^[22]	AST PDG BDG	---	T1-T4
ZEEK ^[23]	Hashes	NN	T4
DeepSim ^[24]	DFG CFG	DNN	T4

CCDLC 等^[21]人实现了一种新的聚类功能，有效地将从 PDG 提取的高级特征与从 Java 字节码依赖图 (Java Bytecode Dependency Graph, BDG) 提取的低级特征相结合，紧密反映程序代码中数据之间的关系。在模型训练方面，CNN 网络采用丢失正则化和校正线性单元实现，采用多线程小批量下降运行，实验结果表明，CCDLC 在代码混淆和语义克隆检测方面取得了良好的效果。CCDLC 系统进一步改进了数据预处理和特征向量转换过程。

Sheneamer 等^[22]人主要对机器学习中的代码数据集进行预处理，以此提高代码漏洞识别的准确性。主要存在的问题是识别代码克隆而构建的机器学习方法表现不佳，并且仅限于 Type-3 型克隆问题，大多数公开可用的代码克隆语料库本质上是不完整的，并且缺少用于语义或类型 IV 克隆的标记样本。在文献[22]中提出了两种方案来标记所有类型的克隆。首先，使用无监督的方法来标记 Type-IV 克隆，并使用专家 Java 程序员对其进行验证。接下来，提出了从第一个方案派生的标记样本来标记或分类未知样本的监督方案。使用六个著名的 Java 代码克隆语料库评估方案的性能，并根据 kappa 协议报告生成克隆的质量指标得分，其中平均误差和准确度得分较高。结果表明，这两种方案都产生了高质量的代码克隆，有利于未来机器学习在检测 IV 型克隆中的应用。

ZEEK 模型^[23]引入了一种新的向量表示方法 `proc2vec`，更好地执行词向量的嵌入学习。首先，将程序汇编代码拆分为基本块，并使用组成代码部分的链作为特征集，将链转换为数字，并将这些数字组合成表示相应代码的向量。与直接令牌符号数值转换相比，数值向量矩阵更稀疏，可以大大降低相似度的计算成本。神经网络模型是一种常见的四层结构，用交叉熵损失函数、0/1 正则化函数、`batchsize` 参数 32 和 3 次数据传递来训练。此方法极大改善网络训练模型。

DeepSim 模型^[24]提出将代码控制流和数据流编码为语义矩阵的编码方法，在此基础上度量代码功能相似度。该模型将从代码对中学习到的隐藏表示串接起来，并将功能相似代码的检测问题转化为二进制分类，这样可以有效地学习语法不同但功能相似的代码检测模式。之后，通过预定义的编码规则生成高维稀疏二进制特征向量语义矩阵。语义特征矩阵将寻找同构子图的问题转换到检测相似模式问题上，并使其易于后续处理。DeepSim 在查全率、精确度和时间性能方面明显优于其他技术。

与一般自然语言文本特征相比，基于语义的方法充分考虑了程序结构、序列和特殊语法信息，并深化了语义特征。然而，基于语义的表示技术需要使用生成器为每种编程语言生成程序依赖图，这具有较差的可扩展性，并且很少讨论神经网络模型的选择和优化。

1.2.2 基于规则的代码漏洞检测

基于规则的代码漏洞检测方法是对漏洞进行模式匹配，根据专家经验对特定的漏洞总结特征，制定漏洞检测规则，建模分析，得到检测模型。

Klocwork 是基于 C/C++ 编码规则的对源代码进行检测的静态代码漏洞检测工具^[25]。Klocwork 主要为企业 DevOps 和 DevSecOps 构建静态漏洞代码检测方法，并可以扩展到任何规模的项目。Klocwork 与大型复杂环境、各种开发人员工具集成，为企业整个提供控制、协作和报告，这使得 Klocwork 称为首选的静态分析工具。Klocwork 可以在项目开发初期检测漏洞，进而提高代码质量，降低项目的报错率。

Jang 提出基于规则的代码审计系统^[26]。该系统分析目标代码的结构来检测恶意代码和软件漏洞。首先根据预定义的规则对含有漏洞的代码进行处理，转换成中间形式，之后通过 FSA 语法定义检测规则，达到检测代码漏洞的目的。所提出的审计系统可以构建安全的泛在计算环境，因此它被认为是一个考虑源代码安全

性的通用软件审计系统。

刘设计了基于安全规则的 C/C++静态代码检测系统 CIS^[27]。该系统提出基于 Lex 和 Yacc 工具的静态解析器，将源代码解析为不含冗余信息的关系语法树；之后，通过一种可扩展性的 XML 中间存储方式，存储关系语法树。该系统可以对分布在不同文件中的关联代码进行检测，并且不需依赖源代码中的注释说明。

Shahriar 等人提出基于规则的 C/C++缓冲区溢出漏洞检测和修复规则^[28]。在本文方案中首先通过对缓冲区溢出漏洞研究分析，总结出该漏洞的原因，主要原因有空字符、库函数调用、指针运算等。之后提出相应的检测规则，并提出八种修复漏洞的方法。使用一些开源 C/C++应用程序评估提出的方法，结果表明，所提出的规则可以识别已有的 BOF 漏洞并挖掘新的 BOF 漏洞。

PMD 是基于 BSD 协议的 Java 静态代码检测工具^[29]。该工具将源码解析成抽象语法树，对不同的节点制定不同的安全检测规则，分析属性和结构找到不符合规定的部分，进而找到漏洞。该工具是开源的并且在代码编写初期可以进行检查，提高代码编写质量。

Fortify 是一个应用程序静态安全检测工具^[30]。该工具可以帮助开发人员和专家分析源码、检测漏洞、提出安全性问题、区分安全性问题的优先级，进而提高应用程序的安全性和开发人员解决问题的效率。Fortify 支持 25 种编译语言，可检测漏洞类别超过 961 个，支持多种平台和 IDE，适用于多种开发环境漏洞检测。Fortify 最大的优势是有集成性强的自定义检测规则，可以全面的检测、匹配漏洞，但也存在检测效率慢的缺点。

基于规则的代码漏洞检测对特定规则的漏洞检测有着准确度高、误报率低的优势，但也存在一定的劣势，比如在前期规则定义上耗时耗力、制定的规则好坏直接影响检测结果。

1.2.3 基于人工智能的代码漏洞检测

人工智能方法在各个领域应用广泛，在代码漏洞检测上也表现出很大的优势。根据使用的神经网络模型和是否需要专家手工定义特征，将漏洞检测技术分为传统机器学习方法和深度学习方法。

(1) 基于传统机器学习的代码漏洞检测

基于传统机器学习的漏洞检测方法首先依靠专家经验进行特征定义，之后采用机器学习方法，如 K 近邻算法、支持向量机、朴素贝叶斯等进行分类。

Chernis 等人在 C 源代码函数中分别提取简单特征和复杂特征，之后经过机器学习中的 K 均值、K 近邻和朴素贝叶斯等分类器进行试验^[31]。通过实验结果分析，可以得到机器学习算法在函数特征检测方面是有效的，并且与复杂特征的 69% 的正确率相比，简单特征 74% 的正确率相对更高。可以说明机器学习在函数漏洞检测方面是有效的。

Lin 等人提出基于 AST 的漏洞检测模型^[32]。首先将源码进行编译，对每个函数生成抽象语法树（AST），对 AST 通过深度优先遍历（DFT）序列化，并在保留语义结构下，进行序列等长处理；之后，通过 CBOW 模型进行向量化处理，得到 100 维向量，选用 23 个代码度量提取特征，通过随机森林分类器训练。实验结果显示使用此方法对代码漏洞检测有效。

Harer 等人提出了一种使用机器学习的数据驱动漏洞检测方法^[33]。该方法从源代码编译和构建过程中提取代码特性，在函数层，提取函数的控制流图；在控制流图中，提取每个控制块的执行信息和变量操作。之后将这些代码特征定义生成 116 维的特征向量，输入到随机森林中进行分类。

Napier 等人提出一种基于文本的机器学习模型^[34]。该方法在提取的源代码片段上以函数为关键点对源代码进行处理，之后基于函数对代码片段构建语义相同的配对模式，主要是解决跨函数代码漏洞问题。实验使用七种机器学习模型、五种自然语言处理技术和三种数据处理方法，结果表明在函数内部测试而不是跨函数测试时，具有较少特征的压缩函数会获得更准确的预测结果。最终得出结论，基于文本的机器学习模型在检测项目内部漏洞是有效的，在检测或跨项目漏洞方面无效。

Braga 等人通过对深度学习模型 code2vec 进行迁移学习，生成机器学习模型进行特征提取，从而扩展了机器学习方法来检测源代码中的漏洞问题^[35]。实验将这种方法与以前不同类型的二进制模型中的方法进行了比较，还将代码混淆用在机器学习源代码相关任务中达到数据增强的目的。实验结果表明，通过迁移学习，code2vec 可以成为源代码漏洞竞争性特征生成器，简单的代码混淆可增强数据生成，进而加强机器学习模型在源代码相关任务中的训练。

Chen 等人提出基于 Petri Net 检测内存泄漏模型^[36]。该模型考虑漏洞出现的特征和原因，从漏洞的主题和环境进行建模，提出一种基于漏洞模型的内存相关漏洞检测框架（MRVD-VM）及其基于漏洞模式的漏洞检测算法（VDA-VM）。实

验结果表明, MRVD-VM 在检测内存泄漏、内存双释放和释放后使用方面明显优于三种基线工具, 基线工具包括 Cppcheck, Flawfinder 和 Splint。

Ashizawa 等人提出一种基于机器学习的静态分析工具 Eth2Vec^[37]。Eth2Vec 可以在重写的代码中检测漏洞, 使用神经网络进行语言处理, 自动学习易受攻击的合约特征, 在此过程中, Eth2Vec 可以通过比较目标合约的代码与学习合约的代码之间的相似性来检测智能合约中的漏洞。实验结果表明, Eth2Vec 在指标 (即精度, 召回率和 F1 分数) 方面优于基于支持向量机的最新模型。

Alani 提出了一种基于可解释机器学习的基于数据包的高效僵尸网络检测系统^[38]。本文提出的方法侧重于特征选择, 用生成的具有七个特征的数据集来训练实现非常高准确性的机器学习分类器。实验结果表明, 依靠从网络数据包中提取的这七个选定特征, 准确率超过 89%。

Dhanya 等人提出一种用于识别网络入侵和攻击类型的鲁棒机器学习模型^[39]。提出的模型已经对 UNSW-NB15 数据集进行了实验, 该数据集包含 49 个不同攻击样本的 9 个特征。与集成模型 (随机森林 (98.86%)、Adaboost (97.87%) 和 XGBoost (98.08%)) 相比, 决策树分类器产生了 99.05% 的最佳准确率。K 近邻分类器针对各种 K 值进行了训练, 并在 K=7 下获得了最佳性能, 准确度为 95.58%。决策树在本文方法中准确性更高。

Clottey 等人将机器学习技术在最近的网络入侵数据集上的性能进行建模和评估^[40]。模型采用五种机器学习算法, 包括 k-近邻 (KNN)、支持向量机 (SVM)、投票融合、随机森林和 XGBoost。使用 UNSW_NB15 数据集对所提出的模型进行测试。使用 KNN 算法的模型使用三个不同的 K 值, 并使用 SVM 开发模型使用两个不同的内核。利用 KNN 开发的模型检测精度最高, 为 84.9%, K 值为 9; 利用高斯核开发精度最高的 SVM 模型, 准确率为 83%, 投票融合的准确率为 83.4%。随机森林模型对二元分类和多类分类的准确率分别为 90.2% 和 70.8%。最后, XGBoost 模型在二元和多类分类的准确率也分别达到了 85% 和 51.77%。

基于传统机器学习的代码漏洞检测方法可以检测出一些漏洞, 说明该方法是有用的。同时, 也存在着一些问题, 首先在前期数据处理方面较复杂, 需要人工定义代码特征, 过度依赖专家经验; 并且, 此方法大多数在函数级别进行检测, 检测粒度较粗糙。

(2) 基于深度学习的代码漏洞检测

深度学习有着学习能力强、覆盖面广的特点，在目标检测、图像识别等领域已经取得很好的成效。将深度学习引用到代码漏洞检测上，可以免去人工提取特征的这一复杂工程，很大程度上避免的专家经验造成的一定的失误，在代码漏洞检测上准确率上有很大的优势。

基于深度学习的漏洞检测技术主要包括两个阶段：在训练阶段，使用控制流和数据流技术提取程序中的关键代码，在词向量转化工具的帮助下将其转换为向量，并将其输入适当的神经网络进行监督训练；在检测阶段，对新软件程序执行相同的数据处理，用于通过学习好的模型检测现有漏洞。目前用于模型训练的网络结构包括 CNN、RNN 和 LSTM。在训练阶段，主要分以下几步进行：样本代码构建、特征提取、词向量生成、神经网络模型训练。其中，漏洞特征提取主要涉及如何选择合适的粒度来表示软件程序和漏洞检测。由于深度神经网络将向量作为输入，需要将程序表示为对漏洞检测具有语义意义的向量，所以需要使用一些中间表示作为程序与其向量表示之间的“桥梁”，向量表示是深度学习的实际输入。漏洞特征提取是将程序转换为某种中间表示形式，想要保留程序的语义信息，可以通过 CFG 和 PDG 技术，将程序之间的数据依赖关系和控制依赖关系表示出来。基于特征提取生成词向量，应用最主流的词向量生成技术，以便将中间表示转换为向量表示，作为神经网络的实际输入。在检测阶段主要分为两步：将训练阶段从历史代码库中提取的代码向量表示作为输入，得到微调模型参数的神经网络；将待检测程序代码，表示为向量形式，通过模型获取检测结果，如图 1.3 所示。

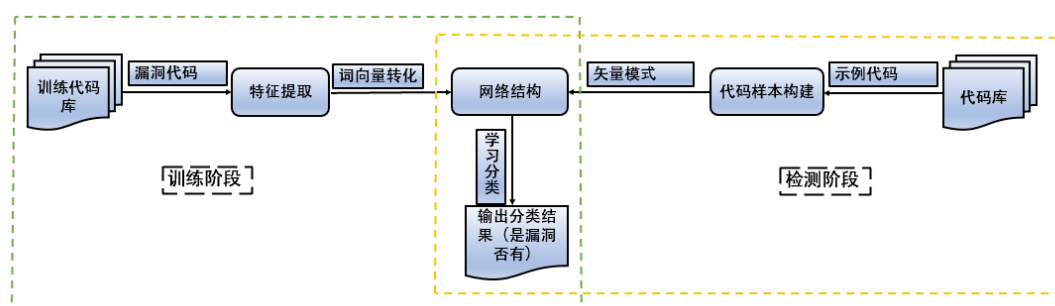


图 1.3 代码漏洞检测流程

Fig. 1.3 Code vulnerability detection process

李等人提出一种基于深度学习的代码漏洞检测系统 VulDeePecker^[41]。该系统定义了一种中间形式 code gadget，code gadget 保留元素之间的语义信息，在特征

提取方面更有全面性。之后将 `code gadget` 通过 `word2vec` 网络转换成向量,该向量表示是神经网络的实际输入。由于程序函数调用的参数可能会受到程序早期语句的影响,也可能会受到后续语句的影响,考虑到程序代码中变量和语句之间的关系,所以使用双向 LSTM 网络结构搭建检测模型。提出首个将深度学习应用于代码漏洞方面的检测系统,实验结果表明, VulDeePecker 有更低的误报率,并且在 3 个软件产品中检测出 4 个新的漏洞,但局限性在于检测漏洞特征有限。

针对 VulDeePecker 现有的缺陷,进一步考虑了程序控制流程,提出了一种基于语法、语义和向量的漏洞检测系统框架 SySeVR^[42]。该方法使用数据流分析用于从程序中提取和生成数据流相关切片,然后用于通过 PDG 生成程序切片并转换为控制程序切片。SySeVR 提出了自己的词向量生成算法,避免了 `word2vec` 模型的向量转换问题,并优化了神经网络模型的特征参数选择。

通过对 VulDeePecker 模型改进,提出多类型漏洞检测系统 μ VulDeePecker^[43]。该模型引入控制依赖性重新定义 `code gadget` 的概念和提取方法,并且提出了用于漏洞类型检测的 `code attention` 概念,构建双向 LSTM 网络,旨在融合来自 `code gadget` 和 `code attention` 的不同类型特征,以适应检测不同类型的信息,达到检测多种漏洞的目的。这种神经网络架构具有独立的价值,因为它为不同的代码特征提供了有效的融合思想。

Wang 等人提出一种用于构建漏洞检测模型的新型学习框架 Funding^[44]。Funding 利用图神经网络来开发一种新的基于图的学习方法,以捕获和推理程序的控制、数据和调用依赖关系。在使用 C、JAVA、和 PHP 编写的大型真实数据集上评估模型 Funding,并将其与六个最先进的代码漏洞检测模型进行比较。实验结果表明,该模型在评估指标数值上明显优于其他方法。

Wang 等人提出基于传统机器学习和基于深度学习的漏洞检测方法,并在确定影响因素下进行性能对比^[45]。该方法主要研究以下因素的性能影响,其中,数据集的质量、分类模型和矢量化方法可以直接影响检测性能,相比之下,函数/变量名称替换可以影响漏洞检测的特征,间接影响性能。实验结果表明:1)深度学习模型可以达到比传统机器学习模型更好的性能。在所有的模型中,BiLSTM 可以实现最好的性能。2)CountVectorizer 可以显著提高传统机器学习模型的性能。3)带有用户定义的变量和函数名称替换的数据集会降低漏洞检测的性能。

Huo 等人提出定位潜在 Bug 的检测模型 CG-CNN^[46]。该模型是一个多实例学习框架,利用控制流图的结构和有序的性质,增强了错误定位的统一特征。在广泛使用的软件项目上的实验结果证明了提出的 CG-CNN 模型的有效性。

段等人提出基于代码属性图和注意力双向 LSTM 的漏洞挖掘方法^[47]。该方法根据生成的代码属性图进行程序切片，去除冗余信息，转化成 144 维特征向量，输入到双向 LSTM 中进行训练。最后，在 SARD 上，模型 F1-score 达到 80%，有效识别单个函数是否有漏洞。

Sun 等人提出利用 Siamese 网络与 BiLSTM 合作处理源代码的模型结构 VDSimilar^[48]。在 OpenSSL 和 Linux 的 876 个漏洞和补丁的数据集上，所提出的模型的 AUC 值约 97.17%，比最先进的基于深度学习的方法更突出。

Chakraborty 等人分析现有的基于 DL 的脆弱性预测方法在训练数据（如数据重复、漏洞类别的不均匀分布等）和模型选择（如简单的基于标记的模型）方面受到的挑战，证明了基于漏洞预测的平衡分布的数据集和多标记的模型设计方法能够带来更好的解决方案^[49]。由此产生的工具比所研究的基准方法精度提高了 33.57%、召回率提高了 28.38%。

Mazuera-Rozo 等人提出使用卷积神经网络（CNN）和循环神经网络（RNN）搭建漏洞检测模型^[50]。在三个不同数据集上，证明该模型优于随机森林分类器。

Cheng 等人提出一种新的基于深度学习的嵌入方法 DeepWukong^[51]。该方法利用图神经网络将代码片段嵌入到一个紧凑的低维表示中，产生一个新的代码表示，保留了高级编程逻辑（以控制流和数据流的形式）以及程序的自然语言信息。实验结果证明了本文方法的有效性。

Zheng 等人定义了一种名为“切片属性图”（SPG）的新型代码表示法，并提出了一种使用改进的 R-GCN 模型与三重注意力机制来识别 SPG 中的潜在漏洞的新漏洞检测方法 VulSPG^[52]。SPG 可以反映丰富的语义和明确的结构信息，同时尽可能地消除不相关的信息以降低图的复杂性。其次，VulSPG 在 R-GCNs 中加入了三重注意力机制，以实现从 SPG 中更有效地学习漏洞模式。我们在两个大规模的数据集上对 VulSPG 进行了广泛的评估，其中包括来自 SARD 的程序和真实世界的项目。实验结果证明了 VulSPG 的有效性。

Li 等人提出了一个可解释的漏洞检测器 IVDetect^[53]。其理念是使用人工智能（AI）来检测漏洞，同时使用智能助手（IA）来提供漏洞声明方面的 VD 解释，因此模型能够更好地分辨出含有漏洞的语句，同时提供细粒度的解释，包括程序依赖图（PDG）中的子图，以及与检测到的漏洞有关的关键语句。实验结果表明，IVDetect 比基线解释模型的准确性提高了 12.3%和 9.0%。

Wu 等人提出了一种基于图表示学习的特征提取方法^[54]。该方法首先将软件功能表示为简化代码属性图（SCPG），它可以保存源代码的句法和语义信息，同时保持自身足够小的计算量。然后，利用图神经网络和多层感知器来学习图的表示并自动提取特征，从而节省了特征工程的时间开销。对比实验证明了所提方法的有效性。

Nguyen 等人提出了一种基于相互信息概念的新方法^[55]，可以在无监督和半监督的情况下检测和隔离细粒度的软件漏洞（即与软件漏洞高度相关的几条语句，其中包括核心漏洞语句）。通过在真实世界的软件项目上进行全面实验，证明本文提出的方法可以识别对漏洞检测决策贡献最大的几条语句，在细粒度层面上检测漏洞。

许等人将滑动窗口和哈希函数融合到深度学习方法中，提出一种新的深度学习代码漏洞分类模型^[56]。该方法分别构建源代码方法体的正负样本抽象语法树，之后对节点进行信息分词，进行独立节点编码。通过拆分树，构成词序列，之后，通过滑动窗口与哈希函数训练出相应的漏洞检测分类模型进行检测分类。实验结果表明，提出方法能够有效检测出代码漏洞。

文等人提出基于关系图卷积网络的源代码漏洞检测方法^[57]。该方法对源程序进行处理，转化成含有语法、语义特征信息的 CPG；之后选中 69 种类型的节点和 5 种语法语义边，之后使用 word2vec 进行向量转化，采用 DGL 画出图数据结构 DGLGraph。最后将向量输入到 RGCN 模型中进行漏洞分类。

Liu 等人提出了一种基于图卷积网络的跨站脚本有效负载检测模型 GraphXSS^[58]。该模型使用图卷积网络和残差网络来训练跨站点脚本检测模型。在实验中，基于图卷积网络（GCN）的模型在小样本条件下的 AUC 值可达 99.7%。与加入残差网络结构后的检测模型相比，该模型在多卷积层下能收敛稳定，准确率达到 99.6%。

Guo 等人提出了一种基于混合语义的图神经网络漏洞挖掘系统 HyVulDect^[59]。该模型使用门控图神经网络用于提取深层语义信息，使用 BiLSTM 模型提取上下文的令牌级特征，最后使用分类器对融合特征进行分类。实验结果表明，HyVulDect 在基准数据集上可以达到 92% 的准确率。

Cheng 等人提出一种选择性的和对比性的价值流嵌入方法 ContraFlow^[60]。ContraFlow 通过使用自监督对比学习的预训练路径嵌入模型来选择和保留可行的价值流（又称程序依赖性）路径，从而大大减少了用于训练基于路径漏洞检测模

型的数据量。通过比较八种基于学习的方法，ContraFlow 在准确度、召回率和 F1-score 方面比这八种基线高出 3.1%、3.9%、5.3%。

Ding 等人提出一种通过集合学习方法来定位脆弱的语句的模型 Velvet^[61]。模型结合了基于图和基于序列的神经网络，成功地捕获了程序图的局部和全局特征，并有效地解释代码语义信息。实验表明，Velvet 在真实世界的数据上取得了比基线静态分析器好 4.5 倍的性能；Velvet 在合成数据和真实世界数据上分别达到了 99.6%和 43.6%的最高准确率，比基线深度学习模型高出 5.3%和 29.0%。

Li 等人提出了一种基于深度学习的技术 Drex^[62]。Drex 通过基于图的代码表示学习来实现的漏洞检测的，首先，为每个方法构建 CFG、DFG 和 AST 的联合图，以便全面描述语句语法和语义；其次，使用基于注意力的图神经网络学习语句嵌入，以区分和解释不同类型的潜在信号运行时产生的异常。实验结果表明该方法有效。

Wang 等人提出一个二进制级别的漏洞检测工具，用于准确的已知和未知漏洞检测 BinVulDet^[63]。该模型使用反编译技术来获取包含高级语义信息的伪代码，以对抗编译多样性的影响，然后利用程序切片技术提取与漏洞相关的数据依赖和控制依赖的语句，使用 BiLSTM 注意力神经网络从切片代码中提取丰富的上下文语义信息，以生成更强大的漏洞模式来检测漏洞。实验结果表明，BinVulDet 的 FPR 和 FNR 平均分别为 1.04%和 0.89%，平均比基线模型低 3.93%和 22.86%。

Han 等人提出结合注意力机制和图网络结构的代码语义漏洞检测模型^[64]。该模型利用源代码在图层面的语义来补充其在文本层面的语义，结合注意力机制对图语义进行优化和调整，得到包括源代码浅层和深层语义的代码语义特征。最后，通过余弦相似度来衡量代码语义特征和报告语义特征之间的关联性。在三个开源的 Java 项目上进行了实验，实验结果表明，该模型明显优于最先进的方法。

Liu 等人提出一种结合多特征和图注意力机制的漏洞检测模型 MF XSS^[65]。该模型结合抽象语法树（AST）和代码控制流图（CFG），将广义的示例数据转换为图结构和代码字符串结构。然后，通过图卷积神经网络、加权聚合和双向循环神经网络，分别提取并融合源代码的逻辑调用特征和上下文执行关系特征，之后进行漏洞检测。通过实验证明，该模型在标准数据集和真实数据上准确率达 99.7%和 98.6%，并在实际的 Web 环境，成功检测到网站中存在 XSS 漏洞。

Zhao 等人提出了一种基于功能指纹和代码差异的漏洞检测模型 VULDEFF^[66]。该模型提出基于上下文触发分段散列算法的轻量级函数指纹方法，此方法可以表

征函数源代码的基本语法特征。特别是，易受攻击函数的指纹包含易受攻击函数的语法特征、漏洞特征和补丁特征。VULDEFF 通过在易受攻击的函数指纹库中搜索目标函数指纹来检测目标函数是否存在漏洞。与五种先进的软件漏洞检测工具相比，VULDEFF 在保证漏洞检测可扩展性的同时，显著降低了误报率和漏报率。VULDEFF 在 111 个开源项目中发现了 10 个新漏洞。

Guo 等人提出一个用于漏洞检测的复合神经网络模型 VulExplore^[67]。该模型网络结构是由一个用于特征提取的 CNN 和一个用于深度表示的 LSTM 网络组成。实验结果表明，该方法获得低于 20% 的假阴性率（FNR）和假阳性率（FPR），同时，召回率和精确度分别达到 80% 以上。

Mei 提出一个上下文嵌入模型，根据物联网应用程序的代码特征，整合了 CLSTM、CBiLSTM（顺序结构）和 CNN-BiLSTM（并行结构）三种网络结构的混合模型^[68]。该方法继承了 CodeBERT 的架构，采用多头注意力机制，在处理代码序列数据时，具有学习长距离上下文依赖性的脆弱代码的能力。为了评估混合语境嵌入的有效性，本文对比了三种嵌入方法（包括 CodeBERT、Word2Vec 和 FastText），结果显示，基于神经特征建立的混合模型在漏洞检测方面优于非混合模型。

Şahin 提出基于门控图序列神经网络的语义漏洞检测模型^[69]。模型首先从图门控序列神经网络（GGNN）的功能连接（FC）中推导出漏洞知识表示的子集，之后将函数表示用图神经网络连接起来。门控图序列神经网络主要作用是捕获长期依赖关系，挖掘潜在漏洞的高级表示，从而检测目标项目的漏洞。基于函数连接的图神经网络可确保模型专注于与漏洞相关的代码，从而使其更适合漏洞挖掘任务。实验结果表明，所提模型能够选择相关的判别特征，并取得优于基准方法的性能。

基于深度学习的代码漏洞检测模型，改善了特征提取工程，用自动化特征提取方法替代传统机器学习人工定义特征，使得提取的特征更有泛化性并且避免专家经验对检测结果的影响。但也存在着模型过度拟合、网络结构单一以及检测粒度较粗等问题。

1.3 本文的研究工作与内容安排

深度学习模型应用到漏洞检测领域提高了漏洞检测效率和准确性，改变了机器学习中过度依赖专家经验的特征提取方式，在一定程度上，实现漏洞检测技术

的自动化，但也存在一定的假阳性和假阴性问题，以及对于网络结构创新不足问题。Transformer 模型在计算机视觉领域发展迅速，在目标特征识别上有很好的效果，但在代码漏洞检测领域，几乎没有应用。

为解决上述问题，本文的研究内容和主要工作如下：

首先，在 NVD 和软件保证参考数据集 (Software Assurance Reference Dataset, SARD) 项目上收集 C/C++ 源代码，构建数据集。针对源代码的存在过多冗余信息，采用程序切片思想对源代码处理，提取与漏洞相关的代码语句，减少无关语句，降低模型训练的复杂度，提高模型的检测效率。

其次，提出基于注意力机制的代码漏洞检测模型。结合 Transformer 模型与双向 LSTM 模型优势，加深网络结构。利用 Transformer 模型处理大量数据的优势以及自注意力机制，用 Transformer 编码器进行全局特征提取，获取准确全面的漏洞特征；利用 BiLSTM 网络进行代码特征上下文语义学习，之后进行分类检测。通过两个模型结合使用，提高模型对代码漏洞的检测效率和准确性。之后，由于代码处理方式不同于自然文本处理方式，代码存在语义上的关联信息，故采用相对距离的位置编码方式解决语义相关联问题，提高敏感函数周围词的注意力权重，进一步提高模型检测代码漏洞的准确率。

最后，在 NVD 和 SARD 上收集 C/C++ 数据集，并在上面进行实验，从以下几个方面进行模型评估：（1）该模型能否正确检测出代码漏洞。（2）与基线相比，该模型性能是否提高以及模型的训练时间开销是否降低。（3）Transformer 编码器的位置编码对模型性能的影响。

论文共分为 5 个章节，每个部分的内容如下：

第一章：绪论。介绍代码漏洞在软件开发领域的危害，以及进行该项工作的重要性；之后，对代码漏洞检测国内外研究现状进行介绍，从传统代码漏洞检测到应用传统机器学习方法解决软件漏洞检测问题、再到用深度学习方法检测代码漏洞，不断优化检测方法，避免了过度依赖人工经验，提高检测效率。最后说明代码漏洞检测的现存问题以及解决办法。

第二章：相关知识。主要介绍代码漏洞相关知识，以及常用的深度神经网络结构，为之后研究做理论基础。

第三章：基于注意力机制的代码漏洞检测算法。整体介绍系统的框架，之后对每部分数据构造、模型搭建进行详细说明。本章在传统的深度学习模型网络加入注意力机制，提高网络结构性能和代码特征提取性能，进而提高模型漏洞检测

的准确率。在 SARD 和 RVD 上搜集数据集进行相关实验，实验结果证明加入注意力机制的网络在代码漏洞检测上准确率以及 F1-score 较高，并且模型训练时间显著降低。

第四章：基于位置编码优化的代码漏洞检测算法。针对自然语言处理中文本语义上的关系，对 Transformer 中的位置编码进行优化，采用基于相对距离的编码方式，将词之间的相对距离注意力权重加入到模型中，提高敏感函数周围词的注意力，进而提高模型代码漏洞检测的准确性。在 SARD 和 RVD 上搜集数据集，进行实验得到模型的准确率、召回率、精确率和 F1-score，通过对比说明位置编码对代码漏洞检测模型性能影响，以及证明本文提出的编码方式更适合代码漏洞检测领域。

第五章：总结。总结全文内容，探讨本文存在的不足，以及对未来研究内容的展望。

第 2 章 相关研究工作

2.1 软件漏洞概述

2.1.1 软件漏洞定义

目前软件程序要求越来越高，代码编写也越来越复杂，开发过程中难免会产生一些错误。通常把功能上的逻辑错误称为 bug^[70]，这个在软件测试过程中可以检测出来，可以进行及时修正，效果明显且易查错。其次，还有一些软件安全方面的逻辑错误，潜藏在程序中，不易发现，但在程序中存在潜在隐患，攻击者可以利用这些漏洞进行用户攻击，造成不可避免的损失，这类安全性逻辑错误称为漏洞^[71]。软件漏洞分类很多，主要常见的漏洞有缓冲区溢出漏洞^[72]、SQL 注入漏洞^[73]、XSS 漏洞^[74]等。很多国家建立软件漏洞安全库，收集各种漏洞，构建庞大的漏洞数据库，为之后的科研提供大量有效的数据。具有权威的外国数据库是美国国家漏洞数据库等；在国内，认可度较高的数据库是中国国家信息安全漏洞库等。这些数据库为研究代码漏洞检测方法提供了丰富的数据。

2.1.2 典型的软件漏洞

（1）缓冲区溢出漏洞

缓冲区是指在内存中存在的一个暂时存储数据的连续地址。如图 2.1 所示，向一个缓冲区写入数据，如果将多于存储空间的数据放入，或者将一段程序放入，超过缓冲区边界，就会造成缓冲区溢出，触发缓冲区溢出漏洞^[72]。

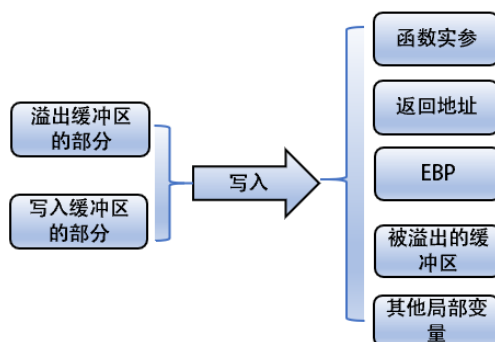


图 2.1 缓冲区溢出示例

Fig. 2.1 Principle of buffer overflow

缓冲区漏洞攻击主要分为三步：将攻击代码注入内存；在运行时跳转到攻击代码位置；之后执行错误代码。缓冲区溢出产生代码漏洞的原理在于，一旦出现缓冲区溢出，就会跳转到攻击代码上，产生潜在的危险。根据缓冲区攻击的对象，缓冲区漏洞可分为破坏栈数据漏洞、破坏堆数据漏洞、更改函数指针类漏洞。

（2）SQL 注入漏洞

SQL 注入漏洞^[73]存在于 Web 服务中，通过恶意 Sql 代码获得访问数据库权限，可以躲避管理员管理，甚至越过管理员权限，对用户信息进行获取、篡改等违法活动，引发严重后果。引发各方极大关注。主要引发漏洞原因是不恰当使用 API 操作数据库以及缺乏有效的输入验证和过滤机制。

（3）XSS 漏洞

XSS 漏洞^[74]是在 Web 页面注入恶意代码，当用户在客户端打开这种含有恶意代码的页面，就会对用户产生攻击，通过这种方式，黑客利用钓鱼方式可以获取用户一系列信息，造成不可挽回的损失。

2.2 典型的深度神经网络

随着机器学习的发展，将机器学习模型^[75]应用到代码漏洞检测方面取得重大突破，常用的机器学习算法^[76]有 K 近邻算法、决策树算法等。深度学习在代码漏洞学习方面表现出强大的优势，主要表现在代码特征提取以及网络训练模型上，下面详细介绍几种典型的深度神经网络。

2.2.1 卷积神经网络

卷积神经网络（Convolutional Neural Networks, CNN）^[77]，主要应用在图象识别方面。之后，将 CNN 应用在文本、声音等特征的学习中，也有着良好的表现。在图 2.2 中展示了卷积神经网络模型，主要是由三部分组成。

卷积神经网络最核心的部分是卷积层，相关计算都在这一层，最重要的特征提取在这一层。卷积层通过特定步长滑动检索像素点，提取特征，一般一层的卷积网络提取特征能力有限，通常采用叠加的方式，通过多层卷积层进行提取特征。在处理高纬度输入时，采用局部连接的方法，控制感受野，可以提升模型提取特征的能力。这也存在一个问题，可能会出现过度拟合的情况，通常在卷积层后加一层池化层，用于数据降维操作，既可以减少计算资源也可以减少网络参数数量。最后，输入到全连接层，对特征进行结合，通过分类器计算每一种分类的概率，

在这个过程中，防止过度拟合会引入 Dropout。深度学习算法不断发展，在 CNN 基础上衍生较多经典模型，应用于不同场景，主要有 AlexNet^[78]、VGGNet^[79]和 Fast R-CNN^[80]等。

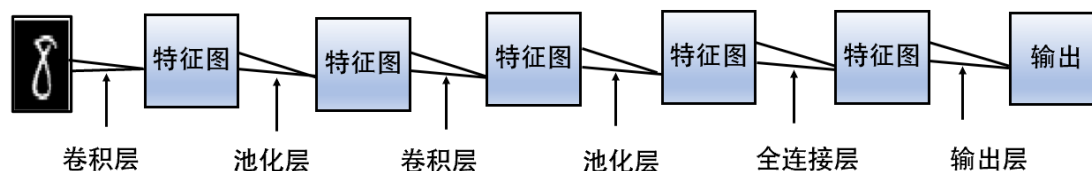


图 2.2 CNN 模型

Fig. 2.2 CNN model

2.2.2 循环神经网络

卷积神经网络主要应用在视觉领域，模仿人类视觉神经网络，只对输入进行处理，没有记忆功能，对以往完成的任务没有记忆。循环神经网络（Recurrent Neural Network, RNN）^[81]加入了记忆单元，在输入时不仅有当前数据，还将上一次网络输出结果一起加入到新网络中。RNN 网络结构如图 2.3 所示。

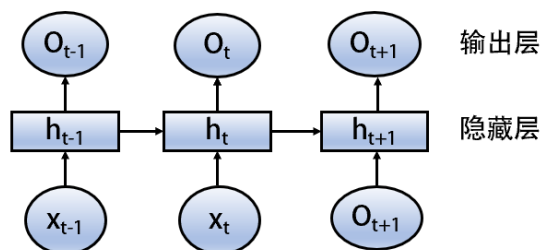


图 2.3 RNN 结构

Fig. 2.3 RNN structure

之后对 RNN 采用基于时间的反向传播算法（Back Propagation Through Time, BPTT）^[82]对模型参数进行更新。在长序列问题处理上，出现梯度下降，模型效果大幅度降低，故只能处理短时问题，并且在处理文本信息时，如果两个相关性的词距离比较远，循环网络处理效果也不好，这称为长时依赖问题。

2.2.3 长短期记忆网络

长短期记忆网络（Long Short-Term Memory, LSTM）^[83]提出后解决了长时依赖问题。网络结构如图 2.4 所示，主要有三部分构成：输入门、遗忘门、输出门。输入门主要对网络输入进行控制；遗忘门主要是由记忆单元组成，可以保留以前网络训练的记忆，并在此次网络训练过程中自主加入，是 LSTM 网络的核心部分；输出门主要控制网络的输出。

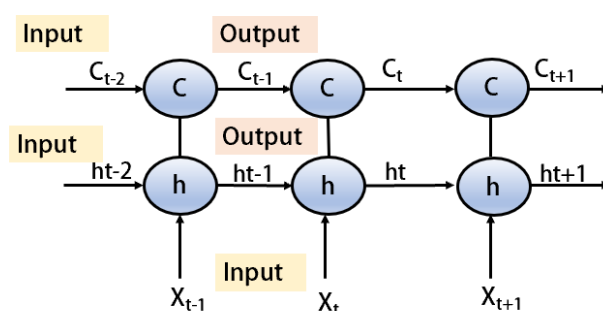


图 2.4 LSTM 内部结构

Fig. 2.4 Internal structure of LSTM

LSTM 网络在隐藏层加入 h_t 和 C_t ， C_t 称为单元状态，决定网络的遗忘程度。首先对 $t-1$ 时刻输出 h_{t-1} 和输入 x_t 做线性变换操作，之后用 sigmoid 激活函数，将结果映射到 0~1 得到记忆的衰减系数 f_t 。

$$f_t = \delta(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.1)$$

t 时刻学到的记忆，会对应一个衰减系数 i_t ，首先做 t 时刻的输入 x_t 及上一时刻的输出 h_{t-1} 对于输入矩阵 W_i 做线性变换，在加上输入门偏差矩阵 b_i ，之后通过 sigmoid 激活函数，映射到 0~1 之间。

$$i_t = \delta(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.2)$$

之后，获得当前学习到的记忆 $\tilde{C}_t$ ，通过线性变化和 tanh 激活函数得到的。

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.3)$$

最后计算 t 时刻下的记忆状态 C_t 。 t 时刻衰减系数 f_t 乘以 $t-1$ 时刻记忆 C_{t-1} ，之后，再加上 t 时刻记忆 $\tilde{C}_t$ 乘以衰减系数 i_t 。

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (2.4)$$

介绍输出门计算，首先计算衰减系数 o_t ，之后通过衰减系数和激活函数 $\tanh$ 相乘得到输出 h_t

$$o_t = \delta(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.5)$$

$$h_t = o_t * \tanh(C_t) \quad (2.6)$$

LSTM 网络通过三个门控制，实现网络长时记忆功能，比其他网络在文本学习方面更有优势。

2.3 Transformer 结构

2017 年谷歌研究团队在《Attention Is All You Need》文章中提出 Transformer 模型^[84]。Transformer 是一种新的注意力 Seq2seq 模型，使用 Self-Attention 机制进行计算输入和输出表示的转换模型。最初，该模型主要应用在自然语言处理领域，进行文本翻译。在文本翻译中，该模型不同于以往以 RNN 为基础的编/解码器^[85]，而是在编码部分采用并行机制，可以一次性输入长句子，利用注意力机制将距离不同的单词进行结合，缩短训练时间。随后，将 Transformer 模型引入到视觉领域，在图像特征提取、图像分类、目标检测等方面取得很好的发展，尤其在图像特征提取上，可以进行全局特征提取并考虑上下文相关的信息。

Transformer 模型主要是编码器-解码器结构。编码器和解码器采用栈结构进行数据存储，主要模块包括注意力机制网络、前馈神经网络、残差网络结构、归一化处理，如图 2.5 所示。

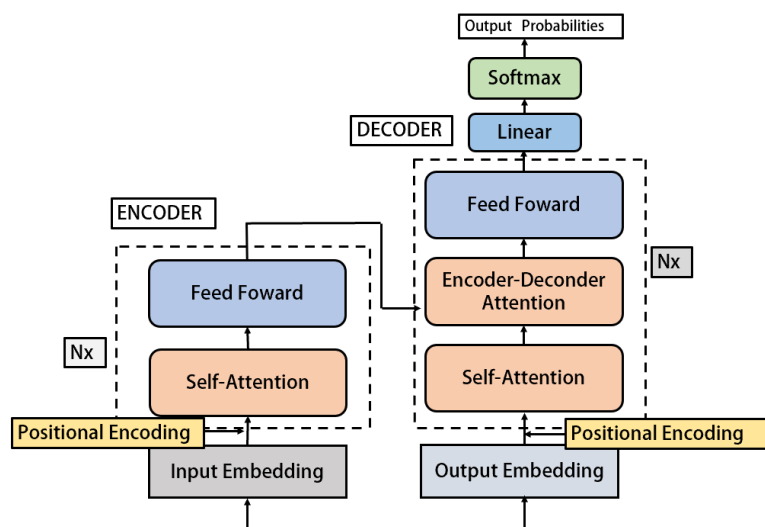


图 2.5 Transformer 体系结构
Fig.2.5 The Transformer Architecture

Transformer 模型存在 6 个 Encoder，每个 Encoder 包括两个网络结构，一个是自注意力机制网络，另一个是前馈神经网络。在输入位置，首先进行词编码，即 Input Embedding，将输入转化成机器识别向量，这一般采用 One-hot 编码方式，编码方式简单，通俗易懂。此模型比 RNN 优势在于可以一次性输入一句话，多个单词同时输入，这就造成位置不清楚。针对这个问题用位置编码进行标记，图中 Positional Encoding 就是位置编码，通常用如下公式计算，编码方式不唯一，但采用三角函数编码可以在未知序列上扩展。

$$PE_{(pos,2i)} = \sin(pos / 10000^{2i/d_{model}}) \quad (2.7)$$

$$PE_{(pos,2i+1)} = \cos(pos / 10000^{2i/d_{model}}) \quad (2.8)$$

其中， pos 代表单词位置， d_{model} 代表词向量维度，位置向量维度和词向量维度相同。位置向量第 i 维 $i \in [0, d_{model}]$ 。之后在 Encoder 中经过自注意力机制，按照公式 2.9 计算。

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.9)$$

关键点在于三个向量，通过三个向量的缩放点乘积来加强自注意力机制， Q 是查询向量、 K 是键向量、 V 是值向量。三个向量在注意力机制网络中，进行输入

训练，得到相应的矩阵 W^Q 、 W^K 、 W^V ，之后与词向量 X 做乘积，得到下一层网络的输入。

Decoder 解码器结构类似于 Encoder 结构，也是存在 6 个 Decoder 连用。Decoder 层将 Encoder 层输出 K 和 V 作为输入，和 Decoder 输入 Q 一起输入到 Attention 机制里，之后经过线性变换，在输入到 softmax 层，就可以预测下一个输出。这个框架在提取长距离特征上有优势。

Decoder 解码器增加一个网络结构，多头注意力网络，借此增加对位置的关注度，并且赋予每个位置矩阵子空间。多头注意力意味着对同一单词进行多次映射，每映射一次，该单词被赋予新的含义。通过空间分解，将一个大空间分解成 8 个小空间，每个空间处理的方式不同，该单词有新的意义，这样模型可以注意到子空间信息。

通俗来讲，就是找 8 个不同的人做这件事，比一个人做效果会好。如图 2.6 所示，多头注意力机制有多组 W^Q 、 W^K 、 W^V 矩阵，将传统注意力中的权重 W 分为多个子注意力，通过多层计算，得到更好的结果。

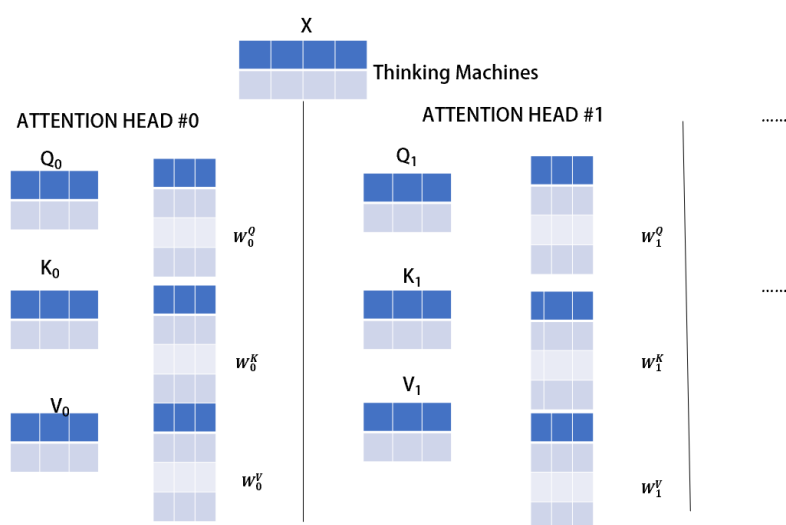


图 2.6 多头注意力权重矩阵

Fig. 2.6 Multi-head attention weight matrix

多头注意力机制，就是为每组注意力设置权重矩阵参数 W^Q 、 W^K 、 W^V ，分别于输入词向量 X 做矩阵乘法，得到 8 组 Q 、 K 、 V 矩阵，通过注意力计算得到 8

组 Z 矩阵，对这 8 个矩阵进行处理，拼接得到一个大矩阵，之后和权重矩阵 W^0 相乘做一次变换得到矩阵 Z ，将维度变为前馈神经网络，输入到网络中，进行训练分类。具体计算过程如图 2.7 所示。

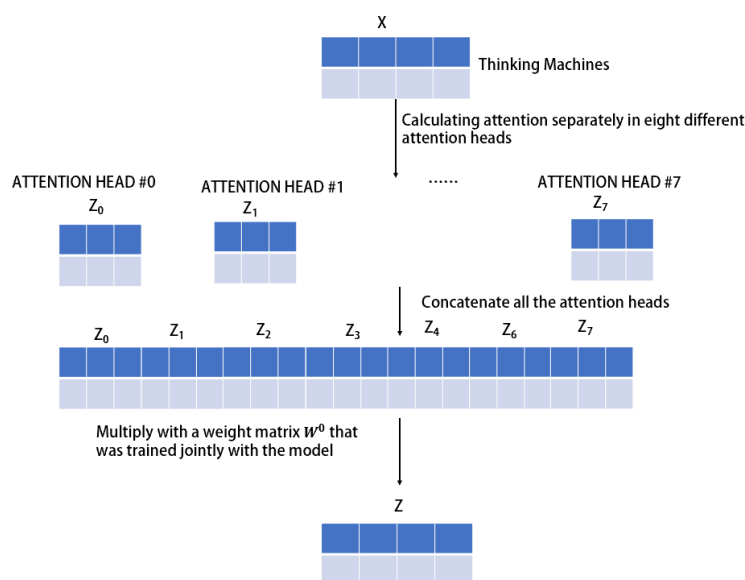


图 2.7 前馈神经网络输入矩阵计算

Fig. 2.7 Calculation of input matrix of feedforward neural network

在 Encoder 中，还有两个重要操作，残差连接和归一化。应对梯度消失的问题使用残差连接，在本质上，残差连接是将网络的输出与最初的输入相加。

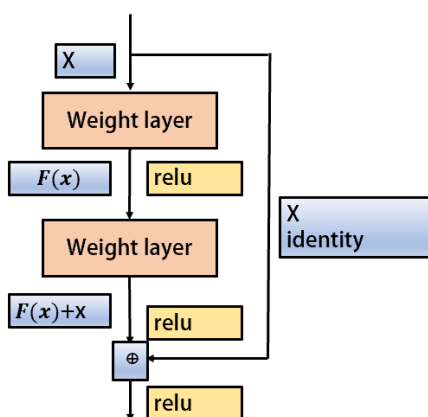


图 2.8 具有两层卷积网络的残差块

Fig. 2.8 Residual block with two-layer convolutional network

如图 2.8 所示，将 X 输入到网络，进行函数 $F(x)$ 计算，将 $F(x)$ 结果和 X 输入到下一层网络中。优化网络部分，就是让残差部分等于 0， $F(x) \rightarrow 0$ 。在使用梯度下降做优化时，随着网络深度增加，输入数据的特征分布会不断发生变化。为保证输入数据的稳定性，一般进行归一化处理，可以在学习率的较大情况下，加速模型的收敛速度。一般常用的方法有 BN（Batch Norm）和 LN（Layer Norm）两种，都是将输入特征转换成均值为 1，方差为 0 的数据，具体形式如下。

$$BN(x_i) = \alpha \times \frac{x_i - \mu_b}{\sqrt{\sigma_b^2 + \varepsilon}} + \beta \quad (2.10)$$

$$LN(x_i) = \alpha \times \frac{x_i - \mu_L}{\sqrt{\sigma_L^2 + \varepsilon}} + \beta \quad (2.11)$$

BN 对一个 batchsize 样本内的每个特征做归一化，LN 是对每个样本的所有特征做归一化。BN 平衡了不同特征之间的大小关系，保留不同样本之间的差异。在视觉处理领域，不同图片进行分类，使用 BN 可以更好的保留不同样本之间的差异。LN 平衡了不同样本之间的差异，保留一个样本中特征之间的差异性，比较适合自然语言处理方面。一个样本的特征就是不同的 word embedding，通过 LN 可以保留特征之间的时序关系。

在 Encoder 中，每层网络需要进行残差连接和归一化处理，之后输出如下：

$$Output = LayerNorm(x + SubLayer(x)) \quad (2.12)$$

其中， x 是输入序列。

2.4 本章小结

本章主要介绍代码漏洞相关知识和经典的深度学习神经网络结构，为后文模型搭建提供理论基础。首先对代码漏洞相关知识进行介绍，分析漏洞产生原因；之后介绍了深度学习中常用的几种神经网络结构，这些在代码漏洞检测模型搭建中发挥重要作用；最后，介绍在处理长文本序列有优势的 Transformer 模型。

第3章 基于注意力机制的代码漏洞检测算法

源代码漏洞出现的时间、位置毫无规律性，这对代码漏洞检测造成了一定的困难。一些传统方法可以进行代码漏洞检测，但存在一些弊端，比如过度依赖于专家经验、耗时、准确率低、代码覆盖率差等。自然语言在文本处理方面有着很大的优势，越来越多的人将文本分类技术应用在代码漏洞检测上，常用的文本分类网络有 CNN、RNN、LSTM 等。2017 年谷歌提出的 Transformer 模型，解决了 CNN、LSTM 等网络不能并行的缺陷，同时也能关联上下文语义信息，改变了传统模型的一般架构，它在自然语言处理中得到了广泛的应用，并取得良好的效果，但在代码漏洞检测领域应用较少。深度神经网络模型在代码漏洞检测方面的应用已取得不错的成果，但是也存在着检测效率低、分类不明确的问题。并且，随着深度学习的发展，在目标检测等机器视觉领域深度学习网络模型创新性较高，但在代码漏洞检测领域深度神经网络模块创新较低、网络结构较单一。

本章针对代码漏洞检测准确率低、网络结构单一等问题，对深度神经网络模型进行优化。首先，在源代码上应用程序切片思想，将关键代码提取出来，缩小数据集，减少数据冗余，降低算法复杂度；之后，结合 Transformer 模型和 BiLSTM 网络的优势，搭建基于深度学习的代码漏洞检测模型，利用多头注意力机制来加强深度学习模型漏洞检测能力，使用 Transformer 编码器进行全局代码语义特征提取，BiLSTM 网络进行代码特征学习与分类检测。最后，在 NVD 和 SARD 中上收集的 C/C++ 数据集中进行模型训练，通过实验证明该模型在代码漏洞检测领域是有效的。

3.1 系统架构

LSTM 网络输入是循环序列，在语言建模方面有着优势，但单向的 LSTM 不能很好的表示代码漏洞特征，因为产生代码漏洞的函数以及函数调用的参数出现的位置不确定，尤其是函数调用的参数，可能出现在函数前面也可能出现在函数后面，影响位置也不确定，所以本章使用双向的 LSTM 网络结构，可以学习代码特征以及上下文信息。在代码漏洞表示阶段，通常代码漏洞语义关系存在上下文中，单纯的学习单一漏洞点的词汇特征，可能会遗失一些重要信息，本章加入 Transformer 结构，可以一次性输入长语句，通过自注意力机制提取漏洞特征，实现并行运算以及遍历全文，提高代码漏洞特征提取的效率以及准确率。本章结合

两个网络的优势，搭建基于注意力机制的代码漏洞检测模型，通过 Transformer 编码器进行代码全局语义特征提取，之后通过 BiLSTM 模型进行双向语义学习和漏洞检测，达到全面学习代码漏洞特征，进而提高模型检测能力。最后，通过训练参数得到基于深度学习的代码漏洞模型。

模型的整体设计流程如图 3.1 所示，主要包括三个模块，分别是数据处理模块、向量转换模块以及本章搭建的深度学习检测模块。数据处理模块是进行数据预处理去除冗余信息，提高特征提取的准确度；向量转换模块是通过深度学习网络结构，把自然语言转化为机器可以识别的向量形式，并尽可能保留代码漏洞特征；Transformer 编码器模块主要是为了提取全局特征，实现并行计算，提高效率，为后面网络训练，提供更准确的数据，主要依靠 Self-Attention 机制发挥作用；BiLSTM 网络模块是采用带反馈信息的双向长短期记忆网络，更好的学习代码上下文关联信息。

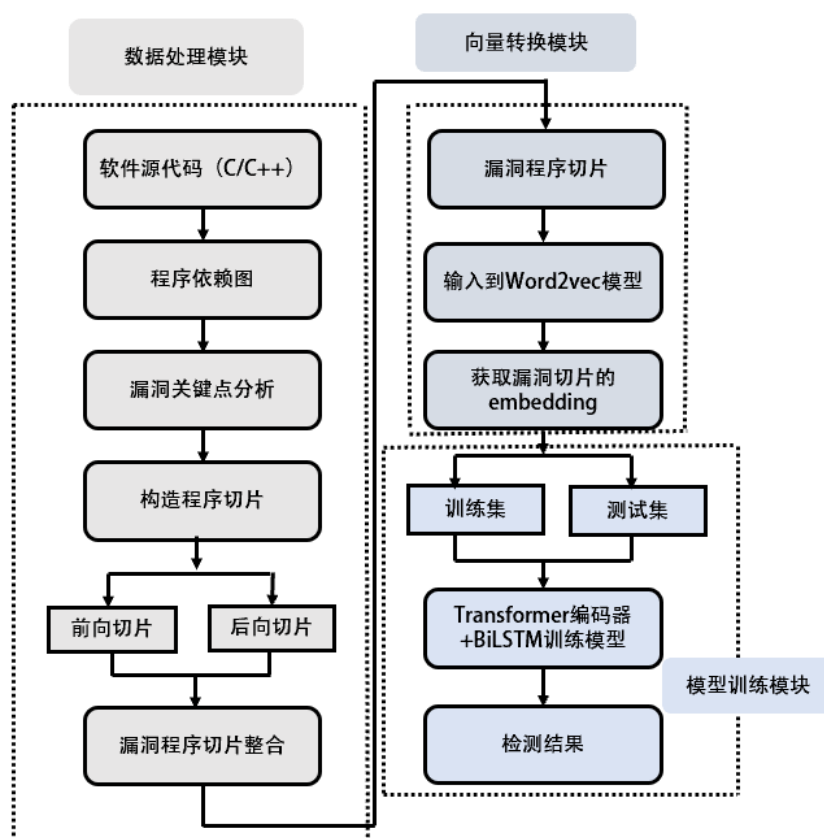


图 3.1 漏洞检测网络搭建流程

Fig. 3.1 Vulnerability detection network construction process

接下来，对这三个模块所用到的技术和网络搭建过程进行详细介绍。通过数据预处理模块和向量转换模块，可以得到对源代码处理后的数据集以及数据集的向量表示，如图 3.2 所示。之后，通过 Transformer 编码器中的自注意力机制提取全局代码特征，之后输入到 BiLSTM 网络中学习和训练，最后得到检测结果，模型搭建如图 3.3 所示。

(1) 数据处理

首先将注释等无关信息在源代码中删除，用Joern工具生成源代码的程序依赖图。根据源代码分析工具Checkmarx公开的漏洞原则，扫描全部代码，获取由于库函数调用，可能会产生漏洞的函数，将这些函数作为敏感函数，并以此函数为关键点利用程序切片算法获取与该函数相关的代码组成代码块。

(2) 向量转换

将源代码转换为神经网络可以识别的向量形式，便于输入到神经网络中。此处采用 Word2Vec 模型进行向量转换，Word2Vec 是无监督学习网络，可以综合相关语义信息，进行上下文特征提取，采用 CBOW 网络结构，将其映射成 300 维词向量矩阵。

(3) Transformer-BiLSTM 网络

将词向量矩阵输入 Transformer 编码器中，通过与位置编码相加输入到模型中，通过自注意力机制、前馈神经网络、残差连接学习特征向量，可以得到全局特征信息以及相关语义信息。之后输入到 BiLSTM 网络进行学习，通过双向 LSTM 学习漏洞前后文特征，通过分类器 softmax 计算样本分类的概率值，得到检测结果。

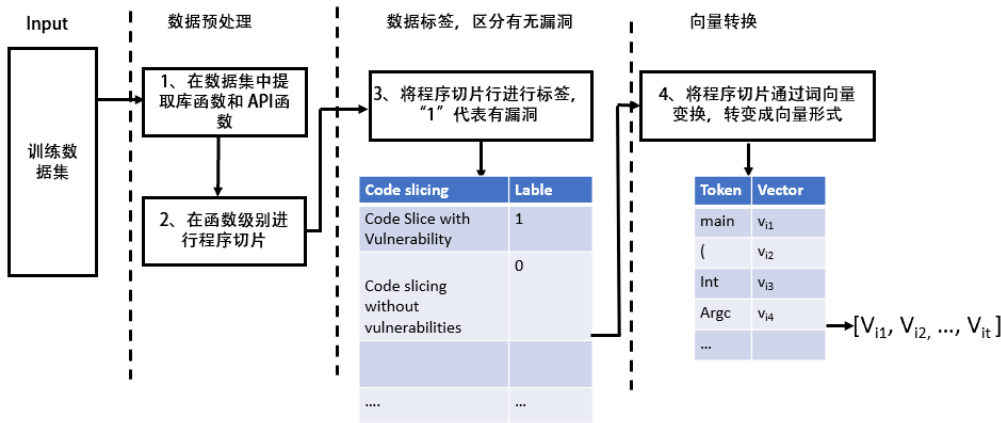


图 3.2 训练数据处理

Fig. 3.2 Training data processing

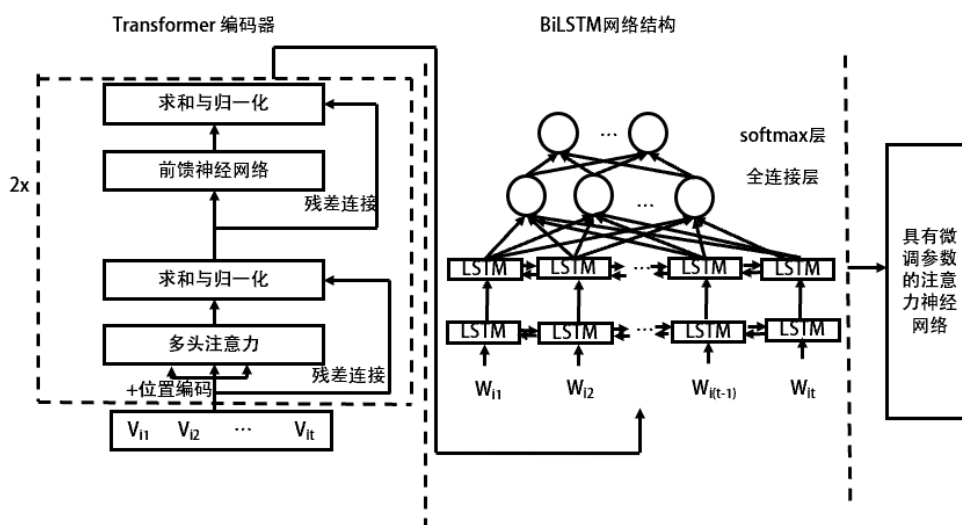


图 3.3 漏洞检测网络模块

Fig. 3.3 Vulnerability detection network module

3.2 模型构建

3.2.1 数据处理

理论上，源代码数据可以直接用来进行代码漏洞检测模型训练，但通常源代码中含有注释信息、文件信息、宏文件等与代码无关的冗余信息，这些信息无形中增加网络训练的负担，对代码漏洞检测准确率也有影响。此模块主要对代码进行预处理，将与代码漏洞相关的语句提取出来，减少代码冗余，提高特征提取的准确度。数据处理引用程序切片^[86]思想，以引起漏洞的函数为关键点，通过程序切片算法，将所有与关键点相关的语句筛选出来，去除其他不相关的信息，解决代码冗余问题。

首先，源代码通过 Joren 工具进行遍历，生成程序依赖图。之后，根据源代码分析工具 Checkmarx 中的漏洞原则，对全局代码遍历，获取调用库函数而可能产生漏洞的函数，并将这些函数作为敏感函数。并以此函数为关键点，利用程序切片算法遍历程序依赖图，获取该函数向前相关语句和向后相关语句^[87]，最终将向前相关语句和向后相关语句组合起来，获得该函数的程序切片。最后再判断敏感函数所在的语句是否产生漏洞，进行标记。因此，该方法将繁琐的源代码转化成较短的由相关代码行组成的代码切片，为后续网络训练提供较为准确的数据并

降低网络训练时间。接下来介绍一下程序切片相关的基本概念和定义，在此基础上提出程序切片算法以及生成程序切片的过程。

(1) 代码切片定义

首先，对代码切片相关定义进行说明。

定义1（代码切片）：在众多复杂的软件程序中，选定某一行代码为切入点，将与该行代码语义和语法相关的语句提取出来，组成一系列相关代码行，这就是程序切片思想。

定义2（控制流和节点）：控制流是指在代码中，通过控制关系相连接得到的语句。每条语句就是一个节点。

定义3（可执行路径）：在控制流图中，在某点 m 处，通过流程图的路径可以到达 n 点，就说这两个点之间存在可执行路径。

定义4（数据依赖）：如果存在一个变量 y 在节点 p 处被定义，之后在节点 q 处引用，并且这两个节点之间存在可执行路径，则节点 p 与 q 有数据依赖关系。

定义5（程序依赖图）：用二元组 (N, E) 表示， N 表示节点 $n_1, n_2, \dots, n_k$ 的集合，节点 $n_1, n_2, \dots, n_k \in N$ 。 E 是边的集合， $E = C \cup D \cup X$ 。 C 是依赖关系集合，如果存在 $CD(n_i, n_j)$ 时，则可把依赖关系 $CD(n_i, n_j)$ 添加到集合 C 中，其中 $1 \leq i, j \leq k$ 。

定义6（向前切片和向后切片）：在一段程序中，根据切片规则 $C = \langle N, V \rangle$ ，进行相关代码片段提取。节点 n 处变量 v 的变化可以影响的其他节点变量变化， $n \in N, v \in V$ ，这些节点处的语句集合称为向前切片；而影响节点 n 处变量 v 变化的语句集合称为向后切片， $n \in N, v \in V$ 。这两种切片方法基本上可以涵盖程序中某一节点相关的所有相关语句。

定义7（敏感函数）：在一段程序代码中，引起代码不规范的危险要素的语句或者库函数，称为敏感函数。例如，badSink()函数，不进行非0判断，直接进行除法操作。

(2) 代码切片过程

代码切片的主要思想是以一个节点为关键点，将与该节点相关的语句提取出来，构成代码块。在代码漏洞检测领域，首先，为一个程序源代码生成一个程序依赖图，将控制依赖和数据依赖关系在图中用带箭头的线表示出来，然后找到敏感函数所在的节点，通过程序切片算法，获取该节点切片集合。最后判断代码切片

中的函数是否包含漏洞,进行标签,有漏洞标为“1”,没有漏洞标为“0”。基于上述介绍,提出生成程度切片的具体算法,如算法 1 所示。

算法 1: 代码切片算法

定义: 程序依赖图: $G(N, E)$, 顶点 $n_i \in N$

可达路径 $(e(i, j)) \in E$, 敏感函数 $f_i \in F$,

含敏感函数的顶点 $n_{(f_i)} \in N_F$, 程序切片图 $P = P_1 + P_2$

输入: $G(N, E), N_F$

输出: P

```

1: function 向前切片( $P_1, n_{(f_i)}$ )
2:    $d \leftarrow n_{(f_i)}$ 
3:   for each  $n_i$  in  $N$  do
4:     if  $(e(n_i, d))$ 
5:        $P_1 \leftarrow P_1 \cup d \cup n_i \cup e(n_i, d)$ 
6:        $d \leftarrow n_i$ 
7:       向前切片( $P_1, d$ )
8:     end if
9:   end for
10: end function
11: function 向后切片( $P_2, n_{(f_i)}$ )
12:    $d \leftarrow n_{(f_i)}$ 
13:   for each  $n_i$  in  $N$  do
14:     if  $(e(d, n_i))$ 
15:        $P_1 \leftarrow P_1 \cup d \cup n_i \cup e(d, n_i)$ 
16:        $d \leftarrow n_i$ 
17:       向后切片( $P_2, d$ )
18:     end if
19:   end for

```

算法 1：代码切片算法（续）

20: end function

21: function main($G(N, E), N_F$)

22: $P_1, P_2, P \leftarrow \phi$

23: for each $n_{(f_i)}$ in N_F do

24: 向前切片($P_1, n_{(f_i)}$)

25: 向后切片($P_2, n_{(f_i)}$)

26: end for

27: $P \leftarrow P_1 \cup P_2$

28: end function

在图 3.4 中，说明算法实现过程。首先对源代码，通过 Joern 工具生成程序依赖图，用代码行号表示该行语句所在的节点，数据依赖用实线表示，控制依赖用虚线表示，通过实线和虚线将节点之间的关系标识出来，如图 3.4 步骤 1。之后，通过 Checkmarx 基于规则的源码分析工具收集可能产生漏洞的函数，将这些函数设置为敏感函数，在程序依赖图中获取敏感函数节点，如图 3.4 步骤 2。以敏感函数为关键点，通过算法 1 获取当前敏感函数的代码切片，如图 3.4 步骤 3 所示。之后对这些代码片段进行标记，如果该敏感函数包含漏洞语句，标记“1”，其它标记“0”，如图 3.4 步骤 4。通过上述过程，对源代码做了简化处理，降低代码冗余度，为后续网络训练做准备。

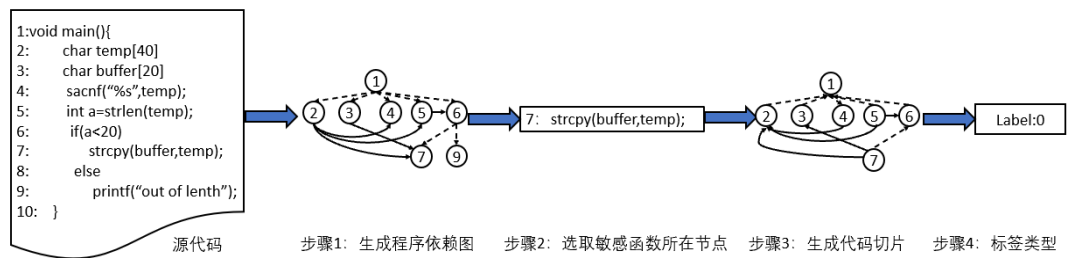


图 3.4 代码切片流程图

Fig. 3.4 Code slicing flow chart

3.2.2 向量转换

从数据处理模块，可以得到包含变量名、函数的一系列代码语句，不能直接送入网络中进行特征学习，需要转换成网络识别的数字模式，也就是词向量。词向量指将文本单词映射成由实数构成的稀疏向量，希望这个向量含有语义信息。从 C/C++模型中提取的代码切片包含多条语句，每条语句都是由词、间隔符组成。之后最重要的问题是怎么把这词正确转换为向量；除此之外，每条语句生成的词向量维度不同，如何统一生成的词向量维度也是一个重要问题。

离散式词表征方式将词表示成长向量，词表维度决定向量维度，长向量中只有一个维度值是 1，其它是 0。这有两方面缺陷，一个是词之间独立，忽略了语义信息，出现断层现象。另一方面随着词规模增加，词维度越来越大，向量中的 0 越多，造成稀疏性变大。分布式思想主要是通过统计模型和机器算法，实现自动学习语料上下文信息，能自动将语料信息映射成向量。本文采用分布式中较为成熟的 Word2Vec 模型。

Word2Vec 是基于深度学习方法设计的并且用来计算单词的密集向量表示的预测模型，这些表示是连续的，这有助于捕获语义和上下文特征。Word2Vec 是一种无监督架构，它将大量未标记的文本数据作为输入，生成所有单词的词汇表，并将这些单词转换为向量空间中的密集嵌入。该模型遵循的是，属于类似上下文词汇有共同的含义，它们的向量表示将是相似的。Word2Vec^[88]范式存在 CBOW 和 Skip-Gram 两种体系结构，主要网络结构由输入层、投射层、输出层组成，为加快训练过程使用 softmax 分类器和采样技术，具体结构如图 3.5 所示。

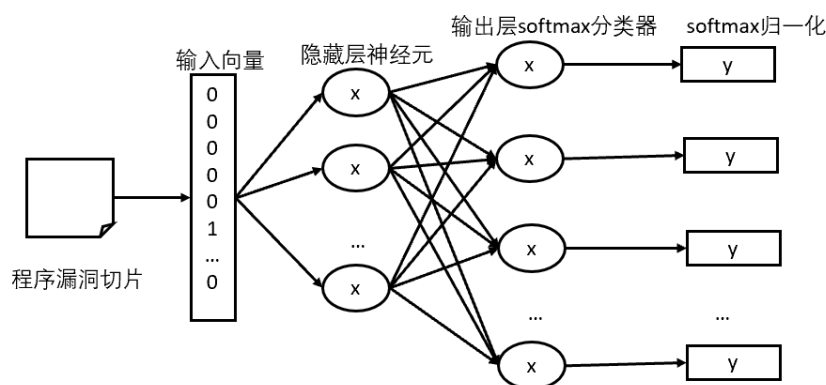


图 3.5 Word2Vec 网络结构

Fig. 3.5 Word2Vec network structure

如图 3.6 所示, CBOW 体系结构在提供带有相邻上下文单词的窗口预测目标单词, CBOW 预测不依赖于单词的顺序, 因此, 它忽略了单词顺序。Skip-gram 体系结构使用目标词来预测相邻上下文词的窗口, 对于不常见的单词, 其性能优于 CBOW。CBOW 算法是用上下文预测中心词, 在计算效率上更有优势, 而 Skip-Gram 算法是用中心词预测上下文, 在计算效率上较慢。因此, 本文采用计算效率更高的 CBOW 结构, 对代码切片进行处理, 转换成向量形式。

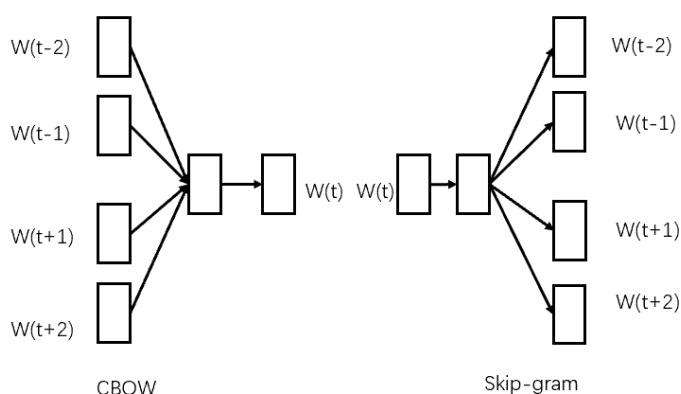


图 3.6 Word2Vec 模型结构

Fig. 3.6 Word2Vec model structure

为了提高词向量生成效率, 本文采用 CBOW 模型, 首先用 One-hot 编码上下单词, 之后输入到网络中, 与权重矩阵 W 相乘, 得到的结果输入隐藏层, 得到隐藏层输出向量 u_t 。网络结构如图 3.7 所示,

$$h_t = \frac{1}{2c} W \left(\sum_{-c \leq i \leq c, i \neq 0} W_{t+i} \right) \quad (3.1)$$

映射层各节点输出如下:

$$u_t = v_{wt}^T h \quad (3.2)$$

将隐藏层向量与输出矩阵 W' 相乘, 得到的向量输入到激活函数 softmax 中, 计算如下:

$$y_t = p(w_t | w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}) = \frac{u_t}{\sum_{j=1}^V \exp(u_t)} \quad (3.3)$$

通过公式 3.4 计算得到每个单词的概率分布。

$$L = \frac{1}{V} \sum_{t=1}^V \log p(w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}) \quad (3.4)$$

其中 V 是词汇量的数量。

之后，定义损失函数，采用梯度下降法训练 W 和 W' 。训练结束后，获取词向量查询表，得到权重矩阵 W 。之后，用 One-hot 对每个单词编码，再乘以权重矩阵 W ，得到每个单词的词向量矩阵。因此，代码切片中的每个词可以转换成神经网络输入的词向量矩阵形式，这就解决了源代码和词向量转换问题。

在代码切片生成词向量过程中出现维度不一致的情况，这导致生成的词向量不能直接输入到神经网络中进行训练。通过对代码切片的长度统计，平均长度 212，最大长度不超过 300。因此，将所有词向量统一成 300 维度，超过 300 维度的，在其尾部截取掉多余部分；不足 300 维度的，在其尾部填充“0”达到 300 维度。因此，就可以得到一系列统一长度的向量，之后输入到下一节的神经网络中进行训练学习。

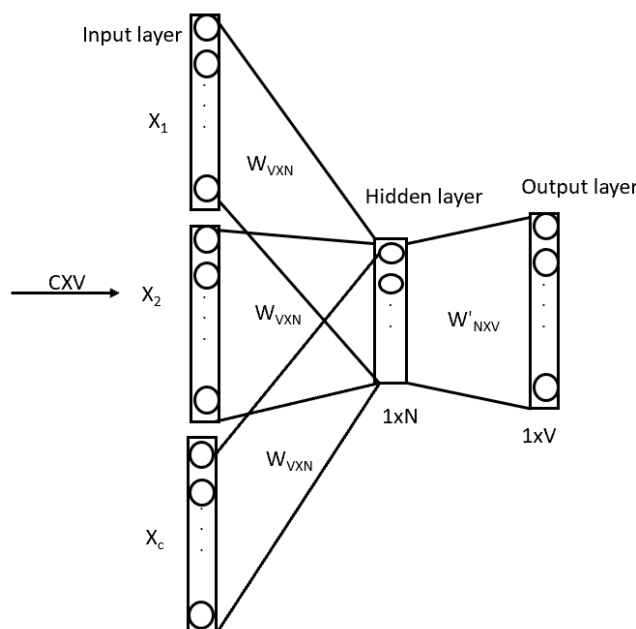


图 3.7 CBOW 的算法实现

Fig. 3.7 Algorithm implementation of CBOW

3.2.3 Transformer 编码器

在现有的 Encoder-Decoder 模型中，基本上网络结构是 CNN、RNN 网络搭建，而 Transformer 模型使用 Attention 机制和 Self-Attention 机制，采用完全的注意力机制，再加上全连接层实现整个框架。在运行时，可以并行处理输入的词向量，提高检测效率，降低算法复杂度，并且能全面获取特征信息，适合用于代码漏洞检测模型中。本章使用 Transformer 编码器对代码特征进行提取，搭建的框架如图 3.8 所示。

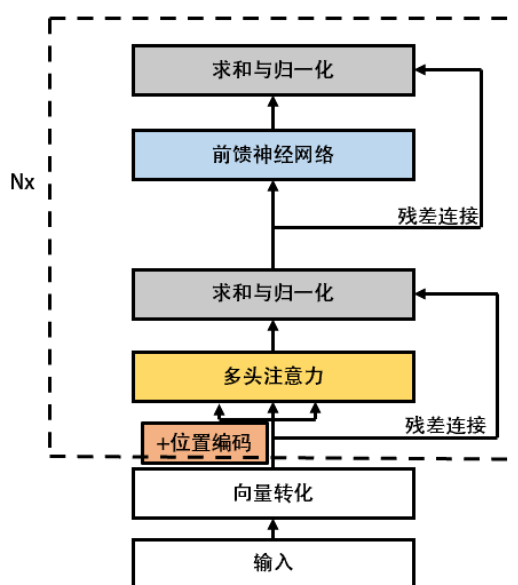


图 3.8 Transformer 编码器

Fig. 3.8 Transformer encoder

（1）整体结构

Transformer 编码器是通过叠加图 3.8 结构，可以进行多层套用，实现最优的编码方式。两部分组成该编码器，一部分是多头注意力网络，提取代码全局特征；一部分是前馈神经网络。每部分都进行归一化处理，输出使用残差连接处理。归一化处理是降低数据维度，防止出现数据爆炸；进行残差连接处理，是在求偏导过程中加入常数，避免出现梯度消失。前馈神经网络要经过全连接层和激活函数处理，计算公式如下：

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (3.5)$$

残差链接的过程是：

$$H(x) = F(x) + x \quad (3.6)$$

因此多头注意力层和前馈神经网络层经过以下计算得到输出：

$$y = \text{LayerNorm}(\text{Sublayer}(x) + x) \quad (3.7)$$

其中， LayerNorm 是归一化处理， Sublayer 是当前子层计算结果。

(2) 自注意力机制

模型对输入创立权重矩阵： Q 、 K 、 V ，将词向量与这三个矩阵相乘。自注意力机制网络主要计算 Q 到 Key-Value 的映射关系，公式如下：

$$\text{Attention}(Q, \text{Source}) = \sum_{i=1}^{Lx} \text{Similarity}(Q, \text{Key}_i) * \text{Value}_i \quad (3.8)$$

$Q = \text{key} = \text{Value}$ ， $Lx = |\text{Source}|$ ， Lx 大小是 Source 的长度。

自注意力是计算句子内部的映射关系，实现并行计算，提高计算效率，得到复杂度是 n^2d ，而 RNN 复杂度 nd^2 ，注意力机制的复杂度低于 RNN， n 代表句子长度， d 代表向量维度。另一方面自主注意力机制可以捕捉全文信息，优于 CNN 模型。

(3) 多头注意力机制

模型通过多头注意力机制增强对不同位置的句子提取信息的能力，计算公式如下：

$$\text{Attention}(Q, K, V) = \text{soft max}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.9)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^o \quad (3.10)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (3.11)$$

Q 代表被表达序列， K 代表表达序列， QK^T 计算 Q 和 K 的相似度，使用点积计算，导致这个乘积很大，在通过softmax分类时，结果超出 0~1 范围，所以通过 $\sqrt{d_k}$ 放缩，将结果放缩到 0~1。在使用多头注意力计算时，首先将序列分为 8 份训练不

同的权重(W_i^Q, W_i^K, W_i^V)得到新的映射 Q 、 K 、 V 的值,之后计算相应的注意力,按照分割方式拼接起来,再映射到原始数据的向量维度,就得到多头注意力的值。

多头注意力机制能捕获不同位置的信息,每个映射的 Q, K, V 是不同的,因此可以叠加多头注意力机制的头数提高模型实现捕捉长距离信息的能力。

(4) 位置编码

自注意力机制对于词之间的位置关系,无法识别。在点云计算中,由于点云数据中含有位置信息,可以忽视模型中的位置编码。但在漏洞检测模型中,数据集没有表示位置的信息,并且单词之间的位置和相对顺序对漏洞学习是有影响的,因此,在模型中需要加入位置信息。模型采用三角函数编码方式进行编码,因为基于三角函数编码方式具有可扩展性。计算公式如下:

$$PE_{(pos,2i)} = \sin(pos / 10000^{2i/d_{model}}) \quad (3.12)$$

$$PE_{(pos,2i+1)} = \cos(pos / 10000^{2i/d_{model}}) \quad (3.13)$$

其中, pos 代表单词位置, d_{model} 代表词向量维度。

3.2.4 BiLSTM 网络结构

长短期记忆网络结构非常适合处理序列数据。在代码漏洞检测方向,特征单词与上下文之间的关系影响漏洞检测的准确性,LSTM 网络能全面学习特征单词之间的关系,因此采用 LSTM 网络处理序列信息。LSTM 主要有三部分构成:输入门、遗忘门、输出门。输入门主要由控制单元组成,遗忘门主要是由记忆单元组成,可以保留以前网络训练的记忆,通过这三个门不断更新信息,从而保留想要的信息。

本章提出的 BiLSTM 网络具有多个 LSTM 层,叠加了两个方向的 LSTM,可以获取两个方向的语义特征信息。BiLSTM 网络结构如图 3.9 所示,主要包括三部分:双向 LSTM 部分,多个 LSTM 进行正向和反向传播学习;全连接层部分,主要是对 BiLSTM 层输出向量进行降维操作,避免信息爆炸.softmax 层部分,进行分类操作,得到分类结果,并在学习阶段,根据反馈结果更新网络参数。

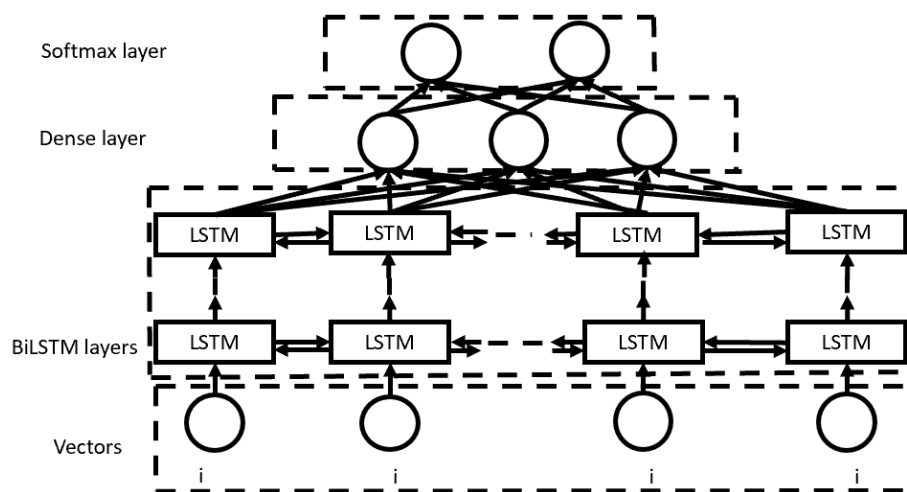


图 3.9 BiLSTM 网络结构

Fig. 3.9 BiLSTM network structure

3.2.5 基于注意力机制的 BiLSTM 网络

基于注意力机制的 BiLSTM 网络如图 3.10 所示。训练数据在模型中的主要计算过程如下：

本节在注意力机制上选取 Transformer 编码器，包括多头注意力机制、残差连接网络、前馈神经网络、归一化处理。首先对词向量 X_{input} 与位置编码相加得到输入 $X_{embedding}$ ，之后按照注意力机制分解，得到 Transformer Encoder 模块的输入，公式如下：

$$Q^{(h)}, K^{(h)}, V^{(h)} = XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)} \quad (3.14)$$

W_Q 、 W_K 、 W_V 为权重矩阵， $h \in [1, n]$ 为 head 索引。

经过多头注意力机制的计算结果与 $X_{embedding}$ 进行残差连接得到 $X_{attention}$ ，并进行归一化处理，得到标准正态分布，从而加速网络训练和收敛。之后，将 $X_{attention}$ 输入到前馈神经网络中，使用 ReLU 激活函数进行两次映射，对 $X_{attention}$ 进行维度的扩展和压缩，公式如下：

$$FFN(X) = \max(0, XW_1 + b_1)W_2 + b_2 \quad (3.15)$$

其中， W_1 、 W_2 、 b_1 和 b_2 分别对应权重矩阵和偏置。最后， $FFN(X)$ 和 $X_{attention}$

再进行一次残差连接和归一化处理，得到编码器输出 X_{hidden} 。

对编码器模块叠加时，底层编码器使用预训练字向量作为输入，其它编码器以上一层输出 X_{hidden} 为输入，并且计算过程中的 X_{input} 、 $X_{embedding}$ 、 $X_{attention}$ 和 X_{hidden} 有相同的维度，实现输入文本序列无监督学习与表征。

之后将 X_{hidden} 输入 LSTM 中进行代码特征学习，将 t 时刻正向 LSTM 输出和反向 LSTM 输出在全连接层进行拼接，得到本章模型的最终输出，并将提取到的特征映射为一维向量。接着在 softmax 层预测分类结果，得到正样本的概率，计算公式如下：

$$\sigma_k = \text{soft max}(z_k) = e^{z_k} / \sum^K e^z \quad (3.16)$$

最后通过网络模型输出，可以计算预测值和实际值的损失值，通过对比这两个损失值进行参数优化，采用交叉熵损失函数计算损失值，计算公式如下：

$$\text{Loss} = -\frac{1}{n} \sum_x [y \ln y_{pred} + (1-y) \ln(1-y_{pred})] \quad (3.17)$$

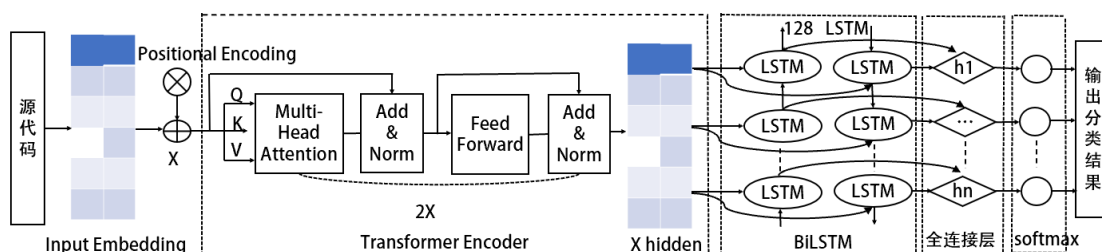


图 3.10 基于注意力机制的 BiLSTM 网络结构

Fig. 3.10 BiLSTM network structure based on attention mechanism

3.3 实验结果与分析

3.3.1 实验环境与数据集

(1) 数据集

数据集来源 NVD 和 SARD 项目。本文主要收集两类漏洞：缓冲区错误（CWE119）和资源管理错误（CWE399）。这些漏洞经常出现在程序中，因此可以收集足够多的数据进行深度学习训练。在这两个数据源中共提取了 33409 个

C/C++程序，通过数据处理，获取 61638 个代码切片，17725 个含漏洞的，43913 个不包含漏洞的。在包含代码漏洞的 17725 个代码切片中，10440 个代码切片包含缓冲区错误漏洞（CWE119），其余 7285 个包含资源管理错误漏洞（CWE399）。在这些数据中，随机抽取 80%数据用作训练集，20%数据用作测试集。

（2）实验环境

本章实验使用 Pytorch 及 Keras 等开源框架搭建网络，GPU 使用 NVIDIA GTX1080, CPU使用Inter i7,使用Pycharm2022 开发工具,借助于 Anaconda3(64-bit) 集成开发包来运行整个实验。主要实验配置如表 3.1 所示。

表 3.1 实验配置

Table 3.1 Experimental configuration

实验配置	参数指标
CPU 版本	Inter i7-8750HCPU@2.20GHz
GPU 版本	NVIDIA GTX1080
内存	16GB
操作系统	Linux 3.10.0-514.6.2.el7.x86_64
软件参数	Pytorch Keras Conda 4.7.12 CUDA 10.2

（3）参数设计

在搭建的模型上用训练数据集训练，并对网络参数微调，获取达到最优效果下的模型参数。具体参数如表 3.2 所示。

表 3.2 网络参数

Table 3.2 Network parameters

参数	值
激活函数	ReLU
Dropout	0.5
优化器	Adam
损失函数	交叉熵损失函数
学习率	0.002
Epoch	50

3.3.2 实验评价指标

在软件缺陷预测领域，经常使用以下指标评估系统。

(1) 准确率：

$$Accuracy = \frac{TP + TN}{All_test} \tag{3.18}$$

(2) 召回率：召回率越高表示模型找到漏洞样本能力越强。

$$TPR = \frac{TP}{FN + TP} \tag{3.19}$$

(3) 精确率：

$$P = \frac{TP}{FP + TP} \tag{3.20}$$

(4) *F1-score* 指标：

$$F1-score = \frac{2 \times P \times TPR}{P + TPR} \tag{3.21}$$

除了上述常用四个指标，使用二分类模型评价指标 *AUC* 对代码漏洞检测系统进行评价，*AUC* 值越接近 1 就代表着分类器模型的性能越好。*AUC* 指 *ROC* 曲线下的面积。*ROC* 曲线是以假阳性率（*FPR*）为横坐标，假阴性率（*FNP*）为纵坐标，公式如下：

$$FPR = \frac{FP}{FP + TN} \quad (3.22)$$

$$FNP = \frac{FN}{FN + TP} \quad (3.23)$$

理想情况下，漏洞检测系统既不漏报（ $FNR = 0$ ）也不误报（ $FPR = 0$ ），以及正确率极高接近 1 和 $F1 - score = 1$ 。这只是理想情况下，在实践中，通常损失一个指标换取另一个指标的有效性，在本章中，倾向于实现较高的准确率和较高的模型评价指标 $F1 - score$ 和 AUC 值。

3.3.3 实验结果分析

本小节首先对本章提出的基于注意力机制的代码漏洞检测模型进行性能评估与算法时间开销计算，并于基准模型进行对比；其次，与现有模型在模型性能指标方面进行对比。

（1）模型漏洞挖掘能力分析

本章搭建基于注意力机制的深度学习网络，采用含有多头自注意力机制的 Transformer 编码器对代码漏洞特征提取，之后输入到 BiLSTM 网络中通过双向 LSTM 学习漏洞特征，最后通过 softmax 进行分类。本章在多头注意力机制模块设置头数 8 个，使用两层 Transformer 编码器进行全局特征提取，之后在数据集 CWE119 和 CWE399 上训练和测试，获取最优模型。

在训练过程中，数据每读取一次，计算为一个 Epoch，通过训练次数不断迭代，在图 3.11 中可以看到 Loss 曲线逐渐收敛，在前 20 次训练过程中，梯度下降较快，之后下降缓慢，逐渐趋向于收敛。40 次之后，训练集的损失值在降低，而测试集的损失值在上升，说明进入过度拟合状态，模型性能不再提高。在训练次数 50 次时，模型准确率接近最大值，如图 3.12 所示，模型 F1-score 也达到最大值，如图 3.13 所示。此时得到最优检测模型，在 50 次训练后达到最好的分类效果。

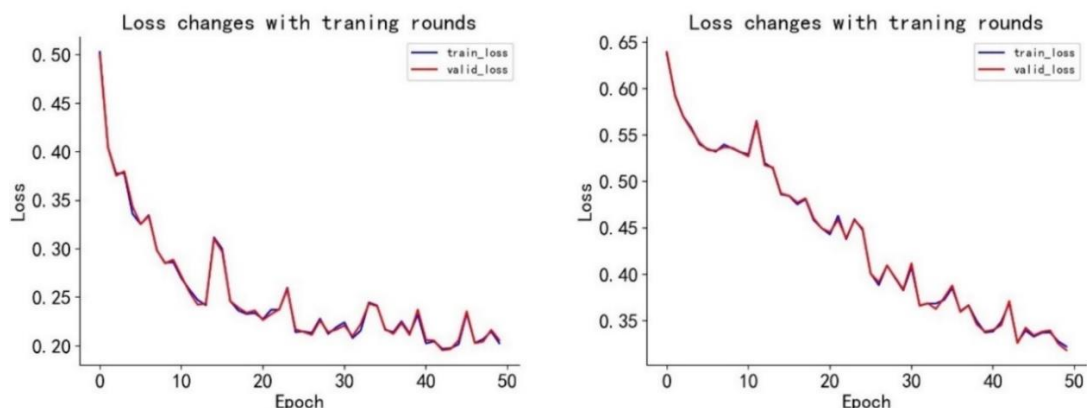


图 3.11 数据 CWE119 和 CWE399 Loss 曲线

Fig. 3.11 Data CWE119 and CWE399 Loss curve

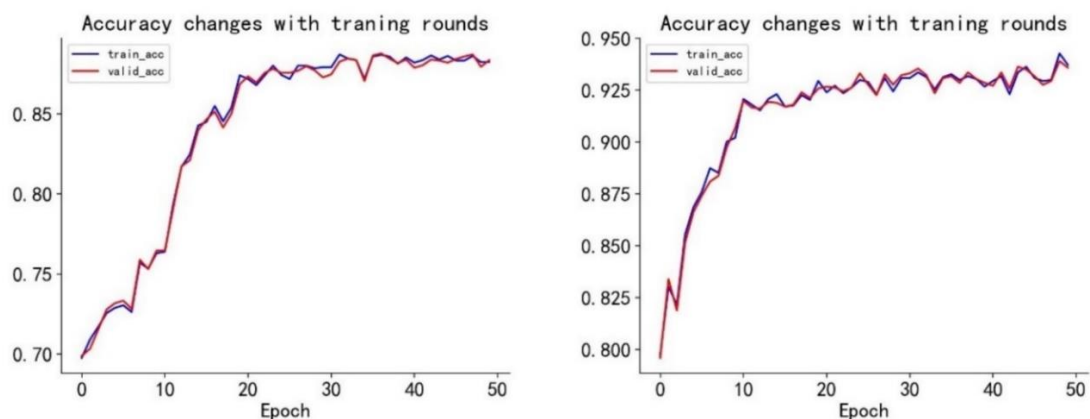


图 3.12 数据 CWE119 和 CWE399 准确率曲线

Fig. 3.12 Data CWE119 and CWE399 Accuracy curve

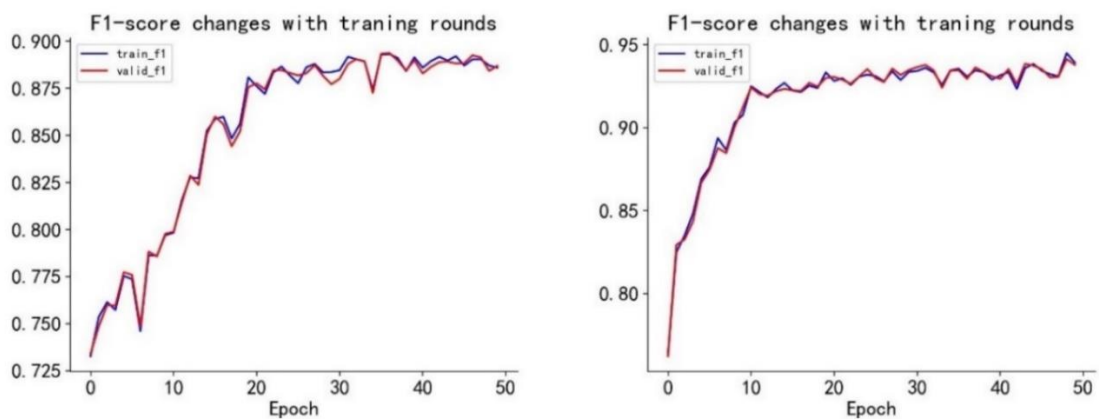


图 3.13 数据 CWE119 和 CWE399 F1-score 曲线

Fig. 3.13 Data CWE119 and CWE399 F1-score curve

之后，对基准模型 VulDeePecker 在本文处理的数据集上进行实验。VulDeePecker 模型^[41]网络结构是 BiLSTM，通过 softmax 分类器，得到分类结果。在本文中，将 VulDeePecker 模型中的 BiLSTM 网络设置成与本文相同的参数，具体设置如下：BiLSTM 网络单元数设为 128；批处理大小设为 16；输出尺寸为 256；使用 Adma 随机梯度下降法，学习率设为 0.002；序列长度为 300；Dropout 设为 0.5。在数据集 CWE399 和 CWE119 上对模型进行训练，通过 50 次迭代运算获取到模型 VulDeePecker 最优模型效果。数据训练集和测试集的 Loss 曲线、准确率曲线、F1-score 曲线如图 3.14、图 3.15、图 3.16 所示。

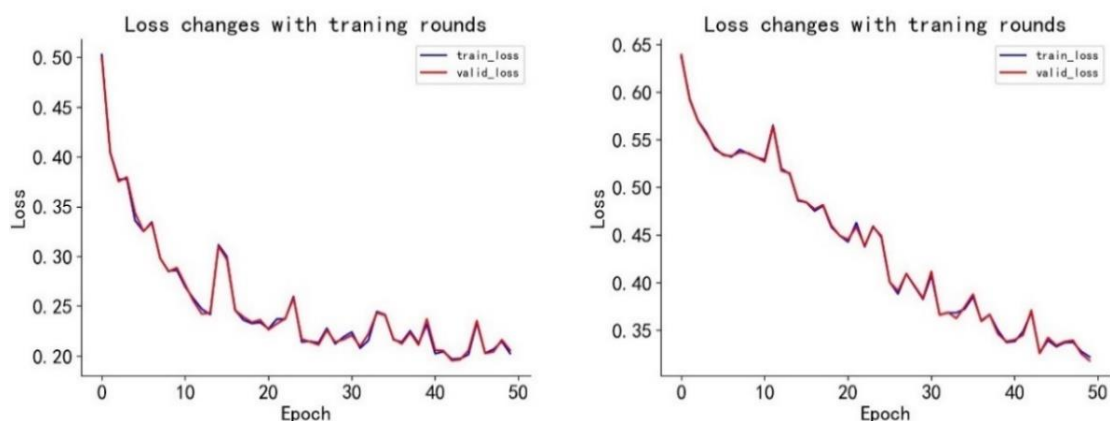


图 3.14 数据 CWE119 和 CWE399 Loss 曲线

Fig. 3.14 Data CWE119 and CWE399 Loss curve

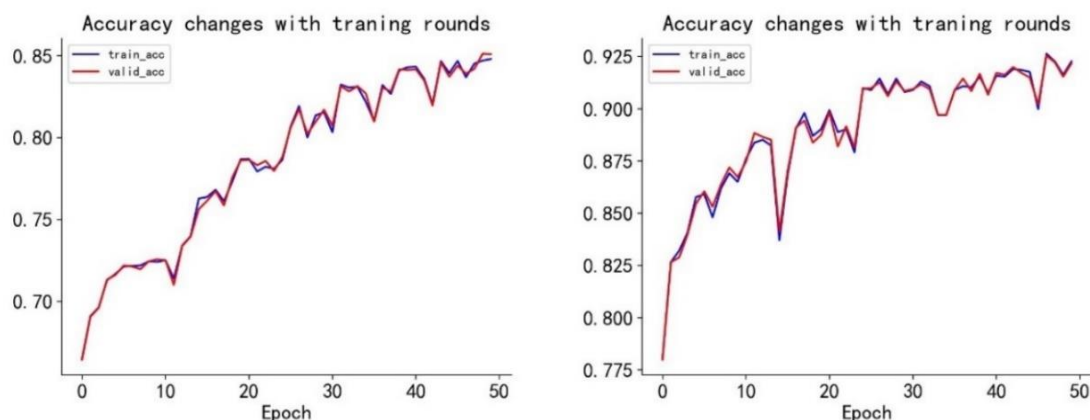


图 3.15 数据 CWE119 和 CWE399 准确率曲线

Fig. 3.15 Data CWE119 and CWE399 Accuracy curve

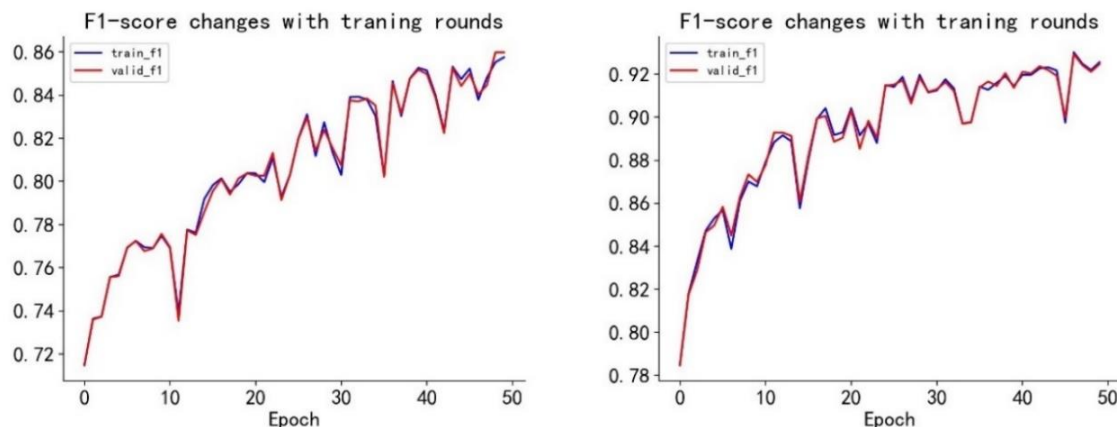


图 3.16 数据 CWE119 和 CWE399 F1-score 曲线

Fig. 3.16 Data CWE119 and CWE399 F1-score curve

观察到图 3.14，可以看到随着训练次数的增加，模型 Loss 曲线一直在梯度下降。在第 50 次位置，趋于拟合，并且在图 3.15 和图 3.16 中观察到准确率、F1-score 也趋向于最大值，说明训练模型达到最优。通过上述训练，得到最优模型。

将测试集输入到两个模型中，得到评价模型的各项指标参数，在数据集 CWE399 上，VulDeePecker 模型的准确率为 92.4%、AUC 值为 92.7%；本章模型的准确率为 94.0%、AUC 值为 93.0%。在数据集 CWE119 上，VulDeePecker 模型准确率为 86.6%、AUC 值为 84.8%；本章模型的准确率为 88.0%、AUC 值为 88.0%，具体性能指标如表 3.3 所示。

表 3.3 本章模型实验结果和结果对比

Table 3.3 Experimental results of the model in this chapter and comparison of results

模型	召回率	精确率	F1-score	准确率	AUC
本章模型(CWE119)	93.0%	83.7%	88.0%	87.4%	88.0%
baseline(CWE119)	94.4%	81.6%	87.5%	86.6%	84.8%
本章模型(CWE399)	97.0%	91.1%	94.0%	93.7%	93.0%
baseline(CWE399)	96.1%	89.5%	92.7%	92.4%	91.3%

观察表 3.3 可知，针对数据集 CWE399，网络检测漏洞代码的准确率是 93.7%，提高 1%；F1-score 为 94%，提高 1.3%。针对数据集 CWE119，网络检测漏洞代码的准确率是 87.4%，提高 0.8%；F1-score 为 88%，提高 0.5%。并且，模型分类性

能指标 AUC 值都有所提高，在数据集上 CEW119 上提高 3.2%，在数据集 CEW399 上提高 2.7%，同时召回率和精确率上也有提升，通过对比模型检测指标，可以说本章模型可以用于代码漏洞检测领域，并比基准模型更适合代码漏洞检测领域。

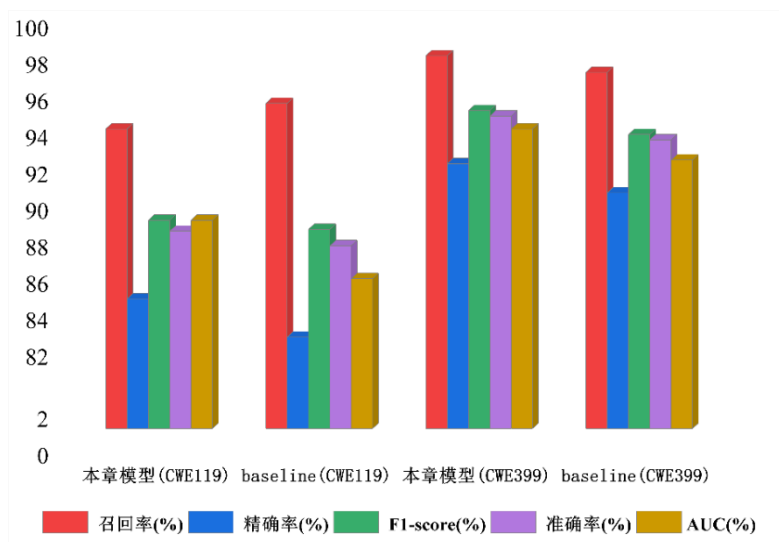


图 3.17 两个模型实验结果对比图

Fig 3.17 Comparison of experimental results of two models

在图 3.17 中对比模型评价指标，可以更加直观地看出，在加入 Transformer 编码器的 BiLSTM 模型中，检测代码漏洞的准确率达到 93.7%和 87.4%，说明模型可以正确检测到一定的代码漏洞，并且模型评价指标 F1-score 值达到 94.0%和 88.0%，说明该模型在漏洞检测上是有效的。并且模型检测漏洞的准确率、精确率、F1-score 和 AUC 值都比基准模型提高了，说明该模型分类性能比原有网络更好，注意力机制提高代码特征提取能力，增加模型分类准确性，提高模型性能。本章模型更适用于代码漏洞检测领域。

此外，深度学习算法时间开销也是模型评价指标之一，在表 3.4 中记录了本章模型和 VulDeePecker 模型的时间开销。这个时间包括两部分，一部分是训练模型时间，一部分是模型检测时间，模型训练时间指将代码表示成向量形式以及训练神经网络的时间，模型检测时间是指将代码表示成向量形式以及神经网络检测程序是否存在漏洞的时间。本章使用训练模型时间当做神经网络主要开销时间，主要是因为漏洞检测模型中，主要耗时在网络训练过程中，其次是将源代码转化成神经网络可识别形式，而模型检测时间开销相对较小。

表 3.4 中的实验结果表明,与 VulDeePecker 相比,本章提出的基于 Transformer 注意力机制的模型显著降低了时间开销。从时间开销方面证明本章模型优于 VulDeePecker 模型,也证明了 Transformer-BiLSTM 网络更适合代码漏洞检测领域,主要因为注意力机制加快网络收敛,提高网络训练速度。通过表 3.4 可以观察到,在数据集 CWE119 上,训练时间和检测时间统计,VulDeePecker 的分别为 1776.87s 和 27.99s,本章模型分别为 1050.90s 和 15.12s。在数据集 CWE399 上,VulDeePecker 分别为 1244.06s 和 17.14s,本章模型分别为 804.60s 和 8.37s。数据集 CWE119 包含的代码切片比数据集 CWE399 中的要多,可以观察到模型的训练时间和检测时间会随着代码切片的增加而有所增加。总体来说,本章模型在两个数据集上都减少了时间开销,注意力机制加快网络收敛,降低模型训练时间。在算法时间开销方面,本章模型比 VulDeePecker 更有优势。

表 3.4 本章模型检测漏洞的时间开销和结果对比

Table 3.4 This chapter compares the time cost and results of model detection vulnerability

Model	数据集	训练时间 (s)	检测时间 (s)
VulDeePecker	CWE119	1776.87	27.99
	CWE399	1244.06	17.14
本章模型	CWE119	1050.90	15.12
	CWE399	804.60	8.37

(2) 与其他漏洞检测模型对比

为进一步评估本章模型用于漏洞检测的有效性,本小节选取几个不同代码漏洞检测模型与本章模型进行比较,通过对比模型的准确率、召回率、精确率和 F1-score,说明本章模型在代码漏洞检测领域有优势。

Flawfinder^[89]和 RAST^[90]都是基于规则的漏洞检测工具,它们通过一个内置的 C/C++ 公共漏洞数据库来匹配目标程序中的漏洞模式,根据专家经验定义相匹配的漏洞检测规则,实现代码漏洞检测。SySeVR^[42]是基于深度学习的检测模型,对语义和语法结构单独考虑,最后生成相关代码切片进行模型训练,主要检测库/API 函数相关的漏洞。VUDDY^[91]是针对代码克隆,提出的一种基于相似度方法的漏洞检测模型。BGNN4VD^[92]使用图神经网络模型进行代码特征提取以及模型训练,通

过构建双向图神经网络进行漏洞检测。由于图结构的复杂度较高，在图结构上表示代码特征较困难，并且网络模型计算消耗较大。

在数据集 CWE399 上采用上述实验方法进行模型训练与检测，得到准确率、召回率、精确率和 F1-score，如表 3.5 所示。

表 3.5 本章模型与其他模型实验结果

Table 3.5 Experimental results of the model in this chapter and other models

Method	F1-score	准确率	召回率	精确率
Flawfinder	28.6%	51.9%	19.9%	50.5%
RATS	19.3%	48.8%	14.8%	27.9%
VUDDY	23.1%	72.9%	15.7%	43.3%
SySeVR	83.2%	87.3%	85.2%	81.3%
BGNN4VD	83.8%	84.7%	86.3%	87.3%
本章模型	94.0%	93.7%	97.0%	91.1%

与 BGNN4VD 相比，本文方法的 F1-score 测量值提高 10.2%，准确率提高 9.0%，召回率提高 10.7%，精确率提高 3.8%。与 SySeVR 相比，本文方法的 F1-score 测量值提高 10.8%，准确率提高 6.4%，召回率提高 11.8%，精确率提高 9.8%。这归因于 Attention 机制，可以学习代码语义上的关联关系，在检测代码漏洞时，与漏洞点相关的语句关注力更强，网络对此处的学习力度更大，进而可以提高模型检测漏洞的准确率和召回率。

此外，SySeVR 中使用了 RNN 网络结构来训练检测模型。由于 RNN 在处理基于图的数据的非顺序特征时性能较差，一些重要特征可能被忽略或屏蔽。相比之下，本章使用双向长短期记忆神经网络在学习基于语义相关的数据特征和数据双向传输学习上下文信息方面更加有效。此外，发现 Flawfinder 和 RATS 的准确率可以达到 50% 左右，但在其他方面却表现不好。这是由于静态分析工具的局限性，通过专家定义代码漏洞模式，并通过词法分析在目标程序代码中对漏洞代码模式进行匹配从而挖掘漏洞，但预定义的模式较为单一，其难以覆盖复杂多变的真实环境，导致了较高的误报率。类似地，VUDDY 在代码相似度模型上检测漏洞，检测的漏洞类型有局限，并且它的召回率和准确率也很低（分别为 15.7% 和 43.3%）。

为了更加直观地看出各个模型的性能对比效果，将 6 个模型的准确率、召回率、精确率和 F1-score 值用直方图表示，如图 3.18 所示。可以直观看出，与这些方法相比，本文方法在代码漏洞检测模型上的准确率、召回率和 F1-score 相对较高，说明本章模型，在检测资源管理错误和缓冲区漏洞方面比其他模型的检测能力好。

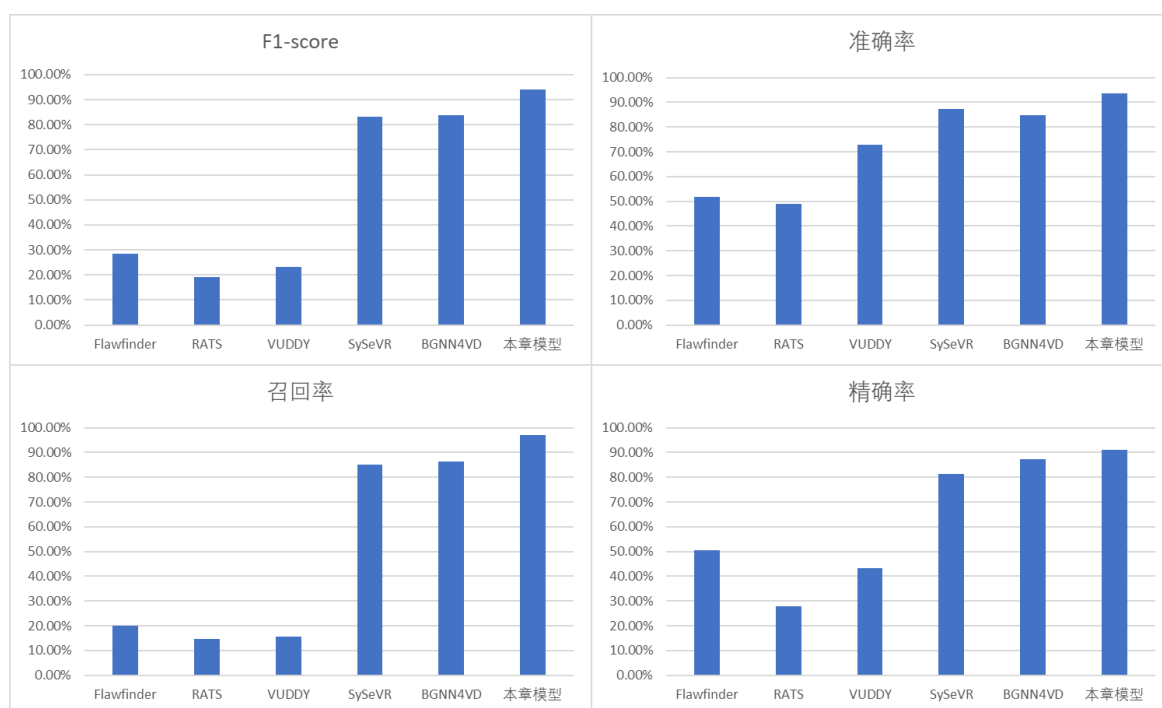


图 3.18 模型实验结果对比图

Fig 3.18 Comparison of experimental results of models

3.4 本章小结

本章针对代码漏洞检测模型准确率低、网络结构单一等问题，改进深度神经网络模型结构，提出一种基于注意力机制的代码漏洞检测模型。首先，对源代码进行数据处理，利用程序切片算法，提取关键代码行，精简数据，为后续代码特征提取和神经网络训练提供准确的数据集，进而提高模型的检测能力；之后，结合 Transformer 模型和 BiLSTM 网络的优势，搭建基于深度学习的代码漏洞检测模型，利用 Transformer 编码器中的多头注意力机制进行代码全局特征提取，使用 BiLSTM 网络进行学习和分类检测；最后，在数据集 CWE119 和 CWE399 上分别

进行模型训练，进行模型参数微调，获取较优模型，然后在模型上通过测试集进行测试，获取模型评价指标值。通过实验评定该方法在代码漏洞检测上的有效性并能够提高漏洞检测的准确率，之后计算模型训练神经网络时间作为算法时间开销，本章模型时间开销明显低于 VulDeePecker 模型，说明本章模型比 VulDeePecker 模型更适合代码漏洞检测。为进一步说明本章方法有效性，将其与目前现有的代码漏洞检测方法相比较，本章方法在代码漏洞检测上更有优势。

第4章 基于位置编码优化的代码漏洞检测算法

在文本分类领域，文本信息有序输入 RNN 和 CNN 模型中，因此在这些模型中本身就包含位置信息，不用考虑缺少位置信息对模型分类效果产生的影响。Transformer 模型是完全的 Attention 机制网络，丢失词序位置信息，因此，在模型输入位置加入位置编码（Positional Encoding），通过这种方式确定语句中词的位置。

在普通的文本分类任务中，传统的编码方法可以解决定位问题，但是它们不适用于软件缺陷检测模型中，因为传统的编码方法没有考虑到上下文的语义信息。代码中的漏洞与上下文有密切关系，并且在本文中基于函数级别进行代码漏洞检测，与漏洞点相对距离越近，代码语义关联越强，此处应该提高注意力权重。其次，代码顺序变化可能并没有引起语义信息的改变，这种语义相同顺序不同的代码，认为在逻辑上是相同的，在进行位置编码时应该模糊这种距离差距带来的影响。本章在 Transformer 编码器中的位置编码模块采用基于相对距离的位置信息编码方式，该编码方式考虑位置相对距离问题，相对位置距离越小，认为代码关联度越大，此处的编码应该密集，进而让模型更加注意这附近词汇对漏洞产生的影响。通过这种方式，为每个词嵌入位置信息，之后进行自注意力特征提取。这种特征提取方式，可以识别这种语义信息上的关联，增强数据的关联性，进而提高检测模型的性能。

4.1 模型构建

文本分类领域中，经常使用的模型是 RNN 和 CNN 模型。在模型中有序输入各种文本信息，因此这些模型本身就包含位置信息，但存在不能并行计算、耗时、空间爆炸等问题。随后，提出 Self-Attention 机制改进，解决并行计算问题，考虑到不同词之间的句法特征和语义特征，但丢失词序信息。对于文本信息，词之间的句法特征和语义特征影响词向量转换，进而影响后续分类任务。Transformer 模型解决这个问题，将词向量与位置编码（Positional Encoding）结合，再进行注意力计算，考虑了位置信息对文本分类任务的影响。在代码漏洞领域，单词位置信息对漏洞特征提取有至关重要影响。因此，本章主要研究加入位置编码的注意力机制与单独使用注意力层在代码漏洞检测模型上的检测效果；其次，考虑到代码

语句的特殊语义信息，将不同代码行之间的位置相似度加入到位置编码中，借此提高代码漏洞模型检测的准确率和模型评定指标 F1-score。

4.1.1 基于多注意力机制的 BiLSTM 网络

目前提出的注意力机制有 Self-Attention 机制、MultiHead Sef-Attention 机制、以及结合自注意力机制、残差连接、前馈神经网络的 Transformer 模型。在代码漏洞检测领域，漏洞出现的位置对代码语义特征向量表示的影响将会对漏洞检测结果产生影响，因此，本小节在 BiLSTM 网络中分别加入 Self-Attention 模块、MultiHead Sef-Attention 模块和 Transformer 编码器模块，搭建三个基于注意力机制的代码漏洞检测模型，在数据集上对模型训练和测试，分别得到模型的召回率、准确率、精确率和 F1-score。模型搭建流程如图 4.1 所示，首先，对源代码进行数据处理，在函数级别进行代码切片处理，得到更小的检测粒度（代码切片），之后输入到 Word2vec 模型中进行向量转换，这两个过程在第三章已经介绍了，这里不再赘述。将得到的关于源代码的向量表示，分别输入到三个基于注意力机制的网络模型中，最后得到检测结果和模型检测指标。

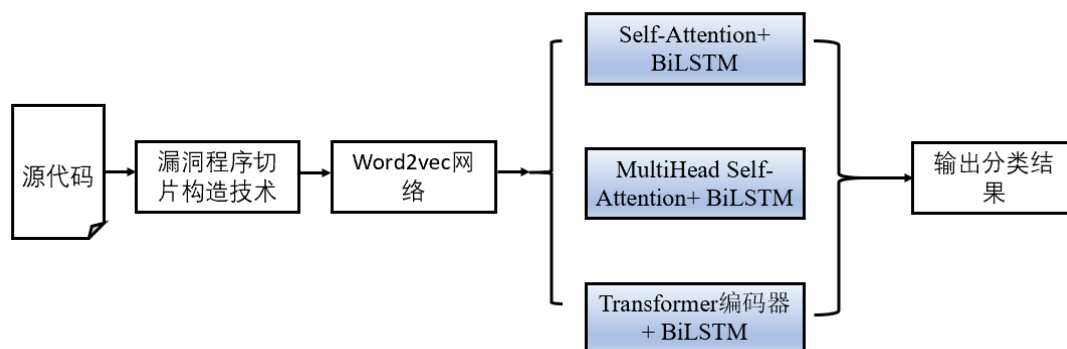


图 4.1 注意力模型构建流程图

Fig. 4.1 Flow chart of attention model construction

基于 Self-Attention 的 BiLSTM 网络结构主要包括 5 个神经网络层，分别是输入层、BiLSTM 层、注意力层、全连接层和输出层，如图 4.2 所示。输入层接受固定长度的向量形式，在第 3.2 节中，我们详细介绍了如何将程序源代码转换为输入到神经网络的向量形式。首先提取库函数调用，生成相应的程序切片，通过 Word2vec 转换为向量形式，并将所有向量删除或填充到固定的长度。注意力层可以分析 BiLSTM 层输出的向量，自注意力机制会计算每个输入和所有输入之间的

关联性（即注意力分数），注意力分数越高说明这两个信息之间的依赖关系越强，从而可以更好地捕捉输入向量中重要的信息。全连接层设置了 Dropout 防止神经网络过拟合，而 softmax 层的作用是对结果进行分类。

基于 MultiHead Self-Attention 的 BiLSTM 网络结构和基于 Self-Attention 的 BiLSTM 网络结构相似，也是 5 个部分，区别只在于注意力层，采用多头注意力机制，允许注意力机制组合使用 Q 、 K 和 V 的不同子空间表示。关于自注意力和多头注意力机制的模型和理论计算在 3.2.3 节进行了介绍，这里不再赘述。

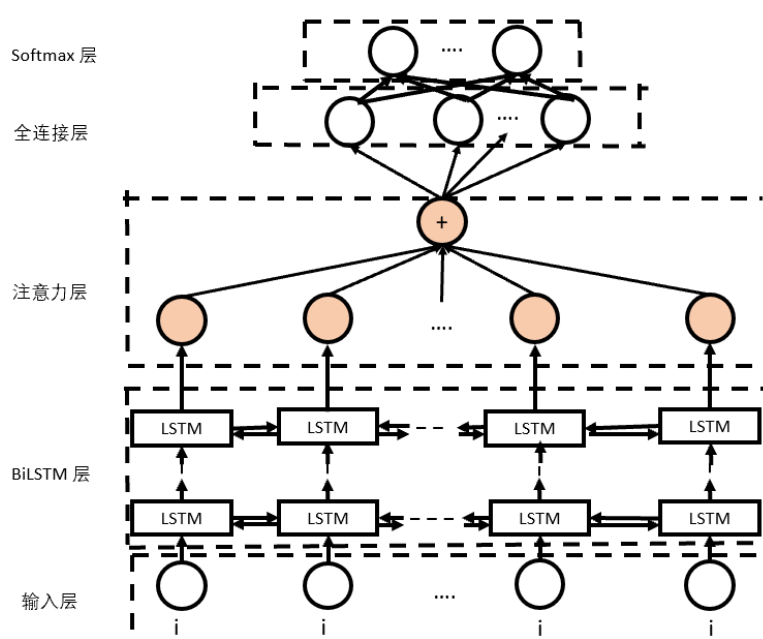


图 4.2 基于注意力机制的 BiLSTM 神经网络结构

Fig 4.2 Structure of BiLSTM neural network based on attention mechanism

4.1.2 位置编码模型

本小节探究位置编码方式对基于注意力机制的代码漏洞检测系统的影响。Transformer 编码器中的位置编码解决 Self-Attention 并行输入长文本信息中造成的位置信息丢失问题，常用的编码方式有 One-hot 编码，用寄存器对位置信息编码，编码维度较大，表示离散关系，不能体现关联关系。绝对位置编码可以表示各个位置不同，但不能明确表示单词的先后顺序。相对位置编码，关注一个 token 与另一个 token 距离的相对位置，给定 k ，存在一个固定的与 k 相关的线性变换矩阵，

从而由 pos 的位置编码线性变换而得到 $pos+k$ 的位置编码，可以表示单词的先后顺序。软件源代码不同于普通文本，普通文本调换单词位置影响语义信息，而代码编写会出现因个人习惯而更改代码位置但逻辑上是相同的，针对代码顺序变化可能并没有引起语义信息的改变的问题，提出基于相对距离的位置编码方式，模糊距离差异带来的影响。该编码方式以敏感函数为中心，计算每个词汇与敏感函数相对位置距离，相对位置距离越小，代码关联度越高，编码越密集，模型对此处的关注力应该越高。

在 Transformer 编码器中应用不同的编码方式提取代码漏洞特征，之后输入到 BiLSTM 网络中，对模型效果进行分析，模型检测流程如图 4.3 所示。首先对源代码进行处理，得到神经网络模型可识别的 embedding，具体过程在 3.2 小节中。之后输入到 Transformer 编码器中进行代码特征提取，Transformer 编码器中的 Positional Encoding 模块采用不同的编码方式，具体编码公式在 4.2 节介绍，之后得到一系列含注意力的 token。再通过 BiLSTM 网络学习，softmax 分类器分类，得到最终检测结果以及模型一系列评价指标值。

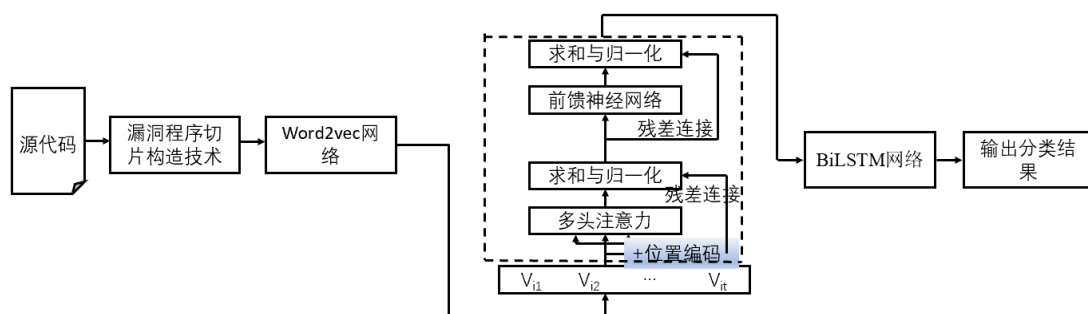


图 4.3 不同编码方式的模型检测流程图

Fig 4.3 Flow chart of model detection with different encoding methods

4.2 位置编码算法

4.2.1 One-hot 位置信息编码

One-hot 编码，又称独热编码。其方法是使用 N 位状态寄存器来对 N 个状态进行编码。使用 One-hot 进行位置编码时，先对位置进行索引编码，对 $h = \text{"int } m; m=5;"$ 这一组代码语句根据 One-hot 编码规则，进行如表 4.1 所示编码。

表 4.1 One-hot 编码
Table 4.1 One-hot coding

word	int	m	;	=	5
index	0	1	2	3	4

根据索引位进行位置编码，生成位置向量，索引为对应“1”，其他对应“0”。表 4.1 中的词句对应的位置编码形式如表 4.2 所示。

表 4.2 One-hot 位置编码
Table 4.2 One-hot location code

word	int	m	;	m	=	5	;
index	0	1	2	1	3	4	2
One-hot	[1,0,0,0,0]	[0,1,0,0,0]	[0,0,1,0,0]	[0,1,0,0,0]	[0,0,0,1,0]	[0,0,0,0,1]	[0,0,1,0,0]

One-hot 编码在处理离散数据有优势，但这种编码方式维度是整个词表的维度，维度比较大。并且编码方式属于词袋模型，是在词相互独立的情况下进行编码，不考虑词之间的顺序，得到的特征是离散的，相对稀疏的，词之间的关联关系体现不出来。

4.2.2 绝对位置信息编码

绝对位置编码通常将位置编码直接融入到词向量中，形式简单：第 k 个向量 x_k 根据位置编码加入位置信息 p_k ，变成 $p_k + 1$ 。最简单的编码形式，不通过公式计算，将位置向量输入到网络进行训练。如果一个向量是 N ，位置向量维度是 D ，则编码维度是 $N \times D$ 的矩阵，之后随着训练数据进行变化。最早在 Facebook 论文^[93]中应用这个方法，在模型 BERT、GPT 等中可以看到这种编码方式。但也有局限性：限制扩展性，在设置好训练长度 1024 后，只能处理 1024 长度的单词，超过该长度没有位置信息匹配，并且不考虑相对位置关系。

4.2.3 相对位置信息编码

之后，改进绝对位置编码方式，引入三角函数概念，提出 Sinusoidal 位置编码，基于三角函数的编码方式。在文献[94]中提出计算公式：

$$p_{k,2i} = \sin(k / 10000^{2i/d}) \quad (4.1)$$

$$p_{k,2i+1} = \cos(k / 10000^{2i/d}) \quad (4.2)$$

位置顺序用 k 表示，位置向量的维度用 d 表示， $p_{k,2i}$ 说明 k 位置编码的第 $2i$ 个分量。采用三角函数周期变化特点，分别用正弦函数表示奇数维度，余弦函数表示偶数维度。位置信息是由三角函数周期性质决定的，通过周期变化得到相应的奇数位置和偶数位置的信息。三角函数公式如下：

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta \quad (4.3)$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta \quad (4.4)$$

对于位置 $p(k + j)$ 计算公式如下：

$$\begin{aligned} PE(k + j, 2i) &= \sin((k + j) / 10000^{2i/d}) \\ &= \sin(k / 10000^{2i/d}) \cos(j / 10000^{2i/d}) + \\ &\quad \cos(k / 10000^{2i/d}) \sin(j / 10000^{2i/d}) \\ &= PE(k, 2i) \cos(j / 10000^{2i/d}) \\ &\quad + PE(k, 2i + 1) \sin(j / 10000^{2i/d}) \end{aligned} \quad (4.5)$$

$$\begin{aligned} PE(k + j, 2i + 1) &= \cos((k + j) / 10000^{2i/d}) \\ &= \cos(k / 10000^{2i/d}) \cos(j / 10000^{2i/d}) - \\ &\quad \sin(k / 10000^{2i/d}) \sin(j / 10000^{2i/d}) \\ &= PE(k, 2i + 1) \cos(j / 10000^{2i/d}) \\ &\quad - PE(k, 2i) \sin(j / 10000^{2i/d}) \end{aligned} \quad (4.6)$$

4.2.4 基于相对距离的位置信息编码

本文在函数级别进行代码漏洞检测，研究可以发现，代码漏洞出现位置相对有规律，基本上在敏感函数上下文，漏洞与敏感函数距离相对较近，出现跨越多行的概率较低。因此，在编码时应考虑位置的相对距离，对相对距离小的位置编码密集，加重注意力权重，使模型更关注该部分代码特征。另一方面，对于普通文本，词语出现的顺序不同，展现出不同的语义信息，但在代码编写方面，不存在这个问题，只要实现的功能相同，个人可根据自己的编码风格进行发挥，在位置编码方面，希望可以模糊代码顺序出现的不同带来的差异，并保持代码之间的语义信息。

通过以上两方面的考虑，在基于相对位置编码方式上，采用基于相对距离的位置编码方式来灵活的捕获语义特征和距离特征。使用基于相对距离的位置编码方式对代码中的每个单词进行位置编码，以可能含有漏洞的函数为中心，将位置记为 p ，与之相关的词汇位置记为 m ，先计算每个词汇到库函数的相对位置间的距离，也就是位置相似度计算，如公式 4.7 所示，距离越小，相似度越高。

$$d = \sqrt{(p - m)^2} \quad (4.7)$$

之后，对相对距离取对数，并向上取整，得到词汇的位置编码，采用公式 4.8 计算。

$$\begin{aligned} pos &= [\log_2 d] + 1, p < m \\ pos &= -[\log_2 d] + 1, m < p \end{aligned} \quad (4.8)$$

举例说明在库函数附近的词汇位置编码过程，假设存在一个带漏洞的库函数用 $k5$ 表示，此位置设为 $pos=0$ ，将其上下文关系表示出来，如图 4.4 所示。采用公式 4.8 对每个单词位置编码进行计算，得到结果如表 4.3 所示。根据以上计算，可以看出，本章提出的编码方式，将位置信息加入到注意力机制模型中，远近不同位置的注意力机制分数也不同，对模型的影响也不同，同时，在距离上与敏感函数距离短的词汇位置编码相对密集，注意力分数会更高，此部分对模型的影响较大。

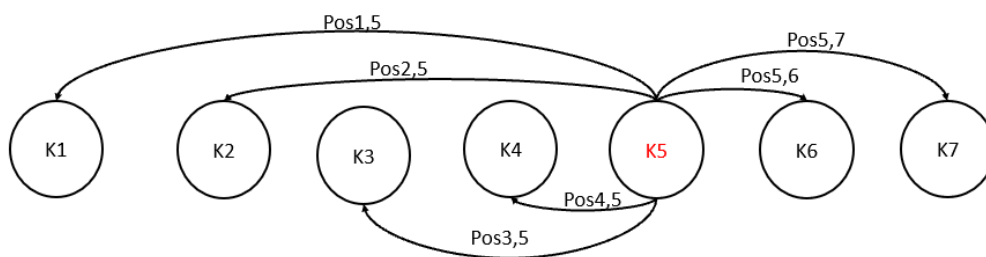


图 4.4 代码切片中位置关系

Fig. 4.4 Positional relationship in code slice

表 4.3 敏感函数上下文的位置编码

Table 4.3 Location encoding of the sensitive function context

单词位置	位置编码
pos1,5	$-(\lfloor \log \sqrt{(5-1)^2+1} \rfloor) = -3$
pos2,5	$-(\lfloor \log \sqrt{(5-2)^2+1} \rfloor) = -2$
pos3,5	$-(\lfloor \log \sqrt{(5-3)^2+1} \rfloor) = -2$
pos4,5	$-(\lfloor \log \sqrt{(5-4)^2+1} \rfloor) = -1$
pos5,6	$(\lfloor \log \sqrt{(6-5)^2+1} \rfloor) = 1$
pos5,7	$(\lfloor \log \sqrt{(7-5)^2+1} \rfloor) = 2$

把公式 4.8 代入到 4.1 和 4.2 可以得到位置编码向量：

$$PE(pos, 2i) = \sin(\lfloor \log_2 \sqrt{(p-m)^2} \rfloor + 1) / 10000^{2i/d}, m > p$$

$$PE(pos, 2i) = \sin(-(\lfloor \log_2 \sqrt{(p-m)^2} \rfloor + 1) / 10000^{2i/d}), m < p \quad (4.9)$$

$$PE(pos, 2i+1) = \cos(\lfloor \log_2 \sqrt{(p-m)^2} \rfloor + 1) / 10000^{2i/d}, m > p$$

$$PE(pos, 2i+1) = \cos(-(\lfloor \log_2 \sqrt{(p-m)^2} \rfloor + 1) / 10000^{2i/d}), m < p \quad (4.10)$$

通过以上方式处理，获得了基于相对距离的词位置编码向量，之后，将位置

信息与输入向量 x_i 一起输入到多头注意力机制中进行学习，对不同位置进行注意力权重分配，相对距离小的位置注意力权重大，计算公式如下所示：

$$q_i = (x_i + PE)W_Q \quad (4.11)$$

$$k_i = (x_i + PE)W_K \quad (4.12)$$

$$v_i = (x_i + PE)W_V \quad (4.13)$$

x_i 是输入行向量， $q_i k_j^T$ 计算如下：

$$\begin{aligned} q_i k_j^T &= (x_i + p_i)W_Q W_K^T (x_j + p_j)^T \\ &= (x_i W_Q + p_i W_Q)(W_K^T x_j^T + W_K^T p_j^T) \end{aligned} \quad (4.14)$$

得到加入注意机制的输出向量：

$$a_{i,j} = \text{soft max}(q_i k_j^T) \quad (4.15)$$

公式 4.9 和 4.10 说明在一定距离内，可以模糊词之间的顺序带来的影响，并且，距离敏感函数相对距离较小的词汇，位置编码更密集，这种编码方式模糊距离不同，词汇带来的不同影响，此编码方式可以灵活的捕获语义特征和距离特征。

4.3 实验结果及分析

4.3.1 实验设置

在 SARD 和 NVD 上广泛采集 C/C++ 相关代码，通过数据处理，最终得到数据集：17725 个含漏洞的代码切片，43913 个不包含漏洞的代码切片。其中，10440 个包含缓冲区错误漏洞（CWE119），7285 个包含资源管理错误漏洞（CWE399）。在这些数据中，随机抽取 80% 的数据用作训练集，20% 的数据用作测试集。

本章实验使用 Pytorch 及 Keras 等开源框架搭建网络，GPU 使用 NVIDIA GTX1080，CPU 使用 Inter i7。Adam 优化器在计算时效率高且内存需求小，参数更新不受梯度影响且自动调节学习率。因此在训练阶段，使用 Adam 优化器，学习率设置 0.002。模型采用 ReLU 激活函数，Dropout 设置 0.5，损失函数采用交叉熵损失函数。

4.3.2 实验评价指标

本节实验评价指标依旧是评估漏洞检测系统的相关指标，主要关注准确率（Accuracy）、召回率（TPR）、精确率（P）、F1-score 和二分类模型评价指标 AUC。具体计算方法在第三章介绍，不再赘述。F1-score 是精确率和召回率的调和平均值，能体现模型的检测能力，本章通过 F1-score 评价模型的检测能力，F1-score 越高，认为模型的检测能力越好。其次，在深度学习算法中，时间开销也是一重大研究问题。代码漏洞检测中的时间开销主要分为模型训练时间和检测时间，本章主要对注意力机制的网络进行改进，采用不同注意力机制以及不同的位置编码算法对代码漏洞检测模型进行改进，因此，将模型训练时间作为主要开销时间。本章通过对比时间开销评价算法性能，时间开销越小，相关的算法在模型上的性能越好。

4.3.3 实验结果分析

本小节主要对模型中的注意力机制进行验证以及对 Transformer 编码器中的位置编码算法进行验证。实验主要讨论三个问题：

问题一：探讨注意力机制的消融实验，分别用 BiLSTM 网络、Transformer 编码器搭建代码漏洞检测模型，与两模块融合搭建的注意力机制网络 TF-BiLSTM 进行对比；问题二：探究含有位置编码模块的注意力机制在代码漏洞检测领域的影响，在 BiLSTM 网络中，将自注意力机制、多头自注意力机制、含有位置编码的 Transformer 编码器模块加入其中，分析不同模型在代码漏洞检测领域的性能；问题三：探究不同位置编码方式对代码漏洞检测模型性能影响，在 Transformer 编码器的位置编码模块，分别将四种编码算法加入到模型中，分析模型漏洞检测能力。

（1）注意力机制的消融实验

代码漏洞检测领域，注意力机制提高代码特征信息提取，通过注意力消融实验验证本文提出的注意力机制模型在代码漏洞检测领域是有效的。首先，将源代码通过程序切片算法转换成含有标签的代码片段。之后通过三个网络结构进行代码漏洞特征学习，再进行分类。在数据集 CWE119 和数据集 CWE399 上进行代码漏洞检测，得到模型评价指标，如表 4.4、表 4.5 所示。

表 4.4 数据 CWE119 的实验结果

Table 4.4 Experimental results for data CWE119

网络结构	准确率	召回率	精确率	F1-score	AUC
BiLSTM	94.4%	81.6%	87.5%	86.6%	84.8%
Transformer 编码器	94.5%	81.6%	85.9%	86.0%	84.5%
TF-BiLSTM	93.0%	83.7%	88.0%	87.4%	88.0%

表 4.5 数据 CWE399 的实验结果

Table 4.5 Experimental results for data CWE399

网络结构	准确率	召回率	精确率	F1-score	AUC
BiLSTM	96.1%	89.5%	92.7%	92.4%	91.3%
Transformer 编码器	96.2%	88.9%	91.7%	91.4%	91.0%
TF-BiLSTM	97.0%	91.1%	94.0%	93.7%	93.0%

之后统计三个模型的训练时间开销，在图 4.5 中展示出不同网络结构模型训练时间开销以及对比结果。

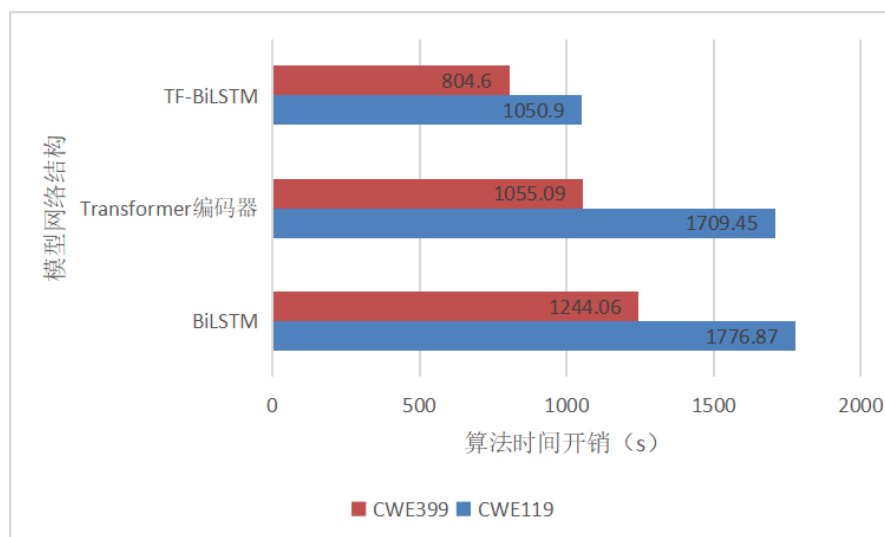


图 4.5 注意力消融实验时间开销对比图

Fig 4.5 Comparison of the time overhead of the attention ablation experiment

观察表 4.5 和 4.6 可知，单独使用 Transformer 编码器进行代码特征提取，之后通过 softmax 分类器得到代码漏洞检测结果与使用双向 LSTM 网络进行特征提取

得到检测结果相当，模型评价指标 F1-score 和 AUC 相差较小。将两模块结合的 TF-BiLSTM 网络的评价指标 F1-score 和 AUC 较两模块单独使用有所提高。在数据集 CWE119 上 F1-score 达到 87.4%，AUC 达到 88%；在数据集 CWE399 上 F1-score 达到 93.7%，AUC 达到 93.0%。说明加入注意力机制的 BiLSTM 网络可以提高代码漏洞模型的性能，最明显的效果是在时间开销方面，加入注意力机制的模型训练时间大幅度缩减，注意机制更关注于有效的代码漏洞特征，加速网络的收敛，降低模型训练时间，因此加入注意机制的 BiLSTM 网络在代码漏洞检测领域比单独使用 BiLSTM 网络更有效。

（2）位置编码对注意力机制的影响

本小节搭建了基于自注意力机制的代码漏洞检测模型、基于多头自注意力机制的代码漏洞检测模型以及基于Transformer编码器的代码漏洞检测模型，使用数据集在三个模型上进行学习和检测代码漏洞，验证封装了位置编码、多头注意力机制的Transformer编码器比其他注意力机制更适用于代码漏洞检测领域。

在训练过程中，数据每读取一次，计算为一个Epoch。通过训练次数不断迭代，模型Loss值逐渐下降，Epoch为50时，趋向于拟合，同时模型的准确率随着训练次数的增加不断提高，最终接近最大值。此时，训练模型达到最优，得到三个模型的准确率、召回率、精确率、F1-score和AUC值，如表4.6和表4.7所示。

表 4.6 数据 CWE119 的实验结果

Table 4.6 Experimental results for data CWE119

Model	准确率	召回率	精确率	F1-score	AUC
Self-Attention+ BiLSTM	86.2%	97.3%	76.3%	85.5%	87.4%
MultiHead Self-Attention+ BiLSTM	87.9%	94.4%	83.5%	88.6%	87.3%
Transformer 编码器+ BiLSTM	88.1%	93.4%	84.1%	88.9%	88.0%

表 4.7 数据 CWE399 的实验结果

Table 4.7 Experimental results for data CWE399

Model	准确率	召回率	精确率	F1-score	AUC
Self-Attention+ BiLSTM	93.4%	96.5%	91.0%	93.7%	92.3%
MultiHead Self-Attention+ BiLSTM	93.4%	96.7%	90.8%	93.6%	92.4%
Transformer 编码器+ BiLSTM	93.9%	97.0%	90.3%	93.9%	93.0%

在数据集 CWE119 上, 加入位置信息的注意力机制网络模型指标 F1-score 高于其他网络, 同样, 在数据集 CWE399 上, 也是加入位置信息的注意力机制网络模型指标 F1-score 高于其他网络, F1-score 指标作为召回率和精确率的调和平均值, 可以很好的评价模型的性能, 实验结果说明在代码漏洞检测领域, 位置信息对模型性能有影响, 并且加入位置信息可以提高代码漏洞检测模型的准确率 F1-score。并且, 在表 4.6 和表 4.7 中可以看到, 使用多头自注意力网络的检测模型比使用自注意力网络的检测模型的性能更好, 在代码漏洞检测上准确率、召回率、精确率和 F1-score 都比较高, 这是因为, 使用自注意力机制模型在对当前位置的信息进行编码时, 会过度的将注意力集中于自身的位置, 而多头自注意力机制避免这个情况, 可以关注到上下文信息。在图 4.6 和图 4.7 中, 可以直观看到模型的对比实验结果。

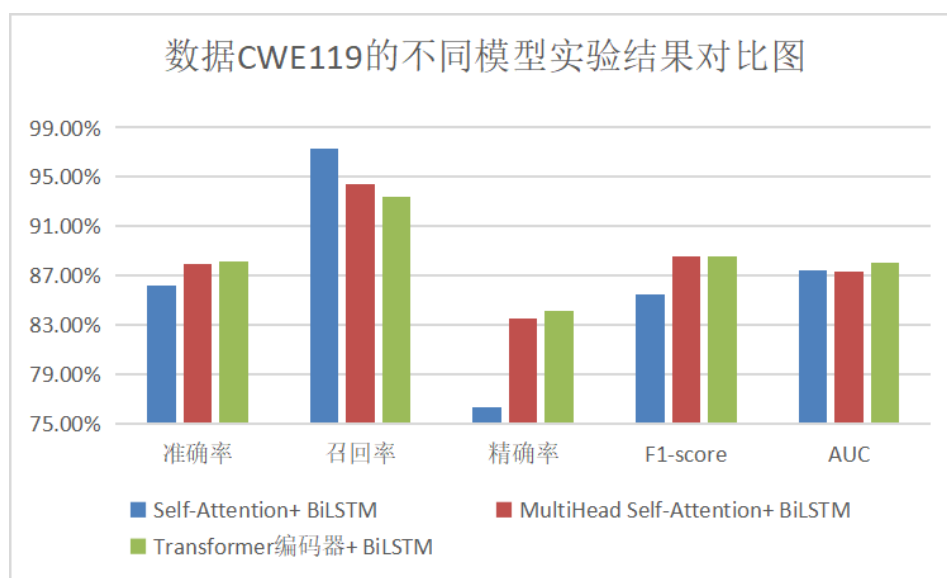


图 4.6 数据 CWE119 的不同模型实验结果对比图

Fig 4.6 Comparison of experimental results of different models for data CWE119

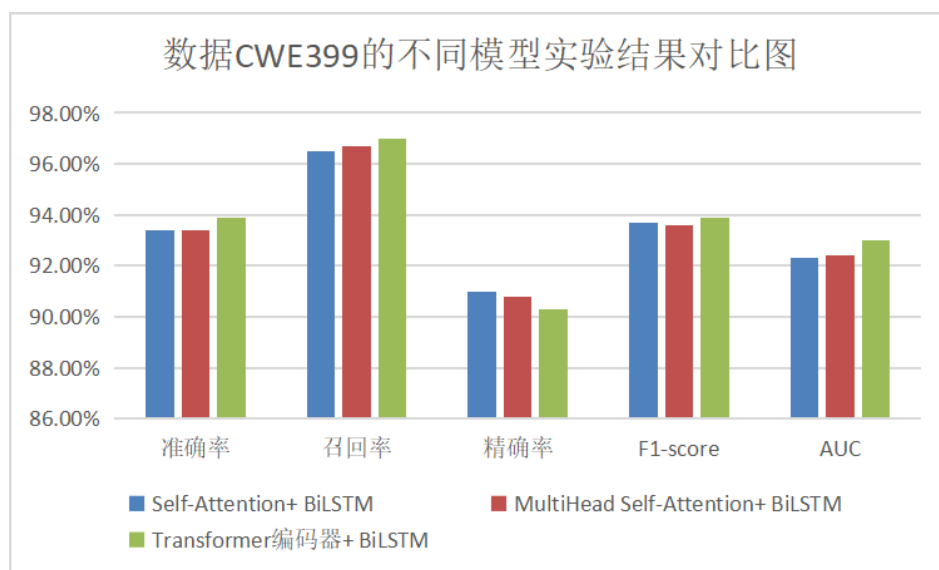


图 4.7 数据 CWE399 的不同模型实验结果对比图

Fig 4.7 Comparison of experimental results of different models for data CWE399

为了更加直观的验证模型的有效性，在图 4.8 中画出三个模型在数 CWE119 和数据 CWE399 上随着训练次数的增加，F1-score 值的变化趋势。可以看到，加入位置编码注意力机制模型的 F1-score 比其他模型更高一点。说明位置编码加强注意力机制提取代码漏洞特征，进而提升漏洞检测系统的性能，该模型更适合代码漏洞检测领域。

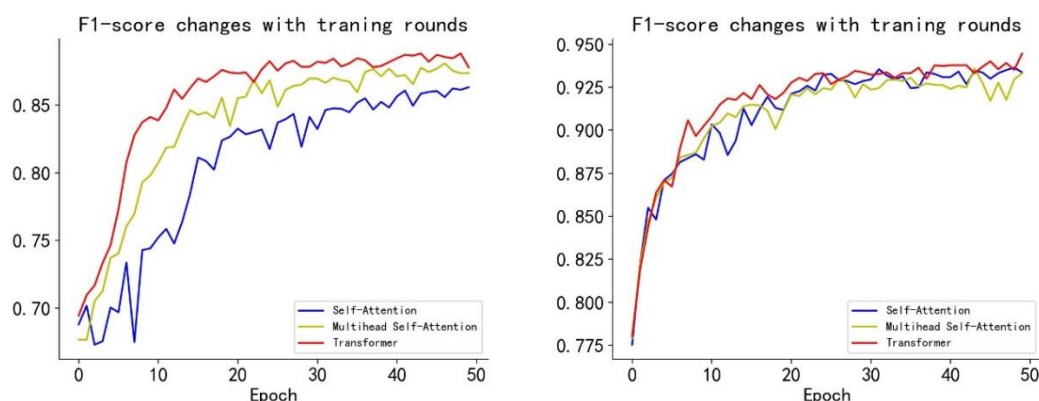


图 4.8 不同模型的 F1-score 曲线

Fig 4.8 F1-score curves of different models

之后，统计三个模型训练时间，在图 4.9 中，将不同注意力机制模型训练时间开销以及对比结果表示出来，

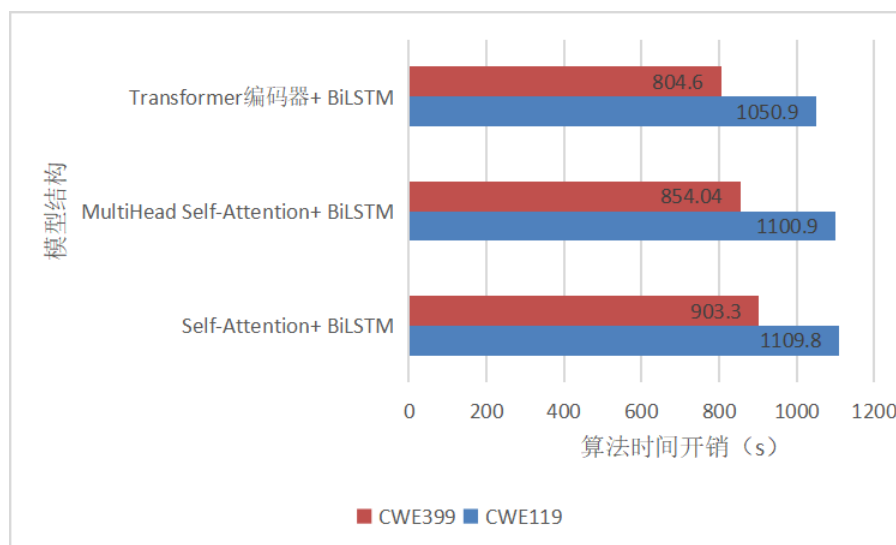


图 4.9 不同注意力机制模型训练时间开销对比图

Fig 4.9 Comparison of the training time overhead of different attention mechanism models

可以观察到, 基于 Transformer 编码器的注意力机制网络在时间开销方面虽然少于其它模型, 但差距不明显, 这是因为注意力机制都会加快网络收敛速度, 降低模型的训练时间, 但 Transformer 编码器中含有位置编码信息, 加重对敏感函数附近代码行权重分配, 以及含有残差连接结构, 可以避免代码特征丢失, 进而在模型分类指标 AUC 值和 F1-score 方面比其他模型更高, 因此, 位置编码加强注意力机制提取代码漏洞特征, 进而提升漏洞检测系统的性能, 基于 Transformer 编码器的注意力机制网络模型更适合代码漏洞检测领域。

(3) 不同位置编码方式对漏洞挖掘系统性能影响

实验使用基于注意力机制的代码漏洞检测模型, 在数据集上通过 Transformer 编码器提取代码漏洞的全局特征, 之后输入 BiLSTM 网络学习和分类, 得到模型的最优参数。对 Transformer 编码器中的 Positional Encoding 分别用 One-hot 编码、绝对位置编码、相对位置编码、相对距离位置编码方式对输入的向量位置信息进行表示, 将单词的位置信息加入到注意力学习中, 提高注意力机制对特征提取的能力, 进而提高模型的性能。

首先对数据集缓冲区漏洞 (CWE119) 进行网络训练。在训练过程中, 数据每读取一次, 计算为一个 Epoch。通过训练次数不断迭代, 模型 Loss 值逐渐下降, Epoch 为 50 时, 趋向于拟合, 同时模型的准确率和 F1-score 随着训练次数的增加不断提高, 在训练次数 50 次时, 接近最大值。此时, 训练模型达到最优, 得到模

型的准确率（Accuracy）、召回率（TPR）、精确率（P）、F1-score 和 AUC 值，如表 4.8 所示。

表 4.8 不同位置编码的实验结果（CWE119）

Table 4.8 Experimental results of different position coding (CWE119)

位置编码	F1-score	准确率	召回率	精确率	AUC
One-hot 编码	87.5%	86.5%	94.3%	81.6%	80.0%
绝对位置编码	88.0%	87.4%	92.9%	83.6%	81.0%
相对位置编码	88.5%	87.9%	93.4%	84.1%	88.0%
相对距离位置编码	89.5%	89.0%	94.3%	85.2%	88.2%

在表 4.8 中可以看到，基于相对距离的位置编码方式在模型评价指标上都有提高，准确率达到 89.0%，召回率 94.3%，模型 F1-score 值达到 89.5%。相对于 One-hot 编码方式，F1-score 提高了 2%；相比于绝对位置编码，F1-score 提高了 1.5%；相比于相对位置编码，F1-score 提高了 1%。

之后，对数据集资源管理错误（CWE399）进行网络训练，在模型达到最优时，用测试数据得到准确率（Accuracy）、召回率（TPR）、精确率（P）、F1-score 和 AUC 值，如表 4.9 所示。

表 4.9 不同位置编码的实验结果（CWE399）

Table 4.9 Experimental results of different position coding (CWE399)

位置编码	F1-score	准确率	召回率	精确率	AUC
One-hot 编码	92.6%	92.3%	96.1%	89.5%	90.0%
绝对位置编码	93.9%	93.7%	92.9%	91.1%	91.0%
相对位置编码	93.6%	93.3%	97.0%	90.3%	93.1%
相对距离位置编码	94.2%	94.1%	95.6%	92.9%	94.1%

在表 4.9 中可以看到，基于相对距离的位置编码方式在模型评价指标上都有提高，准确率达到 95.5%，召回率 97.2%，模型 F1-score 值达到 96.7%。相对于 One-hot 编码方式，F1-score 提高了 1.3%；相比于绝对位置编码，F1-score 提高了 0.3%；相比于相对位置编码，准确率提高了 0.6%。

为直观的对比不同编码方式对代码漏洞检测模型的影响，在图 4.10 和图 4.11 中画出不同模型的实验结果对比图。

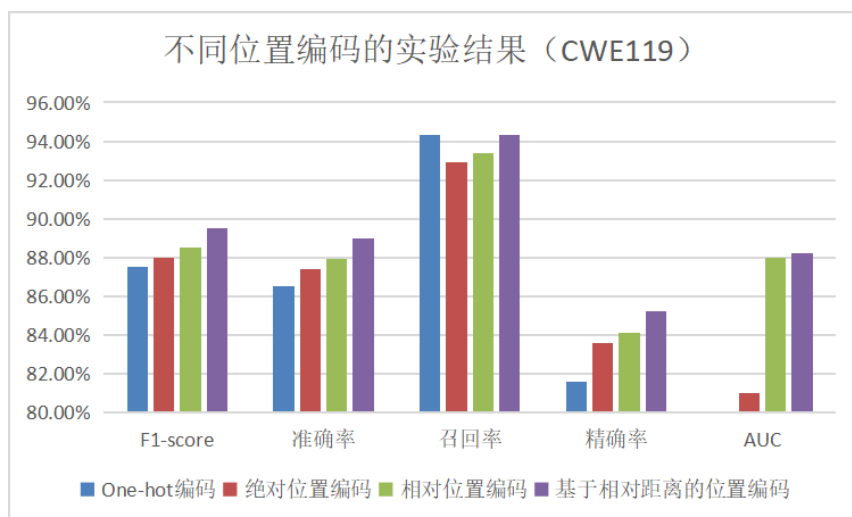


图 4.10 不同位置编码实验结果对比图 (CWE119)

Fig 4.10 Comparison of experimental results of different position encoding (CWE119)

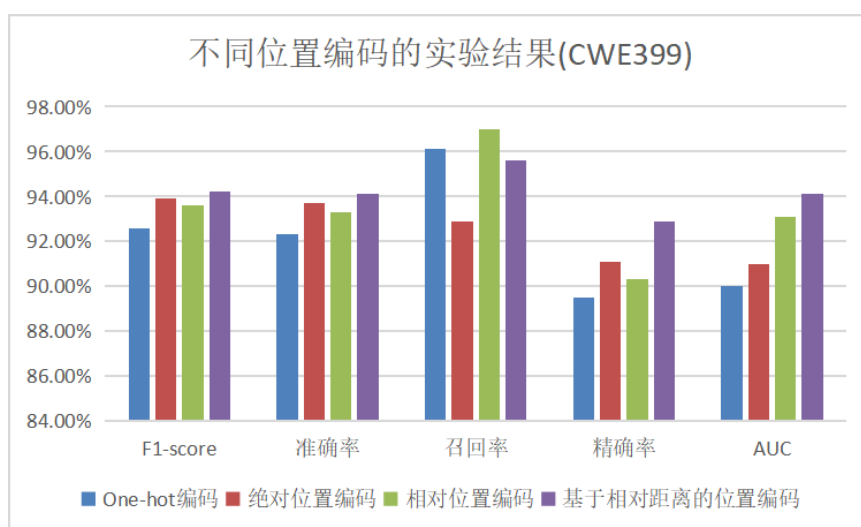


图 4.11 不同位置编码实验结果对比图 (CWE399)

Fig 4.11 Comparison of experimental results of different position coding (CWE399)

在图 4.10 和图 4.11 中，可以看到使用相对距离编码方式的漏洞检测模型准确率、召回率、精确度、F1-score 和 AUC 值相对其他网络都有所升高，说明基于相对距离的位置编码方式适合应用在基于注意力机制的代码漏洞检测模型中。

为了观察代码漏洞检测系统的性能变化情况，画出两个数据集在不同编码方式的模型上的模型评价指标 F1-score 的变化情况，如图 4.12 所示。其中，F1_1 表示模型使用 One-hot 编码方式的 F1-score；F1_2 表示模型使用绝对位置编码方式的 F1-score；F1_3 表示模型使用相对位置编码方式的 F1-score；F1_4 表示模型使用相对距离位置编码方式的 F1-score。

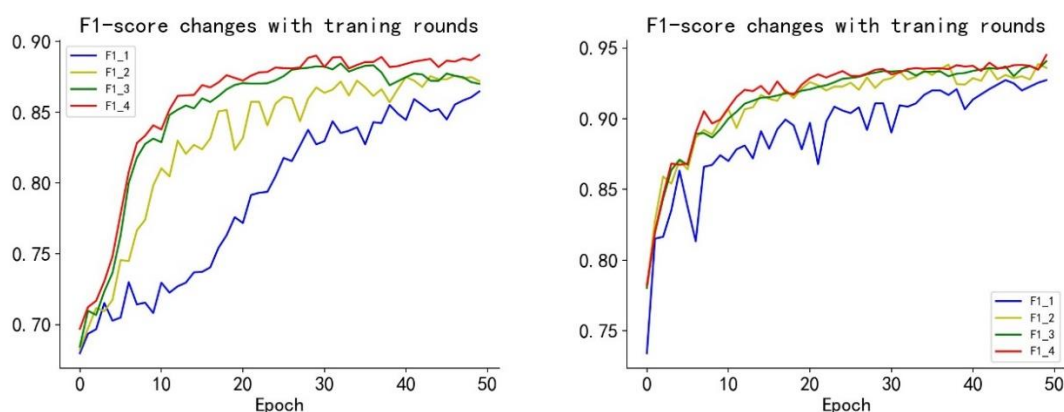


图 4.12 不同位置编码方式的 F1-score 曲线

Fig 4.12 F1-score curves for different position coding methods

在图 4.12 中，可以看到不同位置编码方式对漏洞检测模型的性能是有影响的，并且对比其他方式，基于相对距离位置编码的注意力机制模型有更高的检测能力，因此说明基于相对距离的编码方式适用于代码漏洞检测模型，表现在相对较高的准确率、召回率和 F1-score，同时也说明，使用相对距离位置编码方式进行位置编码，符合代码语法特点。

此外，时间开销也是深度学习需要考虑的一个重要问题，由于代码漏洞检测过程主要耗时在模型训练过程中，因此，本小节统计模型的训练时间来评价不同编码算法对代码漏洞检测模型的性能影响。两个数据集在不同编码方式的模型上的训练时间变化情况以及对比情况，如图 4.13 所示。

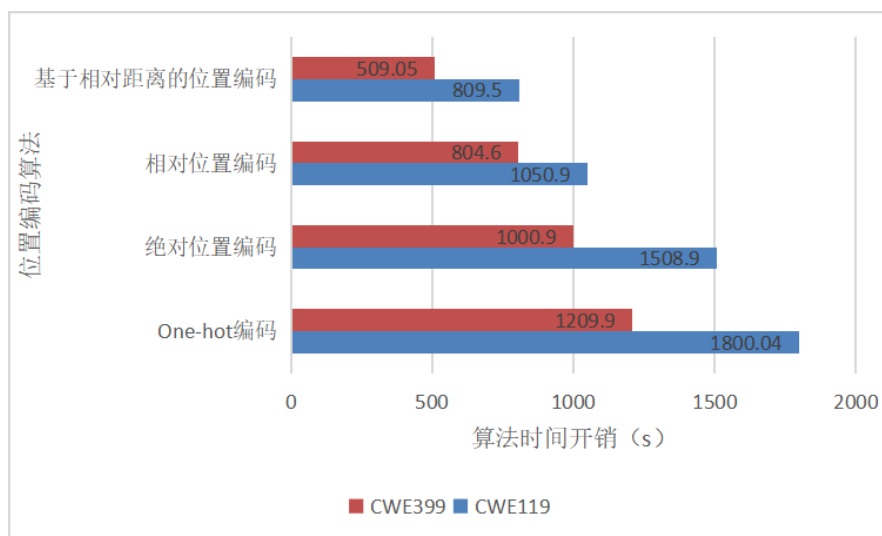


图 4.13 不同位置编码算法时间开销对比图

Fig 4.13 Comparison of the time overhead of different location coding algorithms

观察图 4.13 可知，在代码漏洞检测算法中，Transformer 位置编码对模型训练时间开销有影响，基于相对距离的编码方式将词之间的相对距离加入到注意力机制中，加重敏感函数周围词的注意力权重，降低模型训练的时间。在算法时间开销方面，基于相对距离编码方式的注意机制更适合代码漏洞检测模型。另外，由于数据集 CWE119 包含的代码切片多于数据集 CWE399，因此在数据集 CWE119 上模型训练时间较长。

基于相对距离的位置编码方式对于位置的远近做出不同的处理，距离敏感词汇较近的词汇编码较密集，使模型更关注于这些词汇，在模型训练时此部分的权重较大；相反，在距离敏感词汇较远的词汇，编码趋于稀疏，降低注意力机制的权重。通过实验证明，在代码漏洞检测系统中，基于相对距离的位置编码方式使模型漏洞检测性能更高，此编码方式更适合代码漏洞检测系统。

4.4 本章小结

本章主要探究位置编码对代码漏洞检测系统的影响。首先，用 Self-Attention 机制、MultiHead Self-Attention 机制、Transformer 编码器分别搭建基于注意力机制的网络，之后在数据集上进行实验，实验表示采用 Transformer 编码器搭建的注意力机制模型检测漏洞的准确率和 F1-score 较高，并且模型训练时间开销较小，说明含有位置编码的注意力机制模型更适合代码漏洞检测领域。之后对 Transformer

编码器中的 Positional Encoding 模块采用不同的编码方式进行代码漏洞检测，在数据集上进行试验，得出相应的准确率、召回率、精度、F1-score 和 AUC 值。通过比较模型评价参数 F1-score 的大小，基于相对距离位置编码方式的模型 F1-score 高于其他方式，之后统计模型训练时间，基于相对距离位置编码方式的模型训练时间开销小于其它模型，因此基于相对距离的位置编码方式更适用于漏洞检测系统。

第5章 总结和展望

5.1 总结

互联网技术的快速发展,越来越多的软件程序涌现出来,满足用户多种多样的需求。在代码复杂化的过程中,暴露出的漏洞也越来越多,严重危害用户使用安全,因此,代码漏洞检测技术越来越重要。早期漏洞检测解决方案需要专家对漏洞进行分析,制定相应的检测规则。该方法在准确率上有较好的表现,但由于需要人工制定漏洞检测规则,检测到的漏洞较单一,且受专家经验的限制。随着机器学习算法的发展,将机器学习应用到漏洞检测,解决了需要人工制定规则这一耗时耗力问题,但也存在着需要人工提取漏洞特征的问题。传统机器学习依赖于人类专家来定义特征,这一过程过度依赖专家经验而造成主观判断,而漏洞特征的定义直接影响漏洞检测效率和准确率。随着深度学习的发展,深度学习在人工智能领域展示越来越大的优势,比如图像识别、目标检测等方面。深度学习具有处理大量软件代码和漏洞数据的强大能力,在漏洞检测领域表现出有较高的正确率,并且可以自动学习代码特征,极大提高检测效率和正确性。

基于深度学习的漏洞检测方法主要分为以下几步:首先,将源代码表示成一种中间形式,方便代码特征提取;然后,通过编码模型对特征进行矢量化;最后,通过神经网络进行学习和检测。然而,在代码特征提取方面,传统方法代码特征提取过度依赖专家经验,并且特征提取不全面,影响模型训练;在代码漏洞检测神经网络方面,存在着网络模型结构单一、网络创新较低、网络鲁棒性不强等问题。

针对以上问题,本文主要进行以下工作,引用注意力机制网络进行代码全局特征提取;搭建基于注意力机制的代码漏洞检测系统,改进网络结构,提高系统漏洞检测能力;根据代码结构特点,改进位置编码算法,进一步提高网络检测能力。

本文主要工作如下:

(1) 数据集的构建和预处理

本文使用的数据集是在 SARD 和 NVD 上收集的,之后进行划分,并进行合理编码标识,再进行漏洞检测模型学习与训练。首先,对源代码进行冗余处理,提取与敏感函数相关的代码语句,使用程序切片思想,设计程序切片算法,在保持

语义信息下去除关键点的冗余代码。之后，采用向量转换方法，将代码转换成向量矩阵，为训练神经网络做准备。

（2）设计并实现基于注意力机制的 BiLSTM 网络

Transformer 编码器中的注意力机制可以进行全局代码特征提取，BiLSTM 可以进行双向语义学习，结合两个网络的优势，提出基于注意力机制的代码漏洞检测系统，加深网络层次，优化网络结构，并在数据集上进行相关实验，证明该模型在代码漏洞检测上是有效的，并且优于基准模型；在算法时间开销方面，该模型显著降低模型训练时间，因此该注意力机制模型更适合代码漏洞检测领域。之后，设计自注意力和多头注意力的漏洞检测模型，并与本文模型相比较，通过实验证明用 Transformer 编码器搭建的注意力机制漏洞检测模型性能更好，模型训练时间更短，说明加入位置编码的注意力机制网络更适合于漏洞检测模型。

（3）设计并实现基于相对距离位置编码的注意力代码漏洞检测模型

与普通文本不同，代码编写风格多种多样，语义相同的代码在编写风格上有差异。在代码漏洞检测中，一般漏洞出现在敏感函数附近，针对这一特性，希望距离敏感函数较近的词汇位置编码密集，注意力模型更关注这一部分信息，因此考虑敏感函数周围词的相对距离，采用基于相对距离的编码方式对位置信息编码，之后加入到注意力模型中，提出基于相对距离位置编码的注意力代码漏洞检测模型，并通过实验在模型性能和模型训练时间开销方面证明该方法的准确率、F-score、AUC 值更高，模型训练时间更短，因此该模型更适合代码漏洞检测领域。

5.2 展望

本文在代码漏洞特征提取和漏洞检测方面取得不错的结果，但仍然存在以下几点问题值的继续研究探讨。

（1）数据集缺陷

本文使用的数据集大部分来自 SARD 数据库中，虽然该数据集被行业专业人士广泛应用，并在测试过程中表现出良好的优势，但该数据集多数是被处理过后的数，与现实中真实的软件缺陷源代码还是有差距的，未来希望可以将软件源代码输入到网络中进行漏洞挖掘。此外，分析可执行文件的漏洞也是一个很大的挑战。

（2）系统扩展性

本文重点针对的程序缺陷是缓冲区溢出和资源管理错误，对于其他原因造成的代码漏洞还需进一步研究。除此之外，本文系统主要针对 C/C++ 语言，对于其他语言的漏洞挖掘需要进一步研究。

（3）细粒度漏洞位置

尽管本文检测粒度在函数级别有很好的表现，但不能准确的提供漏洞位置信息，依旧需要人工进行细粒度操作，如何更准确定位漏洞位置，是以后研究的重点。

参 考 文 献

- [1] PHAM V, DANG T. Cvexplorer: Multidimensional visualization for common vulnerabilities and exposures[C]. 2022 IEEE International Conference on Big Data. IEEE, 2022: 1296-1301.
- [2] 国家信息安全漏洞库[EB/OL]. [2022-11-17]. <http://www.cnnvd.org.cn/>
- [3] LIPP S, BANESCU S, PRETSCHNER A. An empirical study on the effectiveness of static C code analyzers for vulnerability detection[C]. Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. 2022: 544-555.
- [4] WANG J, HUANG M, NIE Y, et al. Static analysis of source code vulnerability using machine learning techniques: A survey[C]. 2021 4th International Conference on Artificial Intelligence and Big Data (ICAIBD). 2021: 76-86.
- [5] TANG H, HUANGS, LI Y, et al. Dynamic taint analysis for vulnerability exploits detection[C]. Proceedings of the 2nd International Conference on Computer Engineering and Technology, 2010, 2: 215-218.
- [6] WARTSCHINSKI L, NOLLER Y, Vogel T, et al. Vudenc: Vulnerability detection with deep learning on a natural codebase for python[J]. Information and Software Technology, 2022, 144: 106809.
- [7] FILUS K, BORYSZKO P, DOMANSKA J, et al. Efficient feature selection for static analysis vulnerability prediction[J]. Sensors, 2021, 21 (4) : 1133.
- [8] ZHENG Y, PUJAR S, LEWIS B, et al. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis[C]. 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2021: 111-120.
- [9] TANG W, TANG M, BAN M, et al. CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection[J]. Journal of Systems and Software, 2023, 199: 111623.
- [10] FEI J, CHEN X, ZHAO X. MSmart: Smart Contract Vulnerability Analysis and Improved Strategies Based on Smartcheck[J]. Applied Sciences, 2023, 13 (3) : 1733.
- [11] AMALOU I, MOUHNI N, ABDALI A. Multivariate time series prediction by RNN architectures for energy consumption forecasting[J]. Energy Reports, 2022, 8: 1084-1091.

- [12] KUMAR A, VATSA A. Influence of GFP GAN on Melanoma Classification[C]. 2022 IEEE Integrated STEM Education Conference (ISEC). IEEE, 2022: 334–339.
- [13] PENG Y, PAN X, WANG S, et al. An aero-engine RUL prediction method based on VAE-GAN[C]. 2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD). IEEE, 2021: 953–957.
- [14] ZHAO J, GUO S, MU D. DouBiGRU-A: software defect detection algorithm based on attention mechanism and double BiGRU[J]. Computers & Security, 2021, 111: 102459.
- [15] ROZI M F, OZAWA S, BAN T, et al. Understanding the Influence of AST-JS for Improving Malicious Webpage Detection[J]. Applied Sciences, 2022, 12(24): 12916.
- [16] BELLON S, KOSCHKE R, ANTONIOL G, et al. Comparison and evaluation of clone detection tools[J]. IEEE Transactions on software engineering, 2007, 33(9): 577–591.
- [17] LI L, FENG H, ZHUANG W, et al. CCLearner: A deep learning-based clone detection approach[C]. 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2017: 249–260.
- [18] WEI H, LI M. Supervised Deep Features for Software Functional Clone Detection by Exploiting Lexical and Syntactical Information in Source Code[C]. IJCAI. AAAI Press, 2017: 3034–3040.
- [19] WHITE M, TUFANO M. Deep learning code fragments for code clone detection[C]. Proceedings of the 31st IEEE/ACM international conference on automated software engineering. 2016: 87–98.
- [20] MARASTONI N, GIACOBazzi R, DALLA PREDA M. A deep learning approach to program similarity[C]. Proceedings of the 1st international workshop on machine learning and software engineering in symbiosis. 2018: 26–35.
- [21] SHENEAMER A. CCDLC detection framework-combining clustering with deep learning classification for semantic clones[C]. 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, 2018: 701–706.
- [22] SHENEAMER A, HAZAZI H, ROY S. Schemes for labeling semantic code clones using machine learning[C]. 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, 2017: 981–985.

- [23] SHALEV N, PARTUSH N. Binary similarity detection using machine learning[C]. Proceedings of the 13th Workshop on Programming Languages and Analysis for Security. 2018: 42-47.
- [24] ZHAO G, HUANG J. Deepsim: deep learning code functional similarity[C]. Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2018: 141-151.
- [25] Klocwork. Klocwork K7[EB/OL]. [2021-06-10]. <http://www.klocwork.com>.
- [26] JANG C, KIM J, JANG H, et al. Rule-based auditing system for software security assurance[C]. 2009 First International Conference on Ubiquitous and Future Networks. IEEE, 2009: 198-202.
- [27] 刘春燕. 基于规则的 C/C++代码静态检测方法研究[D]. 大连: 大连理工大学, 2010: 34-45.
- [28] SHAHRIAR H, HADDAD H M, Vaidya I. Buffer overflow patching for C and C++ programs: rule-based approach[J]. ACM SIGAPP Applied Computing Review, 2013, 13(2): 8-19.
- [29] PMD Source Code Analyzer. PMD source code[CP/OL]. [2021-06-10]. <https://pmd-d.github.io/>.
- [30] Fortify: Fortify Static Code Analyzer| Micro Focus [EB/OL]. <https://www.microfocus.com/zh-cn/products/static-code-analysis-sast/overview>.
- [31] CHERNIS B, VERMA R. Machine learning methods for software vulnerability detection[C]. Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics. 2018: 31-39.
- [32] LIN G, ZHANG J, LUO W, et al. Cross-project transfer representation learning for vulnerable function discovery[J]. IEEE Transactions on Industrial Informatics, 2018, 14(7): 3289-3297.
- [33] HARER J A, KIM L Y, RUSSELL R L, et al. Automated software vulnerability detection with machine learning[J]. arXiv preprint arXiv: 1803.04497, 2018.
- [34] NAPIER K, BHOWMIK T, WANG S. An empirical study of text-based machine learning models for vulnerability detection[J]. Empirical Software Engineering, 2023, 28(2): 38.

- [35] DE PAULA RODRIGUES G E, DAHAB R. Detecting Cryptography Misuses With Machine Learning: Graph Embeddings, Transfer Learning and Data Augmentation in Source Code Related Tasks[J]. IEEE Transactions on Reliability, 2023.
- [36] CHEN J, ZHANG C, CAI S, et al. A memory-related vulnerability detection approach based on vulnerability model with Petri Net[J]. Journal of Logical and Algebraic Methods in Programming, 2023, 132: 100859.
- [37] ASHIZAWA N, YANAI N, CRUZ J P, et al. Eth2vec: Learning contract-wide code representations for vulnerability detection on ethereum smart contracts[C]. Proceedings of the 3rd ACM International Symposium on Blockchain and Secure Critical Infrastructure. 2021, 34(01): 47–59.
- [38] ALANI M M. BotStop: Packet-based efficient and explainable IoT botnet detection using machine learning[J]. Computer Communications, 2022, 193: 53–62.
- [39] DHANYA K A, VAJIPAYAJULA S, SRINIVASAN K, et al. Detection of Network Attacks using Machine Learning and Deep Learning Models[J]. Procedia Computer Science, 2023, 218: 57–66.
- [40] CLOTTEY R N, YAOKUMAH W, APPATI J K. Modelling and Evaluation of Network Intrusion Detection Systems Using Machine Learning Techniques[J]. International Journal of Intelligent Information Technologies (IJIIT), 2021, 17(4): 81–99.
- [41] LI Z, ZOU D, XU S, et al. VulDeePecker: A deep learning-based system for vulnerability detection[J]. NDSS, 2018: 23–26.
- [42] LI Z, ZOU D, XU S, et al. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(4): 2244–2258.
- [43] ZOU D, WANG S, XU S, et al. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection[J]. IEEE Transactions on Dependable and Secure Computing, 2019, 18(5): 2224–2236.
- [44] WANG H, YE G, TANG Z, et al. Combining graph-based learning with automated data collection for code vulnerability detection[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 1943–1958.

- [45] ZHENG W, GAO J, WU X, et al. The impact factors on the performance of machine learning-based vulnerability detection: A comparative study[J]. Journal of Systems and Software, 2020, 168: 110659.
- [46] HUO X, LI M, ZHOU Z H. Control flow graph embedding based on multi-instance decomposition for bug localization[C]. Proceedings of the AAAI conference on artificial intelligence. 2020, 34(04): 4223–4230.
- [47] 段旭, 吴敬征, 罗天悦, 等. 基于代码属性图及注意力双向 LSTM 的漏洞挖掘方法[J]. 软件学报, 2020, 31(11): 3404–3420.
- [48] SUN H, CUI L, LI L, et al. VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches[J]. Computers & Security, 2021, 110: 102417.
- [49] CHAKRABORTY S, KRISHNA R, DING Y, et al. Deep learning based vulnerability detection: Are we there yet[J]. IEEE Transactions on Software Engineering, 2021.
- [50] MAZUERA-ROZO A, MOJICA-HANKE A, LINARES-VASQUEZ M, et al. Shallow or deep? An empirical study on detecting vulnerabilities using deep learning[C]. 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC). IEEE, 2021: 276–287.
- [51] CHENG X, WANG H, HUA J, et al. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2021, 30(3): 1–33.
- [52] ZHENG W, JIANG Y, SU X. Vu1SPG: Vulnerability detection based on slice property graph representation learning[C]. 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2021: 457–467.
- [53] LI Y, WANG S, NGUYEN T N. Vulnerability detection with fine-grained interpretations[C]. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 292–303.
- [54] WU Y, LU J, ZHANG Y, et al. Vulnerability detection in C/C++ source code with graph representation learning[C]. 2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2021: 1519–1524.
- [55] NGUYEN V, LE T, DE VEL O, et al. Information-theoretic source code vulnerability highlighting[C]. 2021 International Joint Conference on Neural Networks (IJCNN). IEEE, 2021: 1–8.

- [56] 许健, 陈平华, 熊建斌. 融合滑动窗口和哈希函数的代码漏洞检测模型[J]. 计算机应用研究, 2021, 38(08): 2394-2400.
- [57] 文敏, 王荣存, 姜淑娟. 基于关系图卷积网络的源代码漏洞检测[J]. 计算机应用, 2022, 42(06): 1814-1821.
- [58] LIU Z, FANG Y, HUANG C, et al. GraphXSS: an efficient XSS payload detection approach based on graph convolutional network[J]. Computers & Security, 2022, 114: 102597.
- [59] GUO W, FANG Y, HUANG C, et al. HyVulDect: A hybrid semantic vulnerability mining system based on Graph Neural Network[J]. Computers & Security, 2022: 102823.
- [60] CHENG X, ZHANG G, WANG H, et al. Path-sensitive code embedding via contrastive learning for software vulnerability detection[C]. Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. 2022: 519-531.
- [61] DING Y, SUNEJA S, ZHENG Y, et al. VELVET: a noVel Ensemble Learning approach to automatically locate VulnErable sTatements[C]. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, 2022: 959-970.
- [62] LI R, CHEN B, ZHANG F, et al. Detecting runtime exceptions by deep code representation learning with attention-based graph neural networks[C]. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, 2022: 373-384.
- [63] WANG Y, JIA P, PENG X, et al. BinVulDet: Detecting vulnerability in binary program via decompiled pseudo code and BiLSTM-attention[J]. Computers & Security, 2023, 125: 103023.
- [64] HAN J, HUANG C, SUN S, et al. bjXnet: an improved bug localization model based on code property graph and attention mechanism[J]. Automated Software Engineering, 2023, 30(1): 12.
- [65] LIU Z, FANG Y, HUANG C, et al. MFXSS: An effective XSS vulnerability detection method in JavaScript based on multi-feature model[J]. Computers & Security, 2023, 124: 103015.
- [66] ZHAO Q, HUANG C, DAI L. VULDEFF: Vulnerability detection method based on function fingerprints and code differences[J]. Knowledge-Based Systems, 2023, 260: 110139.

- [67] GUO J, WANG Z, LI H, et al. Detecting vulnerability in source code using CNN and LSTM network[J]. Soft computing, 2023, 27(2): 1131–1141.
- [68] MEI H, LIN G, FANG D, et al. Detecting vulnerabilities in IoT software: New hybrid model and comprehensive data analysis[J]. Journal of Information Security and Applications, 2023, 74: 103467.
- [69] ŞAHİN C B. Semantic-based vulnerability detection by functional connectivity of gated graph sequence neural networks[J]. Soft Computing, 2023: 1–17.
- [70] CHOI Y H, NAM J. WINE: Warning miner for improving bug finders[J]. Information Software Technology, 2023, 155: 107109.
- [71] SINGH K, GROVER S S, KUMAR R K. Cyber Security Vulnerability Detection Using Natural Language Processing[C]. 2022 IEEE World AI IoT Congress (AIIoT). IEEE, 2022: 174–178.
- [72] BRUST C A, SONNEKALB T, GRUNER B. ROMEO: A Binary Vulnerability Detection Dataset for Exploring Juliet through the Lens of Assembly Language[J]. Computers & Security, 2023: 103165.
- [73] ERDODI L, SOMMERVOLL A A, ZENNARO F M. Simulating SQL injection vulnerability exploitation using Q-learning reinforcement learning agents[J]. Journal of Information Security and Applications, 2021, 61: 102903.
- [74] LIU M, ZHANG B, CHEN W, et al. A survey of exploitation and detection methods of xss vulnerabilities[J]. IEEE Access, 2019, 7: 180–182.
- [75] RAY S. A quick review of machine learning algorithms[C]. 2019 International conference on machine learning, big data, cloud and parallel computing (COMITCon). IEEE, 2019: 35–39.
- [76] MAHESH B. Machine learning algorithms-a review[J]. International Journal of Science and Research (IJSR). [Internet], 2020, 9: 381–386.
- [77] LIU P, XU C, XIE J, et al. A CNN-based transfer learning method for leakage detection of pipeline under multiple working conditions with AE signals[J]. Process Safety and Environmental Protection, 2023, 170: 1161–1172.
- [78] DHAR P, DUTTA S, MUKHERJEE V. Cross-wavelet assisted convolution neural network (AlexNet) approach for phonocardiogram signals classification[J]. Biomedical Signal Processing and Control, 2021, 63: 102142.

- [79] SIMONYAN K, ZISSERMAN A. Very Deep Convolutional Networks for Large-Scale Image Recognition[M]. arXiv, 2015[2022-10-29]. <http://arxiv.org/abs/1409.1556>.
- [80] REN S, HE K, GIRSHICK R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(6): 11.
- [81] SHERSTIMNSKY A. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network[J]. Physica D: Nonlinear Phenomena, 2020, 404: 132306.
- [82] SUN X, FU Y, LIU T. A hybrid ACO algorithm for capacitated vehicle routing problems[C]. 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC). IEEE, 2017: 510-514.
- [83] HOCHREITER S, SCHMIDHUBER J. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735-1780.
- [84] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.
- [85] KARITA S, CHEN N, HAYASHI T, et al. A comparative study on transformer vs rnn in speech applications[C]. 2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU). IEEE, 2019: 449-456.
- [86] HORWITZ S, REPS T, BINKLEY D. Interprocedural slicing using dependence graphs[J]. ACM Transactions on Programming Languages and Systems, 1990, 12(1): 26-60.
- [87] SINHA S, HARROLD M J, Rothermel G. System-dependence-graph-based slicing of programs with arbitrary interprocedural control flow[C]. Proceedings of the 21st International Conference on Software Engineering. IEEE, 1999: 432-441.
- [88] BANA S H. work2vec: Using Language Models to Understand Wage Premia[EB/OL]. Stanford Digital Economy Lab. (2021-10-8)[2022-10-29]. <https://digitaleconomy.stanford.edu/publications/job2vec-using-language-models-to-understand-wage-premia/>.
- [89] FERSCHKE O, GUREVYCH I, RITTBERGER M. A Modular System for Predicting Quality Flaws in Wikipedia[C]. Proceedings of the Notebook For Pan at Clef 2012. 2012: 1-10.
- [90] SHIN Y, MENEELY A, WILLIAMS L, et al. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities[J]. IEEE transactions on software engineering, 2010, 37(6): 772-787.

- [91] KIM S, WOO S, LEE H, et al. Vuddy: A scalable approach for vulnerable code clone discovery[C]. 2017 IEEE Symposium on Security and Privacy (SP). IEEE, 2017: 595–614.
- [92] CAO S, SUN X, BO L, et al. Bgnn4vd: constructing bidirectional graph neural-network for vulnerability detection[J]. Information and Software Technology, 2021, 136: 106576.
- [93] GEHRING J, AULI M, GRANGIER D, et al. Convolutional sequence to sequence learning[C]. International conference on machine learning. PMLR, 2017: 1243–1252.
- [94] WU K, PENG H, CHEN M, et al. Rethinking and improving relative position encoding for vision transformer[C]. Proceedings of the IEEE/CVF International Conference on Computer Vision. 2021: 10033–10041